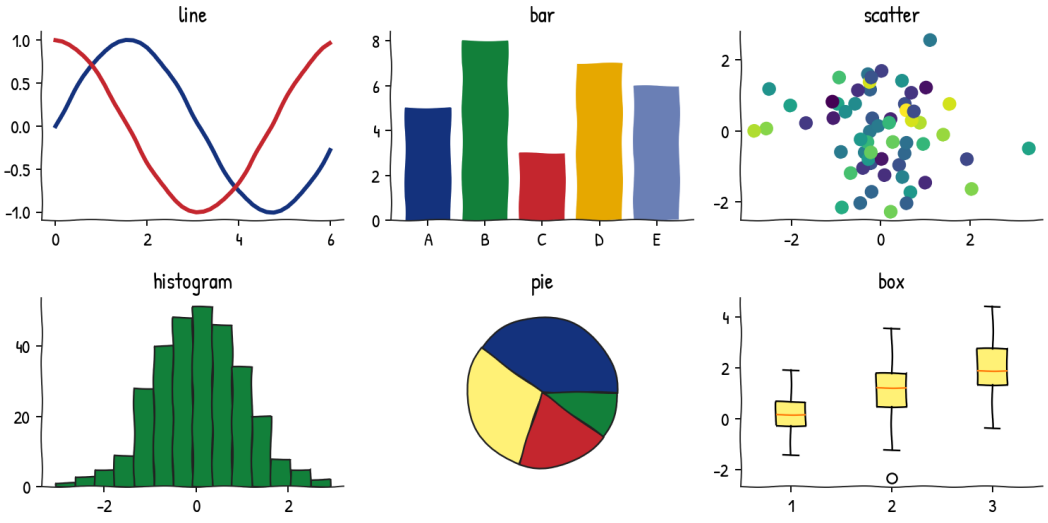
MATPLOTLIB & SEABORN

COMPLETE VISUALIZATION NOTES

ZERO TO PRO

Beginner -> Intermediate -> Advanced -> Industry-ready



Python | Matplotlib | Seaborn | Pandas | NumPy

By Chandan Chaudhari

GitHub: github.com/chandanc5525

For: college students, data analysts, ML practitioners, interview prep

How to use these notes

ink colours & the learning pattern

These notes are written like a **classroom notebook**: every page follows the same simple pattern, and the ink colour tells you what kind of information you are reading.

Ink colour code

Ink / effect	Meaning	Example
Dark blue	Headings, labels, syntax names	**Chapter titles**, **Syntax:**
Black	Normal explanations and all Python code	`plt.plot(x, y)`
Green	Important concepts, pro tips, customisation	**Figure contains Axes**
Red	Warnings, mistakes, exam tips	**Always label your axes**
Yellow highlighter	Definitions, formulas, key takeaways	Axes is the plotting area

The learning pattern used for every topic

Step	What you get
Concept	the idea in simple English
Why it matters	when and why you would use it
Syntax	the general form of the call
Example	complete, runnable Python code
Visualization	the chart that code really produces
Explanation	what the parameters and important lines do
Customization	colours, labels, titles, sizes, styles, legends
Common mistakes	frequent beginner errors and how to fix them
Pro tip	an industry-level recommendation
Practice	exercises (solutions are in Part XI)

How to run the code

- Every figure in this book was produced by running the code printed above it - nothing is mocked up.
- The figures use a hand-drawn look so they match the notebook. Your own charts will look cleaner (standard Matplotlib style) - the code is identical.
- Copy code into a Jupyter cell or a *.py* file. Run cells top-to-bottom; the imports shown in Part I are assumed everywhere.
- Tested with Python 3.12, Matplotlib 3.10, Seaborn 0.13, NumPy 2.x and Pandas 3.x. Version-dependent behaviour is flagged in ***red*** or with a NOTE box.
- All datasets are small ***synthetic*** datasets created in code (seeded for reproducibility), so you never need to download a file. *sns.load_dataset()* is shown too, but needs internet.

> ***PRO TIP:*** Do not just read - ***type*** every example yourself, change one thing, and predict the result before running it. That is the fastest way to learn plotting.

Table of Contents

everything in one place

* How to use these notes . . . 2

Table of Contents . . . 4

P1 Python & Data Visualization Foundations . . . 6

Foundations . . . 6

P2 Matplotlib Fundamentals . . . 11

1 Introduction to Matplotlib . . . 11

2 Basic Charts . . . 14

3 Figure and Axes . . . 23

4 Plot Customization . . . 27

P3 Intermediate & Advanced Matplotlib . . . 33

A Intermediate Matplotlib . . . 33

B Images, 2-D Fields, 3-D and Polar . . . 39

C Layouts, Formatting & Production . . . 43

P4 Seaborn Fundamentals . . . 50

S Seaborn Foundations . . . 50

5 Relational Plots . . . 55

6 Distribution Plots . . . 58

7 Categorical Plots . . . 63

P5 Advanced Seaborn & Statistical Visualization . . . 67

D Matrix and Grid Plots . . . 67

E Regression, Residuals and Uncertainty . . . 74

P6 Professional Chart Design . . . 79

8 Designing Charts That Communicate . . . 79

P7 Real-World Data Analysis Projects . . . 83

9 Business & Operations Projects . . . 83

10 Statistical & Modelling Projects . . . 88

P8 Matplotlib versus Seaborn . . . 93

11 Matplotlib vs Seaborn . . . 93

P9 Common Mistakes and Debugging 96

12 Debugging Visualizations 96

P10 Cheat Sheets & Revision Notes 100

13 Cheat Sheets 100

P11 Practice and Interview Preparation 104

14 Exercises 104

15 Solutions 105

16 Interview Questions 107

P12 Capstone Projects 109

17 Capstones 109

P1 Python & Data Visualization Foundations

what, why, tools, data, setup

Before drawing anything we need the big picture: what visualization is for, which libraries do what, and what kind of data you can feed them. Spend time here - everything later depends on it.

- 0.1 What is data visualization?
- 0.2 Where it is used
- 0.3 Matplotlib vs Seaborn, and the ecosystem
- 0.4 Installing and importing
- 0.5 Plot inputs: lists, arrays, Series, DataFrames
- 0.6 Data vocabulary and missing data
- 0.7 Setting up a notebook or script

0 Foundations

the ground everything else stands on

0.1 What is data visualization?

Concept: Data visualization means turning numbers into pictures (charts, plots, maps) so that patterns, trends, comparisons and outliers become visible to the human eye.

Why it matters: A table of 10,000 rows is unreadable; a single chart can show the trend in two seconds. Visualization is used both to ***explore*** data (for yourself) and to ***explain*** results (to others).

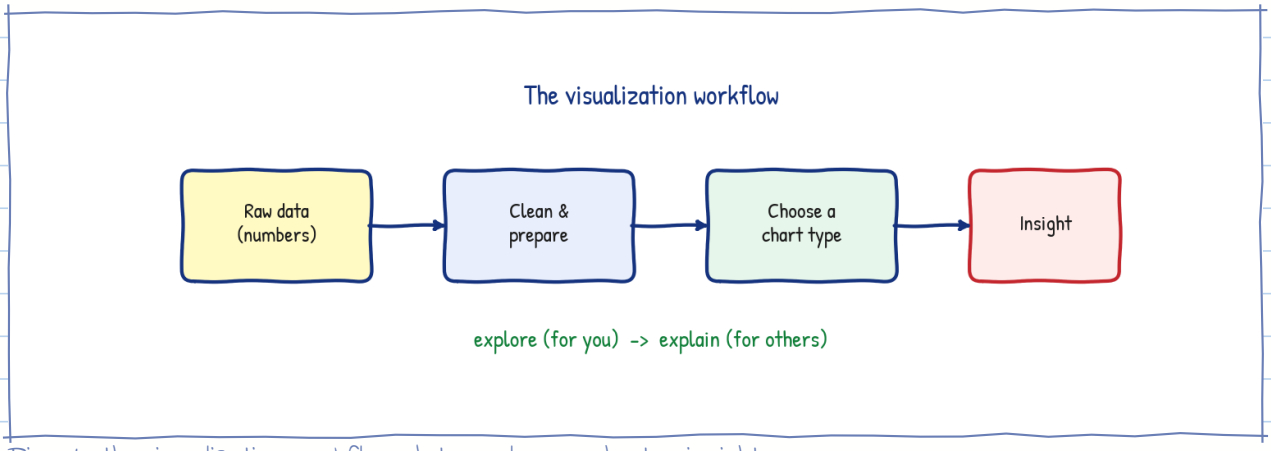


Fig 0.1 - the visualization workflow: data -> clean -> chart -> insight.

0.2 Where visualization is used

Field	Typical questions	Typical charts
Business	Which region earns most? Is revenue growing?	bar, line, stacked area, dashboard
Analytics (EDA)	What does each column look like? Any outliers?	histogram, box, pairplot, heatmap
Statistics	Is the distribution normal? Do groups differ?	KDE, ECDF, violin, Q-Q, regression plot
Machine learning	Is the data balanced? Is the model learning?	class-count bar, loss curve, confusion-matrix heatmap, residual plot
Science / research	How does y depend on x? How uncertain are we?	scatter + fit, error bars, contour

KEY IDEA: A good chart answers **one question** clearly. If you cannot say the question in one sentence, the chart is probably doing too much.

0.3 Matplotlib vs Seaborn - and how the tools fit together

Concept: **Matplotlib** is the low-level, do-anything drawing library (the foundation). **Seaborn** is built on top of Matplotlib and gives short, beautiful, statistical plots straight from Pandas DataFrames. **NumPy** supplies arrays, **Pandas** supplies tables, and **Jupyter Notebook** is where you run code and see charts inline.

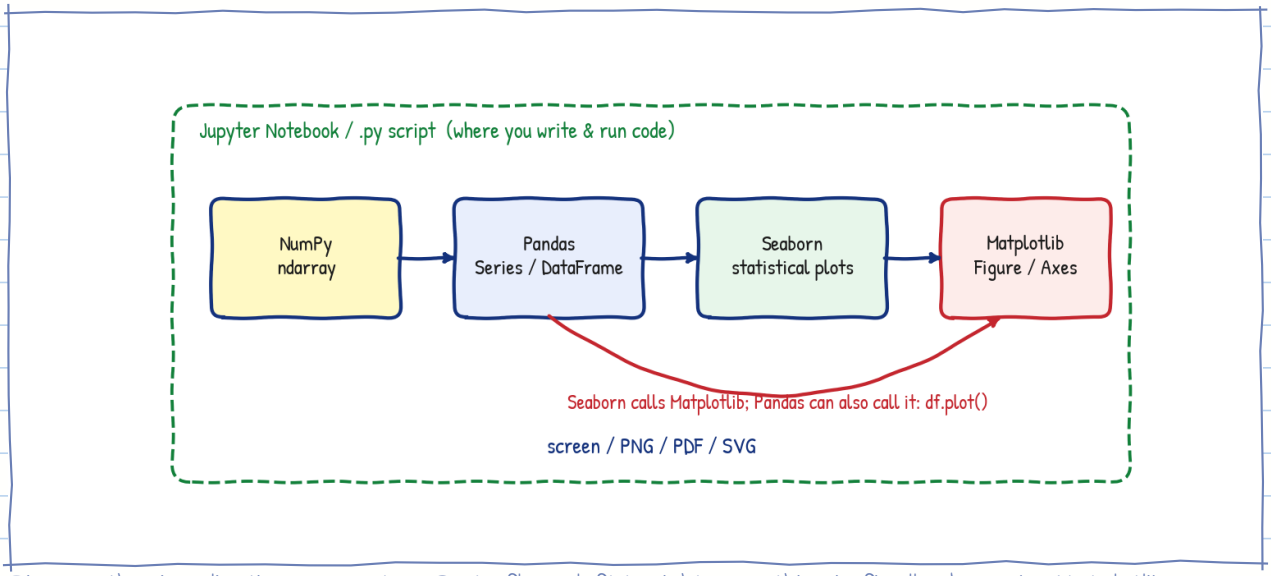


Fig 0.2 - the visualization ecosystem. Data flows left to right; everything is finally drawn by Matplotlib.

	Matplotlib	Seaborn
Level	low-level, full control	high-level, quick statistical plots
Input	lists, arrays, Series	mostly tidy Pandas DataFrames
Default look	plain (but improving)	attractive themes & palettes
Best for	custom figures, layouts, publication	EDA, distributions, group comparison

PRO TIP: Learn **Matplotlib first** (Parts II–III). Seaborn returns Matplotlib objects, so Matplotlib skills let you fine-tune every Seaborn chart.

0.4 Installing and importing

Syntax: Install once from a terminal (or a notebook cell starting with `%pip`):

```
pip install matplotlib seaborn pandas numpy jupyter
# inside a notebook cell use:   %pip install matplotlib seaborn pandas numpy
```

Example: The four standard imports - the **conventions** used by everybody:

```
import matplotlib.pyplot as plt   # plotting interface (pyplot)
import seaborn as sns             # statistical plots
import pandas as pd               # tables
import numpy as np                # arrays and maths

print(plt.matplotlib.__version__, sns.__version__, pd.__version__, np.__version__)
```

WARNING: Forgetting an import gives `NameError: name 'plt' is not defined`. Never rename the aliases (`plt`, `sns`, `pd`, `np`) - everybody's examples assume them.

0.5 Plot inputs: lists, arrays, Series and DataFrames

Concept: Plotting functions accept almost any sequence of numbers. The four common containers are below; Seaborn prefers a **DataFrame** plus column names.

```
import numpy as np, pandas as pd

months = ['Jan', 'Feb', 'Mar', 'Apr']
sales_list = [120, 135, 150, 170]                  # 1) Python list
sales_arr  = np.array(sales_list)                  # 2) NumPy array
sales_ser  = pd.Series(sales_list, index=months)   # 3) Pandas Series
df = pd.DataFrame({'month': months, 'sales': sales_list,
                   'profit': [20, 25, 31, 40]})    # 4) DataFrame
print(df)
print(df['sales'].mean(), sales_arr.max(), sales_ser['Mar'])
```

Container	Plot with	Notes
list / array	`plt.plot(x, y)`	x and y must be the same length
Series	`plt.plot(s)` or `s.plot()`	index becomes the x-axis
DataFrame (Matplotlib)	`plt.plot(df['x'], df['y'])`	pass the columns yourself
DataFrame (Seaborn)	`sns.lineplot(data=df, x='x', y='y')`	pass column **names** as strings

0.6 Data vocabulary and missing data

Term	Meaning	Example
x-values / y-values	horizontal / vertical positions	month (x), sales (y)
Numerical – continuous	any value in a range	height, price, temperature
Numerical – discrete	countable whole numbers	number of orders
Categorical – nominal	labels with no order	city, colour, gender
Categorical – ordinal	labels with an order	low < medium < high
Missing data	unknown value, stored as *NaN* / *None*	a sensor failed on day 4

Example: Missing values do not crash a plot – Matplotlib simply leaves a **gap**. Find them before plotting:

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt

s = pd.Series([3, 4, np.nan, 6, 7, np.nan, 9])
print(s.isna().sum(), 'missing values')        # count NaNs
plt.figure(figsize=(6, 2.6))
plt.plot(s, 'o-', color='navy', label='raw (gaps at NaN)')
plt.plot(s.interpolate(), '--', color='crimson', label='interpolated')
plt.legend(); plt.title('NaN makes a gap'); plt.show()
```

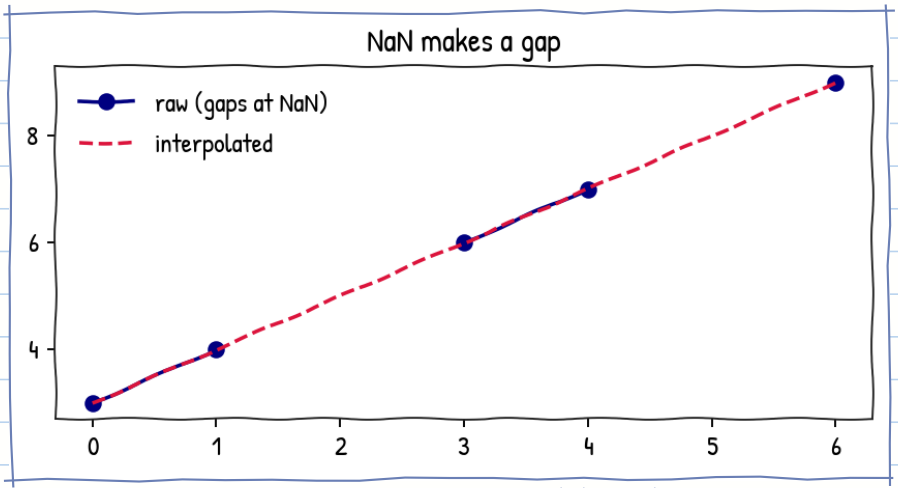


Fig 0.3 – the blue line breaks at each NaN; the red dashed line fills it by interpolation.

COMMON MISTAKE: Silently filling or dropping NaNs changes the story. Always **state** how you handled missing values (dropped / filled with mean / interpolated) on or beside the chart.

0.7 Setting up a notebook or script

Syntax: Jupyter needs one magic line so charts appear inside the notebook; a plain *.py* script needs *plt.show()* to open a window.

```
%matplotlib inline        # Jupyter only: show charts under the cell

import numpy as np, pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

rng = np.random.default_rng(42)          # reproducible random numbers
sns.set_theme(style='whitegrid')         # optional: nicer defaults
plt.rcParams['figure.figsize'] = (7, 4)  # default size for every figure
```

PRO TIP: Always create data with a **seeded** generator (*np.random.default_rng(42)*). Same seed = same numbers = reproducible charts, which matters for reports and debugging.

Practice: (1) Create a list, array and Series of five numbers and plot each with *plt.plot()*. (2) Make a DataFrame with a NaN and count it with *isna().sum()*. (3) Print your four library versions.

P2 Matplotlib Fundamentals

pyplot, basic charts, Figure & Axes, customization

Part II teaches the four ideas that every Matplotlib chart is built from: the **pyplot** interface, the **basic chart types**, the **Figure/Axes** model, and **customization**. Master these and Parts III–VI become easy.

1 Introduction to Matplotlib

the pyplot interface

Concept: Matplotlib is Python's foundational plotting library. Its *pyplot* module (imported as *plt*) works like a state machine: each *plt.* call changes the current figure, in the same order you would draw by hand on paper.
Why it matters: Nearly every other Python plotting tool (Seaborn, Pandas *.plot()*, many ML libraries) draws through Matplotlib, so knowing it lets you control any of them.

1.1 The essential pyplot functions

Function	What it does	Typical use
plt.figure()	creates a new blank canvas	*plt.figure(figsize=(8,4))*
plt.plot(x, y)	draws lines / markers	*plt.plot(x, y, 'o-')*
plt.title()	title above the plot	*plt.title('Sales 2025')*
plt.xlabel() plt.ylabel()	axis labels (with units!)	*plt.xlabel('Month')*
plt.xlim() plt.ylim()	visible axis range	*plt.ylim(0, 100)*
plt.grid()	background grid	*plt.grid(alpha=.3)*
plt.legend()	key for labelled artists	*plt.legend(loc='upper left')*
plt.savefig()	save to a file	*plt.savefig('a.png', dpi=300)*
plt.show()	display (required in scripts)	last line of a script

1.2 Your first complete chart

Syntax: *plt.plot(x, y, fmt, label=..., color=..., linewidth=...)*

```
import numpy as np
import matplotlib.pyplot as plt
```

```
x = np.linspace(0, 10, 100)              # 100 points from 0 to 10
plt.figure(figsize=(7, 3.6), dpi=100)    # width, height in inches
plt.plot(x, np.sin(x), label='sin(x)')   # first line
plt.plot(x, np.cos(x), label='cos(x)')   # second line
plt.title('Sine and Cosine')             # title
plt.xlabel('x (radians)'); plt.ylabel('value')
plt.xlim(0, 10); plt.ylim(-1.5, 1.5)     # axis ranges
plt.grid(True, alpha=0.3)                # light grid
plt.legend()                             # uses the label= texts
plt.savefig('trig.png', dpi=300, bbox_inches='tight')
plt.show()
```

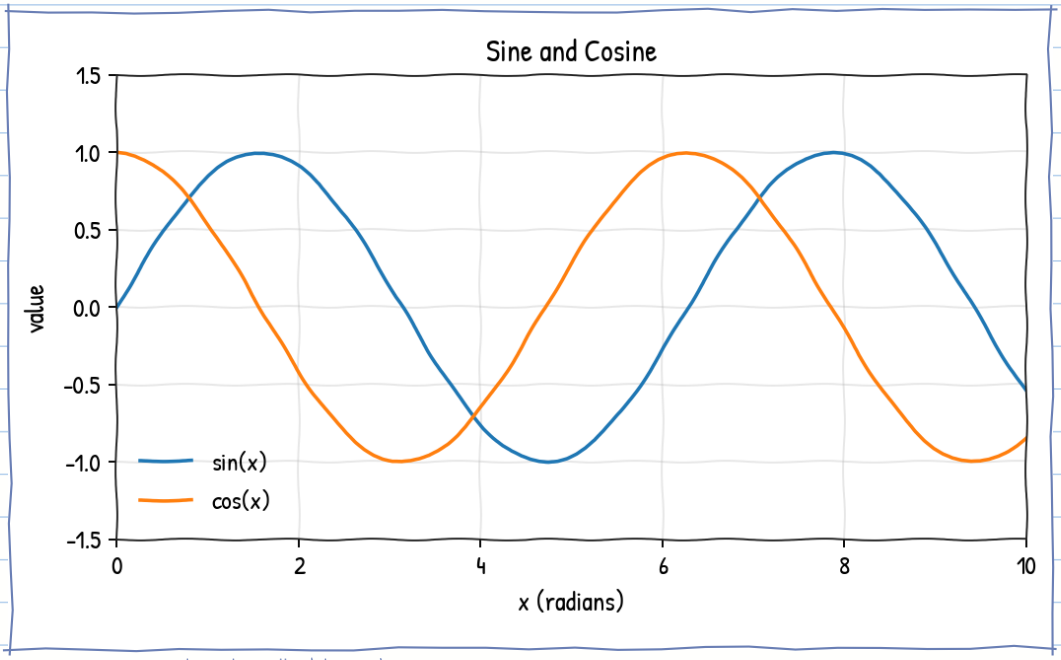


Fig 1.1 - a complete labelled line chart.

Explanation: `figsize=(7, 3.6)` is inches; pixels = inches x dpi. `label=` text is only shown once `plt.legend()` is called. `bbox_inches='tight'` trims empty white margins when saving. `alpha` is transparency (0 invisible - 1 solid).

1.3 Saving figures: PNG, JPG, SVG, PDF

```
import matplotlib.pyplot as plt
plt.plot([1, 2, 3], [2, 4, 3])
plt.savefig('chart.png', dpi=300)    # raster, best general choice
plt.savefig('chart.jpg', dpi=200)    # raster, smaller, lossy (needs Pillow)
plt.savefig('chart.svg')             # vector - scales infinitely, edit in Inkscape
plt.savefig('chart.pdf')             # vector - best for LaTeX / papers
```

Format	Type	Best for
PNG	raster (lossless)	web, slides, notebooks - default choice
JPG	raster (lossy)	photos; avoid for charts (blurry text)

Format	Type	Best for
SVG	vector	websites, editing in design tools
PDF	vector	research papers, print, LaTeX

COMMON MISTAKE: !!Call `plt.savefig()` BEFORE `plt.show()`.!! In scripts `show()` closes the figure, so saving afterwards writes a blank image.

PRO TIP: Use `dpi=300` for print/papers and `dpi=150` for slides. Set `facecolor='white'` when saving if you will paste into a dark-mode document. In Jupyter the magic `%matplotlib inline` shows figures under the cell automatically.

Practice: Plot y = x squared for x from -5 to 5, add title, axis labels and a grid, then save it as both PNG and PDF.

2 Basic Charts

13 chart types: purpose, syntax, misuse

For each chart: ***purpose***, ***when to use it***, ***syntax***, ***important parameters***, and ***common misuse***. Use this chapter as your chart dictionary.

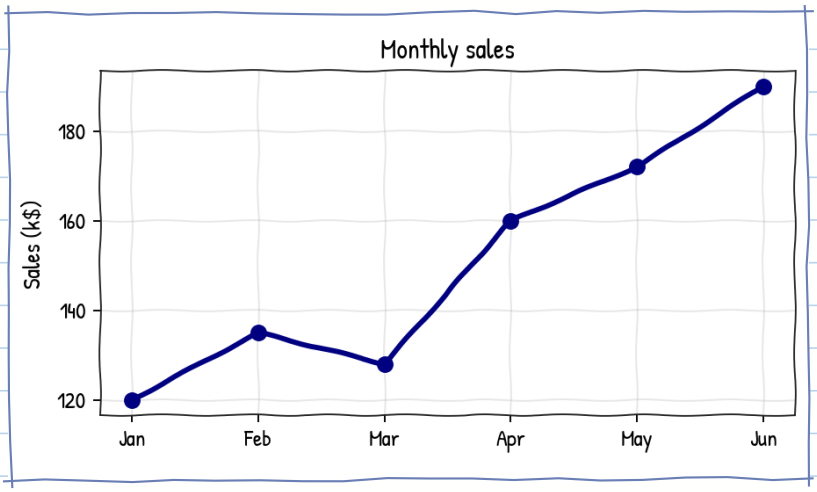
Chart	Question it answers	Data needed
Line	How does y change along x (often time)?	ordered x, numeric y
Scatter	Are two numeric variables related?	two numeric columns
Bar	How do categories compare?	category + value
Histogram	What is the shape of one variable?	one numeric column
Pie	What share does each part take of a whole?	few categories summing to 100%
Box	Median, spread and outliers per group?	numeric by group
Area / stack	How do totals and parts evolve?	ordered x, 1+ series
Stem / step	Discrete samples / piecewise-constant values	ordered discrete data
Error bar	How uncertain is each value?	values + errors

2.1 Line plot

Concept: Connects points in order with straight lines. **Why:** shows trends over time or any ordered axis.

Syntax: `plt.plot(x, y, color, linestyle, linewidth, marker, label)`

```
import matplotlib.pyplot as plt
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
sales = [120, 135, 128, 160, 172, 190]
plt.figure(figsize=(6, 3))
plt.plot(months, sales, color='navy', linewidth=2.5, marker='o', markersize=7)
plt.title('Monthly sales'); plt.ylabel('Sales (k$)'); plt.grid(alpha=.3)
plt.show()
```

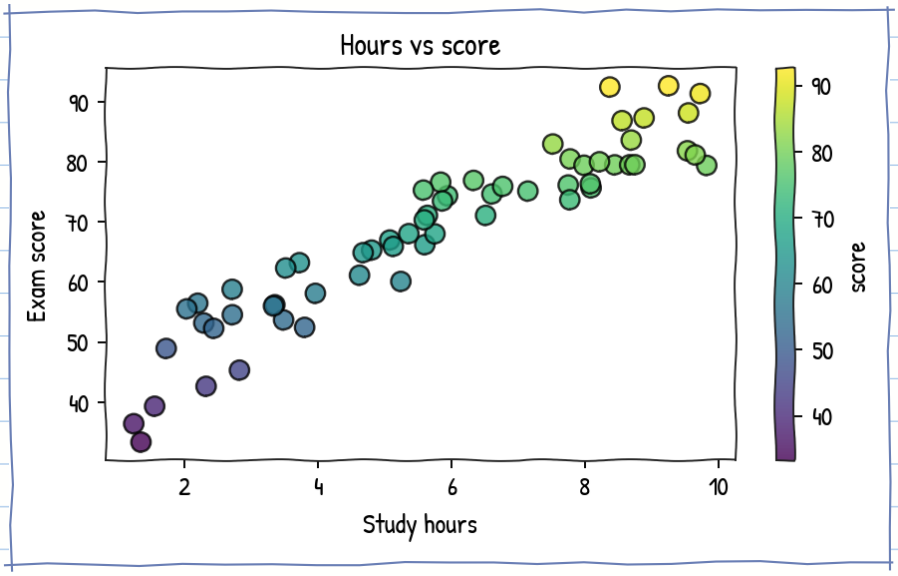


COMMON MISTAKE: Using a line for ***unordered categories*** (cities, products) implies a trend that does not exist. Use bars instead.

2.2 Scatter plot

Concept: One dot per observation; position = two numeric values. **Why:** reveals correlation, clusters and outliers.
Syntax: `plt.scatter(x, y, s=size, c=colour_or_values, alpha=, cmap=, marker=)`

```
import numpy as np, matplotlib.pyplot as plt
rng = np.random.default_rng(1)
hours = rng.uniform(1, 10, 60)
score = 40 + 5 * hours + rng.normal(0, 6, 60)
plt.figure(figsize=(6, 3))
sc = plt.scatter(hours, score, c=score, cmap='viridis', s=60, alpha=0.8, edgecolors='k')
plt.colorbar(sc, label='score')
plt.xlabel('Study hours'); plt.ylabel('Exam score'); plt.title('Hours vs score')
plt.show()
```



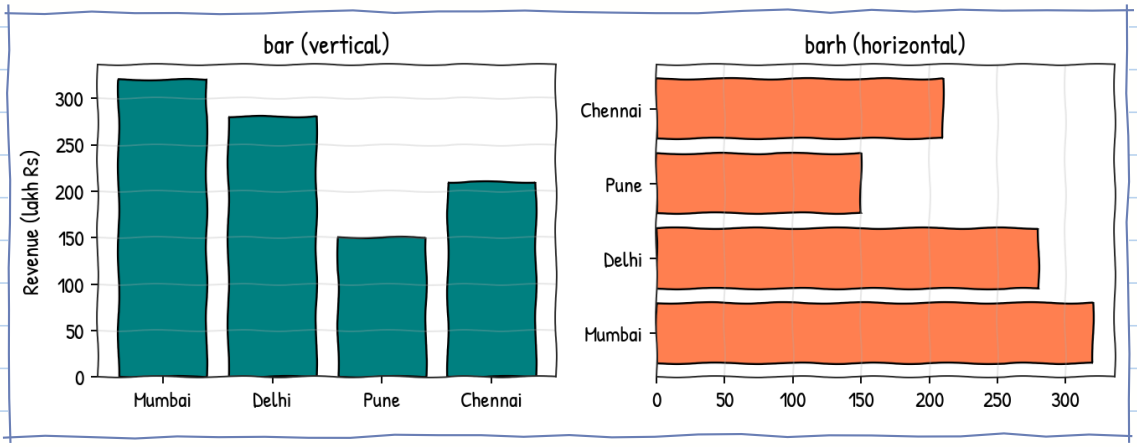
Explanation: `s` = marker area, `c` = colour (a single colour or one number per point mapped through `cmap`), `alpha` helps when points overlap.

WARNING: Overplotting: thousands of points hide each other. Lower *alpha*, shrink *s*, or use a hexbin / 2-D histogram (Part V).

2.3 Bar charts - vertical and horizontal

Concept: Bar length shows a value for each category. **Why:** best for comparing amounts between groups.
Syntax: *plt.bar(x, height, width=0.8)* and *plt.barh(y, width)*

```
import matplotlib.pyplot as plt
cities = ['Mumbai', 'Delhi', 'Pune', 'Chennai']
rev = [320, 280, 150, 210]
fig, (a1, a2) = plt.subplots(1, 2, figsize=(8, 3))
a1.bar(cities, rev, color='teal', edgecolor='k'); a1.set_title('bar (vertical)')
a2.barh(cities, rev, color='coral', edgecolor='k'); a2.set_title('barh (horizontal)')
for a in (a1, a2): a.set_xlabel('') ; a.grid(axis='y' if a is a1 else 'x', alpha=.3)
a1.set_ylabel('Revenue (lakh Rs)')
plt.tight_layout(); plt.show()
```

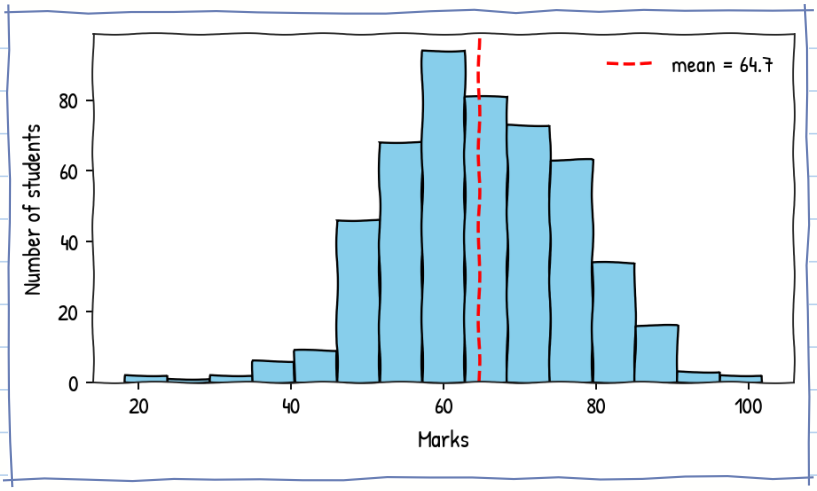


PRO TIP: Use **horizontal bars** when category names are long, and sort bars by value. Bars must start at **zero** - a truncated bar axis exaggerates differences.

2.4 Histogram

Concept: Splits a numeric range into **bins** and counts how many values fall in each. **Why:** shows shape: centre, spread, skew, outliers.
Syntax: *plt.hist(data, bins=10, density=False, color, edgecolor, alpha)*

```
import numpy as np, matplotlib.pyplot as plt
rng = np.random.default_rng(0)
marks = rng.normal(65, 12, 500)
plt.figure(figsize=(6, 3))
plt.hist(marks, bins=15, color='skyblue', edgecolor='black')
plt.axvline(marks.mean(), color='red', ls='--', label=f'mean = {marks.mean():.1f}')
plt.xlabel('Marks'); plt.ylabel('Number of students'); plt.legend(); plt.show()
```



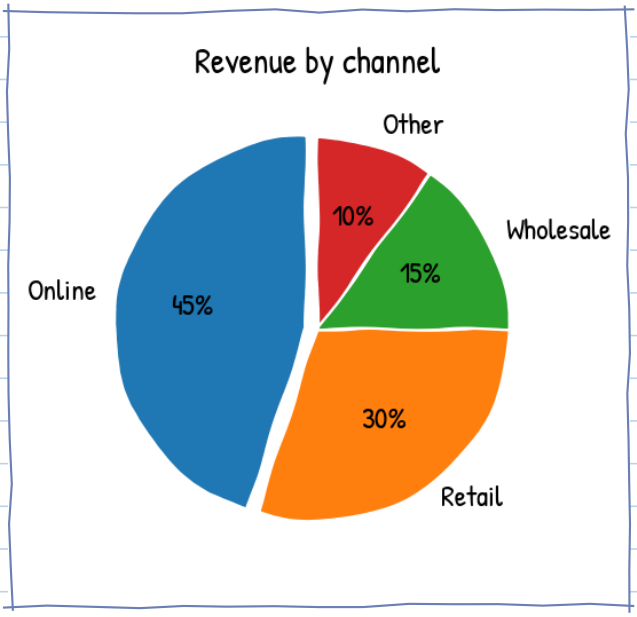
COMMON MISTAKE: A histogram is **not** a bar chart: its x-axis is a continuous number line, bars touch, and bin choice changes the look. See Part III for bin strategies.

2.5 Pie chart

Concept: A circle divided into slices proportional to each part's share. **Why:** only for **2–5** categories that add to a meaningful whole.

Syntax: `plt.pie(sizes, labels=, autopct='%1.0f%%', startangle=90, explode=, colors=)`

```
import matplotlib.pyplot as plt
share = [45, 30, 15, 10]
plt.figure(figsize=(4.2, 3.2))
plt.pie(share, labels=['Online', 'Retail', 'Wholesale', 'Other'], autopct='%1.0f%%',
        startangle=90, explode=[0.06, 0, 0, 0], wedgeprops=dict(edgecolor='white'))
plt.title('Revenue by channel'); plt.show()
```



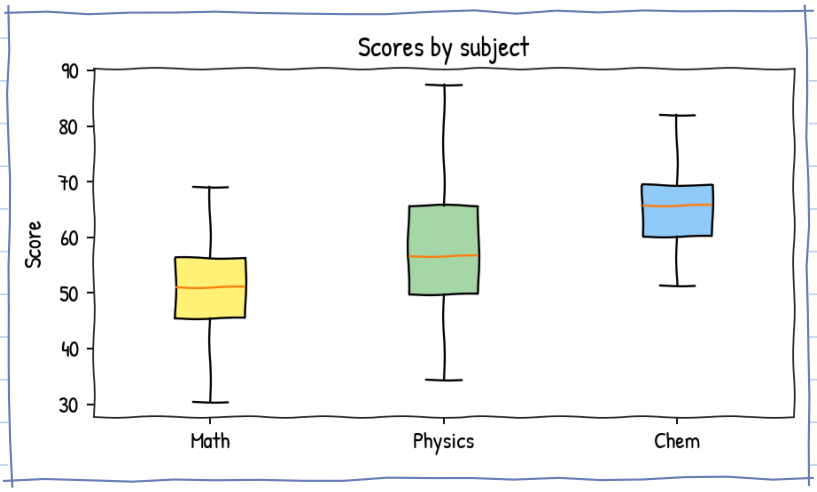
WARNING: Humans compare angles poorly. With more than ~5 slices or similar sizes, switch to a sorted bar chart.

2.6 Box plot

Concept: Draws the **five-number summary**: minimum, Q1, median, Q3, maximum; dots beyond the whiskers are outliers (> 1.5 x IQR). **Why:** compares distributions of several groups compactly.

Syntax: *plt.boxplot([g1, g2, g3], labels=[...], patch_artist=True, vert=True, showmeans=False)*

```
import numpy as np, matplotlib.pyplot as plt
rng = np.random.default_rng(2)
data = [rng.normal(m, s, 80) for m, s in [(50, 8), (58, 12), (65, 6)]]
plt.figure(figsize=(6, 3))
bp = plt.boxplot(data, tick_labels=['Math', 'Physics', 'Chem'], patch_artist=True)
for patch, c in zip(bp['boxes'], ['#fff176', '#a5d6a7', '#90caf9']): patch.set_facecolor(c)
plt.ylabel('Score'); plt.title('Scores by subject'); plt.show()
```



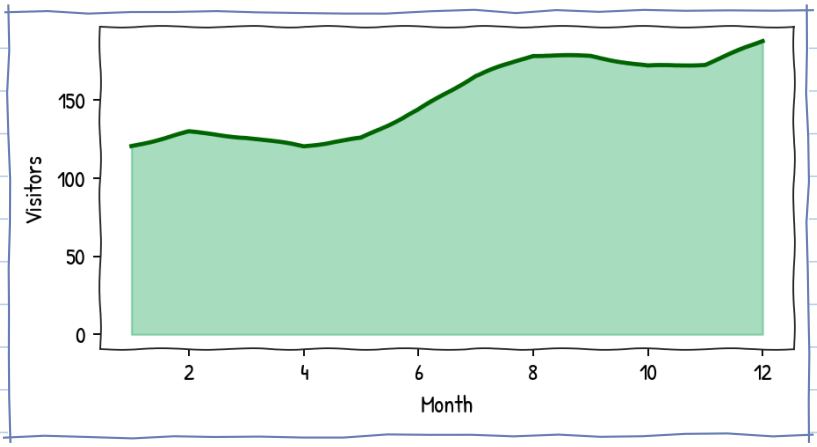
NOTE: Matplotlib 3.9 renamed the box plot argument *labels* to *++tick_labels++*. Older code still uses *labels=* (now deprecated) - a classic version trap.

2.7 Area plot

Concept: A line plot with the region below filled. **Why:** emphasises magnitude/volume over time (cumulative feel).

Syntax: *plt.fill_between(x, y1, y2=0, alpha=)*

```
import numpy as np, matplotlib.pyplot as plt
x = np.arange(1, 13); y = 100 + 8 * x + 15 * np.sin(x)
plt.figure(figsize=(6, 2.8))
plt.fill_between(x, y, color='mediumseagreen', alpha=0.45)
plt.plot(x, y, color='darkgreen', lw=2)
plt.xlabel('Month'); plt.ylabel('Visitors'); plt.show()
```

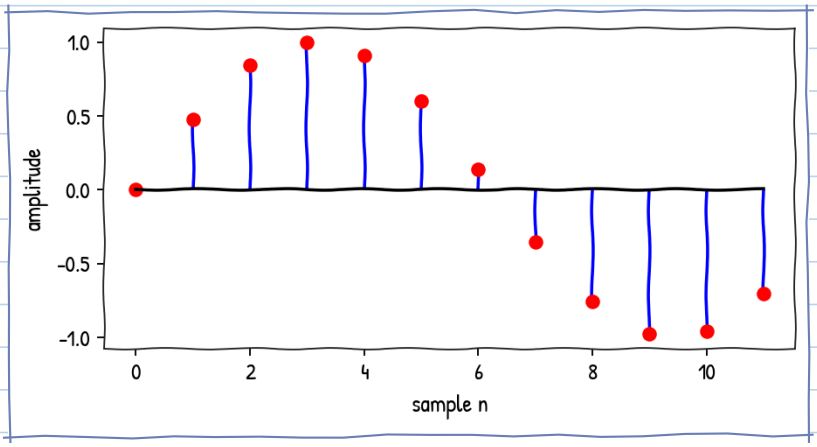


2.8 Stem plot

Concept: Vertical stick from the baseline to each value with a dot on top. **Why:** discrete samples, signals, differences from a baseline.

Syntax: *plt.stem(x, y, linefmt=, markerfmt=, basefmt=)*

```
import numpy as np, matplotlib.pyplot as plt
n = np.arange(0, 12)
plt.figure(figsize=(6, 2.8))
plt.stem(n, np.sin(n / 2), linefmt='b-', markerfmt='ro', basefmt='k-')
plt.xlabel('sample n'); plt.ylabel('amplitude'); plt.show()
```

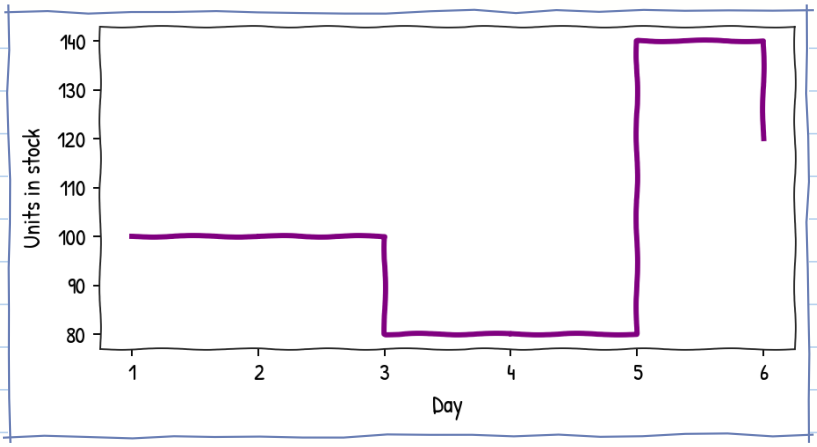


2.9 Step plot

Concept: Holds each value constant until the next x, creating stairs. **Why:** quantities that change in jumps - prices, inventory, interest rates, ECDFs.

Syntax: *plt.step(x, y, where='pre' | 'post' | 'mid')*

```
import matplotlib.pyplot as plt
days = [1, 2, 3, 4, 5, 6]; stock = [100, 100, 80, 80, 140, 120]
plt.figure(figsize=(6, 2.8))
plt.step(days, stock, where='post', color='purple', lw=2.5)
plt.xlabel('Day'); plt.ylabel('Units in stock'); plt.show()
```

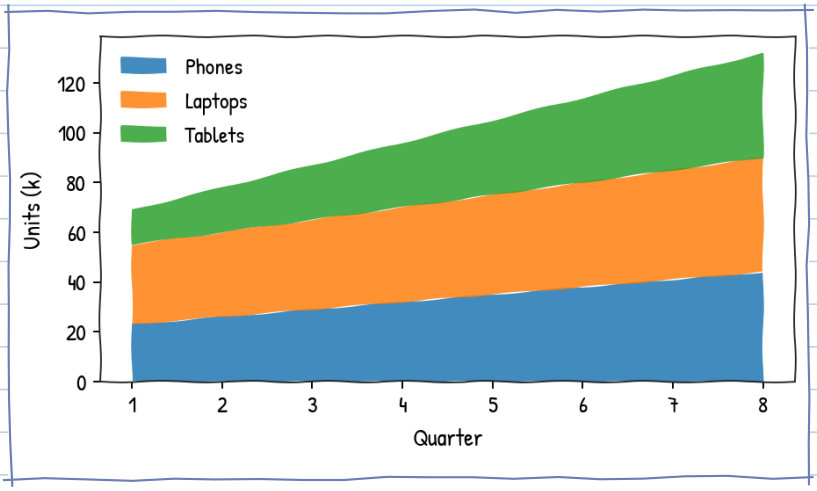


2.10 Stack plot

Concept: Stacks several series so their sum shows the total and each layer shows a part. **Why:** composition over time.

Syntax: `plt.stackplot(x, y1, y2, y3, labels=[...], colors=[...])`

```
import numpy as np, matplotlib.pyplot as plt
q = np.arange(1, 9)
a, b, c = 20 + 3 * q, 30 + 2 * q, 10 + 4 * q
plt.figure(figsize=(6, 3))
plt.stackplot(q, a, b, c, labels=['Phones', 'Laptops', 'Tablets'], alpha=.85)
plt.legend(loc='upper left'); plt.xlabel('Quarter'); plt.ylabel('Units (k)'); plt.show()
```



WARNING: In a stacked chart only the **bottom** layer has a common baseline. Layers on top are hard to compare - keep the number of layers small.

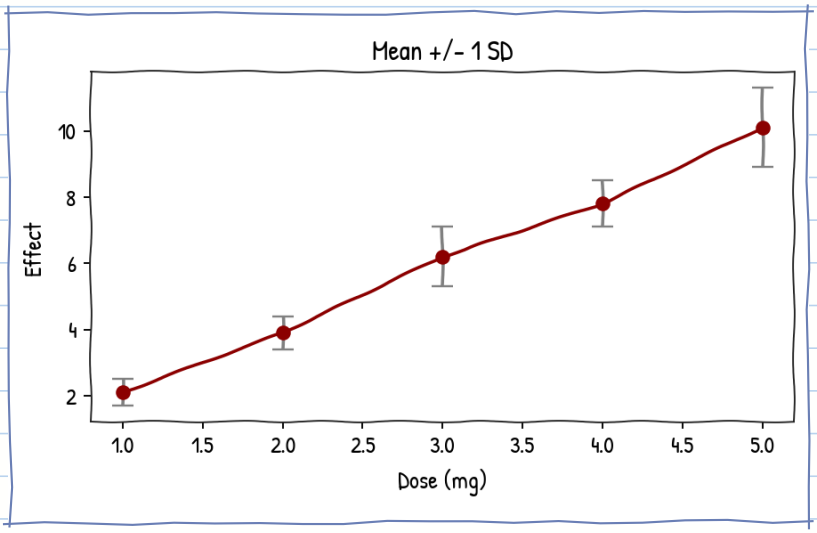
2.11 Error-bar plot

Concept: Dots (or bars) with lines showing uncertainty: standard deviation, standard error or a confidence interval.

Why: an average without an error bar hides how reliable it is.

Syntax: `plt.errorbar(x, y, yerr=, xerr=, fmt='o', capsize=4)`

```
import matplotlib.pyplot as plt
dose = [1, 2, 3, 4, 5]; effect = [2.1, 3.9, 6.2, 7.8, 10.1]; sd = [0.4, 0.5, 0.9, 0.7, 1.2]
plt.figure(figsize=(6, 3))
plt.errorbar(dose, effect, yerr=sd, fmt='o-', color='darkred', ecolor='gray', capsize=5)
plt.xlabel('Dose (mg)'); plt.ylabel('Effect'); plt.title('Mean +/- 1 SD'); plt.show()
```

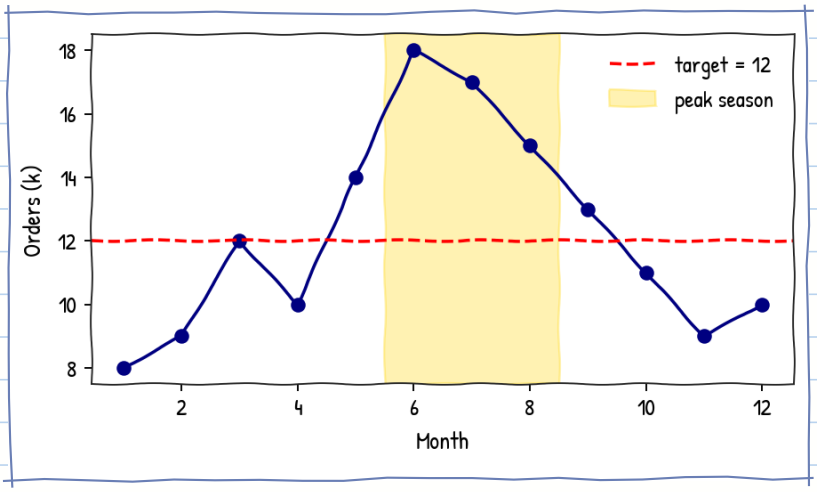


EXAM TIP: Always say WHAT the bars mean (SD, SEM or 95% CI) in the caption. They are different quantities.

2.12 Reference lines and spans

Syntax: `plt.axhline(y)` `plt.axvline(x)` `plt.axhspan(y1,y2)` `plt.axvspan(x1,x2)`

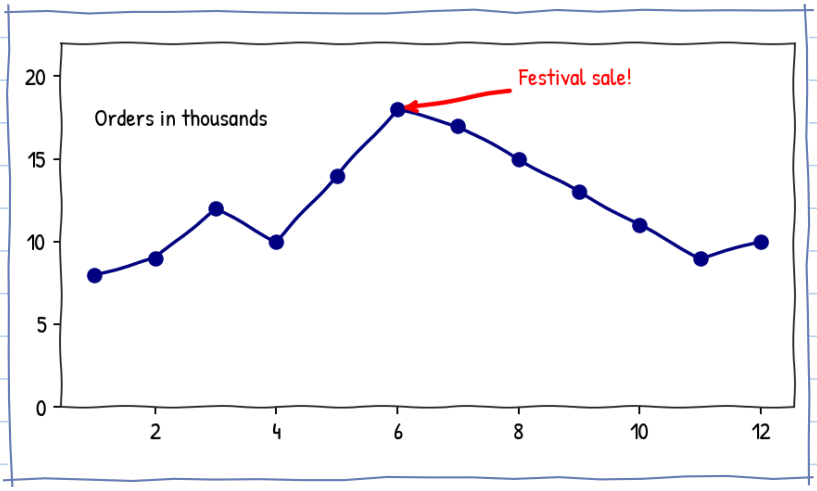
```
import numpy as np, matplotlib.pyplot as plt
x = np.arange(1, 13); y = np.array([8, 9, 12, 10, 14, 18, 17, 15, 13, 11, 9, 10])
plt.figure(figsize=(6, 3))
plt.plot(x, y, 'o-', color='navy')
plt.axhline(12, color='red', ls='--', label='target = 12')
plt.axvspan(5.5, 8.5, color='gold', alpha=.3, label='peak season')
plt.legend(); plt.xlabel('Month'); plt.ylabel('Orders (k)'); plt.show()
```



2.13 Text and annotations

Syntax: `plt.text(x, y, 'txt')` `plt.annotate('txt', xy=(point), xytext=(label_pos), arrowprops=dict(arrowstyle='->'))`

```
import numpy as np, matplotlib.pyplot as plt
x = np.arange(1, 13); y = np.array([8, 9, 12, 10, 14, 18, 17, 15, 13, 11, 9, 10])
plt.figure(figsize=(6, 3))
plt.plot(x, y, 'o-', color='navy')
plt.annotate('Festival sale!', xy=(6, 18), xytext=(8, 19.5), color='red',
             arrowprops=dict(arrowstyle='->', color='red', lw=2))
plt.text(1, 17, 'Orders in thousands', fontsize=11, style='italic')
plt.ylim(0, 22); plt.show()
```



Practice: Make a scatter plot of two random arrays coloured by a third; then make a bar chart of 5 categories, sorted descending, with a horizontal average line.

3 Figure and Axes

the object-oriented heart of Matplotlib

Concept: Figure = the whole window/page. Axes = one plot area inside it (with its own x/y Axis, title, labels, data). Axis = the number line (x or y) with ticks and tick labels. One Figure can hold many Axes.

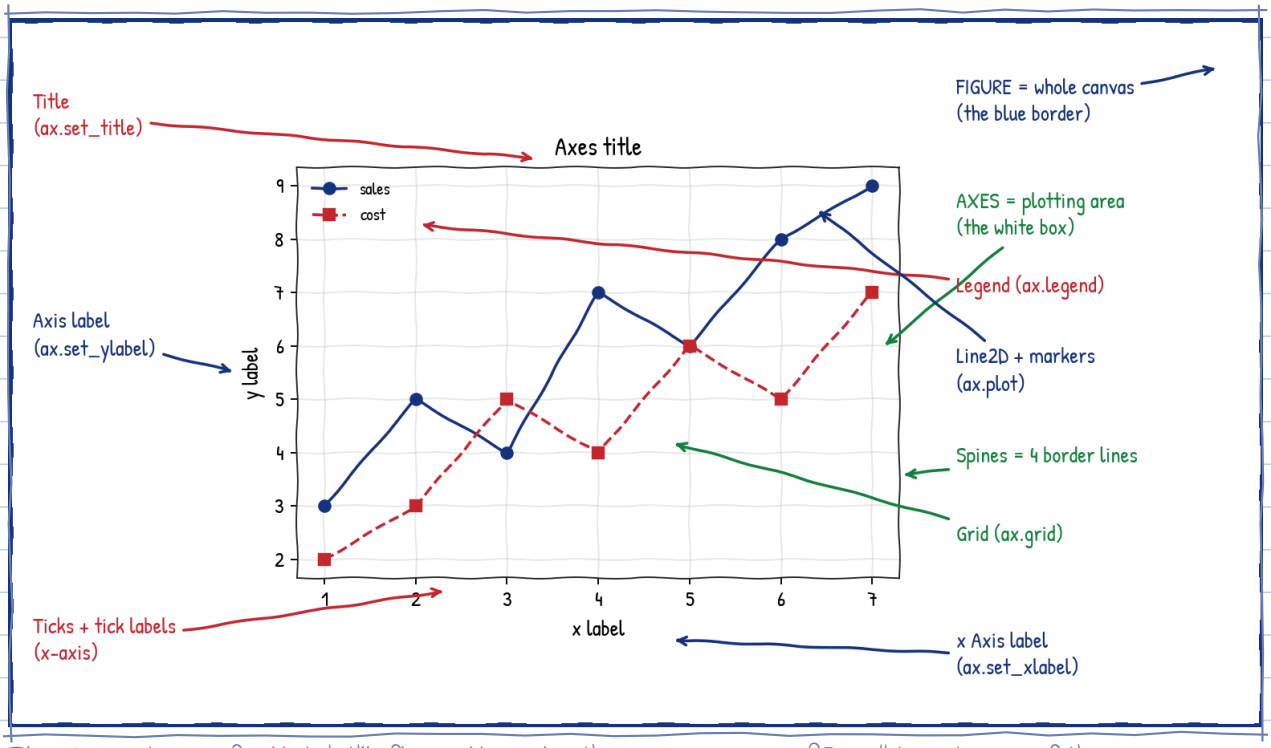
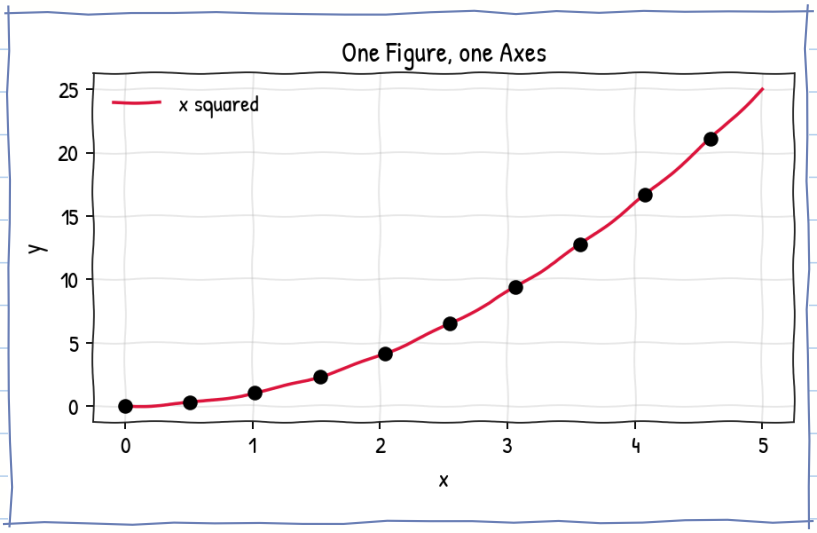


Fig 3.1 - anatomy of a Matplotlib figure. Memorise these names; every API call targets one of them.

3.1 The object-oriented pattern

Syntax: `fig, ax = plt.subplots(nrows=1, ncols=1, figsize=(w, h))` then call methods on `ax`: `ax.plot()`, `ax.scatter()`, `ax.bar()`, `ax.set_title()`, `ax.set_xlabel()` ...

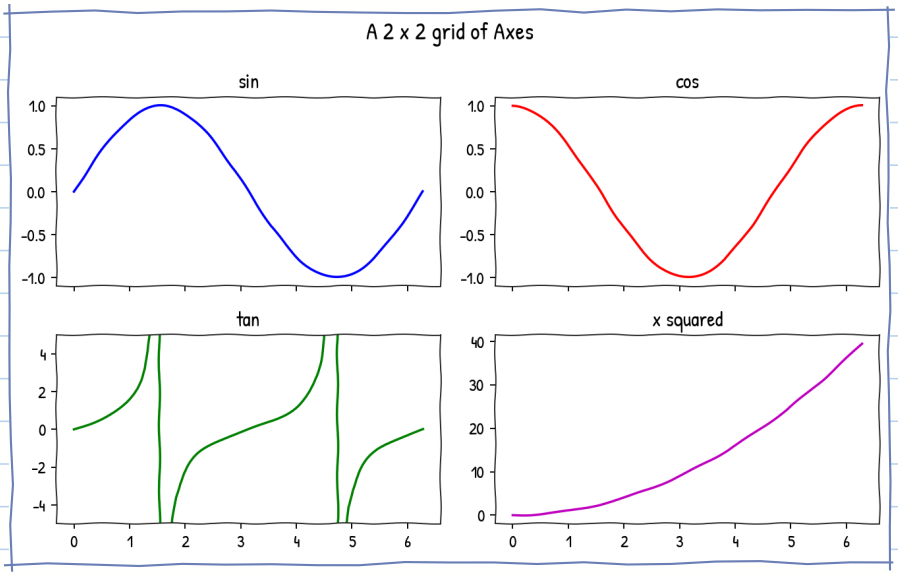
```
import numpy as np, matplotlib.pyplot as plt
x = np.linspace(0, 5, 50)
fig, ax = plt.subplots(figsize=(6, 3))
ax.plot(x, x ** 2, label='x squared', color='crimson')
ax.scatter(x[::5], x[::5] ** 2, color='black', zorder=3)
ax.set_title('One Figure, one Axes'); ax.set_xlabel('x'); ax.set_ylabel('y')
ax.legend(); ax.grid(alpha=.3)
plt.show()
```



KEY IDEA: Rule: ***plt.xxx()* works on the current Axes (quick scripts).** *ax.set_xxx()** targets a specific Axes (real projects). Note the renames: *plt.title -> ax.set_title*, *plt.xlabel -> ax.set_xlabel*, *plt.xlim -> ax.set_xlim*.

3.2 Multiple charts: rows and columns of subplots

```
import numpy as np, matplotlib.pyplot as plt
x = np.linspace(0, 2 * np.pi, 100)
fig, axs = plt.subplots(2, 2, figsize=(8, 5), sharex=True)   # axs is a 2x2 array
axs[0, 0].plot(x, np.sin(x), 'b');   axs[0, 0].set_title('sin')
axs[0, 1].plot(x, np.cos(x), 'r');   axs[0, 1].set_title('cos')
axs[1, 0].plot(x, np.tan(x), 'g');   axs[1, 0].set_title('tan'); axs[1, 0].set_ylim(-5, 5)
axs[1, 1].plot(x, x ** 2, 'm');      axs[1, 1].set_title('x squared')
fig.suptitle('A 2 x 2 grid of Axes', fontsize=15)
fig.tight_layout()
plt.show()
```



Explanation: *axs[row, col]* indexes the grid (starting at **0**). With a single row or column *axs* is 1-D: *axs[i]*. Use *axs.flat* or *axs.ravel()* to loop over all panels.

3.3 plt.subplot() vs plt.subplots()

	plt.subplot(r, c, i)	plt.subplots(r, c)
Creates	ONE axes per call (index i starts at **1**)	the Figure and ALL Axes at once
Returns	the Axes	fig, axs
Style	pyplot / state-machine	object-oriented (recommended)
Index	1 ... r*c (1-based)	axs[row, col] (0-based)

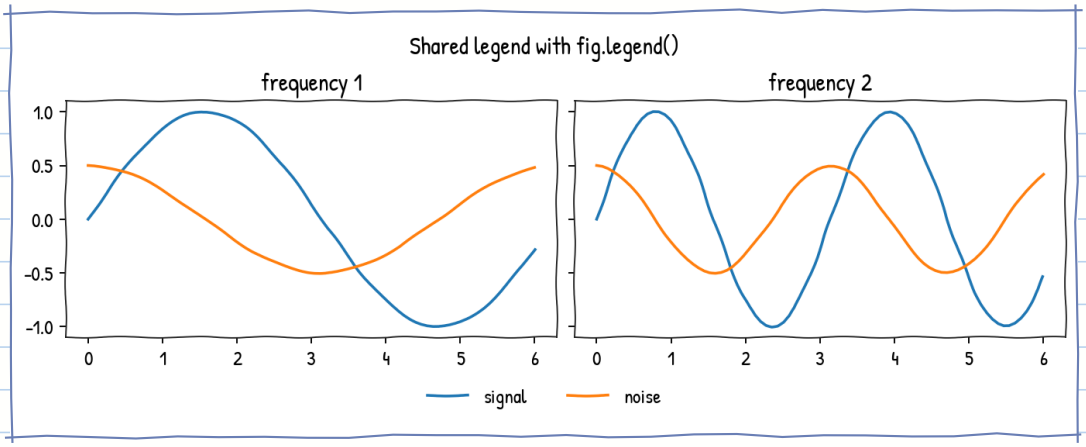
COMMON MISTAKE: !!Confusing plt.figure() and plt.subplots():!! plt.figure() only makes an empty canvas; plt.subplots() makes the canvas ***and*** the Axes. Calling plt.subplots() repeatedly inside a loop creates many separate figures.

3.4 Sharing axes, layout and spacing

Tool	Purpose
sharex=True, sharey=True	panels share the same scale so they are directly comparable
figsize=(w,h) / dpi	size in inches / resolution
fig.tight_layout()	auto-adjusts padding so labels do not overlap
layout='constrained'	newer, smarter automatic layout: plt.subplots(layout='constrained')
fig.subplots_adjust(hspace=, wspace=)	manual spacing between panels
fig.suptitle()	one title above all panels

3.5 A common legend for the whole figure

```
import numpy as np, matplotlib.pyplot as plt
x = np.linspace(0, 6, 80)
fig, (a1, a2) = plt.subplots(1, 2, figsize=(8, 3), sharey=True, layout='constrained')
for a, k in zip((a1, a2), (1, 2)):
    a.plot(x, np.sin(k * x), color='tab:blue',  label='signal')
    a.plot(x, 0.5 * np.cos(k * x), color='tab:orange', label='noise')
    a.set_title(f'frequency {k}')
h, l = a1.get_legend_handles_labels()
fig.legend(h, l, loc='outside lower center', ncol=2)
fig.suptitle('Shared legend with fig.legend()')
plt.show()
```



NOTE: *Figure-level* customization (*fig.suptitle*, *fig.legend*, *fig.savefig*, size) affects the whole canvas; ***axes-level*** customization (*ax.set_title*, *ax.set_xlim*, *ax.legend*) affects one panel. Seaborn uses the same words - see Part IV.

Practice: Build a 1 x 3 figure that shows a line, a scatter and a histogram of the same random data, share the y-axis where sensible, add a suptitle and save it.

4 Plot Customization

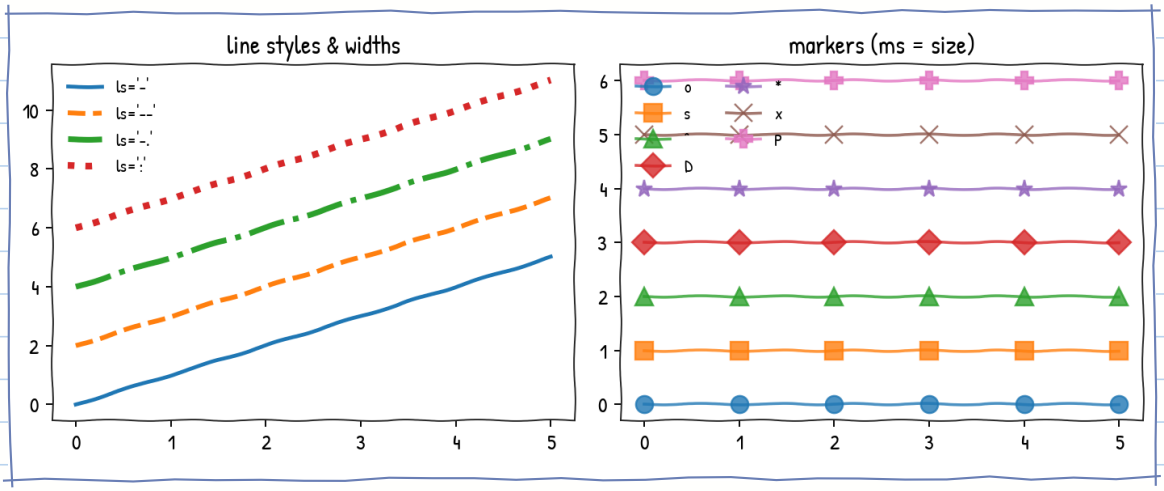
colours, lines, markers, fonts, legends, styles

4.1 Colours

Way to specify	Example	Notes
Single letter	`'r' 'g' 'b' 'k' 'c' 'm' 'y' 'w'`	quick sketches
Named colour	`'steelblue'`, `'tomato'`, `'gold'`	148 CSS names
Cycle colour	`'C0'` … `'C9'` or `'tab:blue'`	default 10-colour cycle
Hex code	`'#14327d'`	exact brand colours
RGB tuple (0–1)	`(0.08, 0.2, 0.49)`	floats, **not** 0–255
RGBA tuple	`(0.08, 0.2, 0.49, 0.5)`	4th value = transparency
Grey string	`'0.6'`	0 = black, 1 = white

4.2 Line styles, widths, markers and alpha

```
import numpy as np, matplotlib.pyplot as plt
x = np.arange(6)
fig, (a, b) = plt.subplots(1, 2, figsize=(8.5, 3.4))
for i, ls in enumerate(['-', '--', '-.', ':']):
    a.plot(x, x + i * 2, ls, lw=2 + i * 0.5, label=f"ls='{ls}'")
a.set_title('line styles & widths'); a.legend(fontsize=9)
for i, m in enumerate(['o', 's', '^', 'D', '*', 'x', 'P']):
    b.plot(x, x * 0 + i, m + '-', ms=9, alpha=.8, label=m)
b.set_title('markers (ms = size)'); b.legend(ncol=2, fontsize=9, loc='upper left')
plt.tight_layout(); plt.show()
```

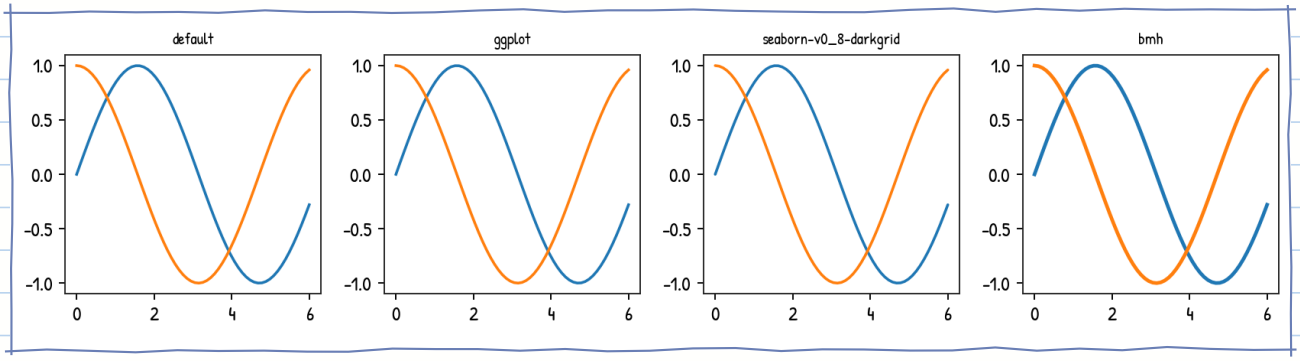


Parameter	Short	Meaning
linestyle	ls	'-' solid, '--' dashed, '-.' dash-dot, ':' dotted
linewidth	lw	thickness in points
marker	-	'o' 's' '^' 'v' 'D' '*' '+' 'x' '.'
markersize	ms	marker size
color	c	any colour from section 4.1
alpha	-	0 transparent … 1 opaque

Syntax: Format-string shortcut: `plt.plot(x, y, 'ro--')` = red circles with dashed line (colour + marker + linestyle).

4.3 Plot styles with plt.style.use()

```
import numpy as np, matplotlib.pyplot as plt
x = np.linspace(0, 6, 60)
names = ['default', 'ggplot', 'seaborn-v0_8-darkgrid', 'bmh']
fig, axs = plt.subplots(1, 4, figsize=(10, 2.6))
for ax, s in zip(axs, names):
    with plt.style.context(s):
        ax.plot(x, np.sin(x)); ax.plot(x, np.cos(x)); ax.set_title(s, fontsize=9)
plt.tight_layout(); plt.show()
print(len(plt.style.available), 'styles, e.g.', plt.style.available[:5])
```

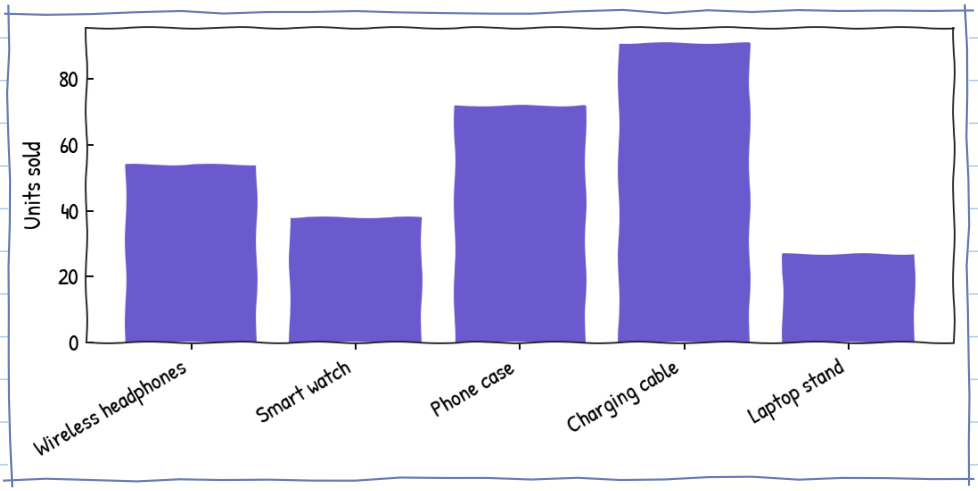


Explanation: `plt.style.use('ggplot')` changes defaults for **everything after it**. `with plt.style.context('bmh'):` applies a style temporarily. List all with `plt.style.available`.

NOTE: Since Matplotlib 3.6 the old Seaborn styles are named `seaborn-v0_8-...` (e.g. `'seaborn-v0_8-whitegrid'`). Old code using `'seaborn-whitegrid'` fails with an OSError.

4.4 Ticks, tick labels and rotation

```
import numpy as np, matplotlib.pyplot as plt
products = ['Wireless headphones', 'Smart watch', 'Phone case', 'Charging cable', 'Laptop stand']
sales = [54, 38, 72, 91, 27]
fig, ax = plt.subplots(figsize=(6.5, 3.2))
ax.bar(products, sales, color='slateblue')
ax.set_xticks(range(len(products)))                  # fixed tick positions first
ax.set_xticklabels(products, rotation=30, ha='right')   # then labels, rotated
ax.set_yticks(np.arange(0, 100, 20))
ax.tick_params(axis='y', labelsize=11, direction='in')
ax.set_ylabel('Units sold')
plt.tight_layout(); plt.show()
```



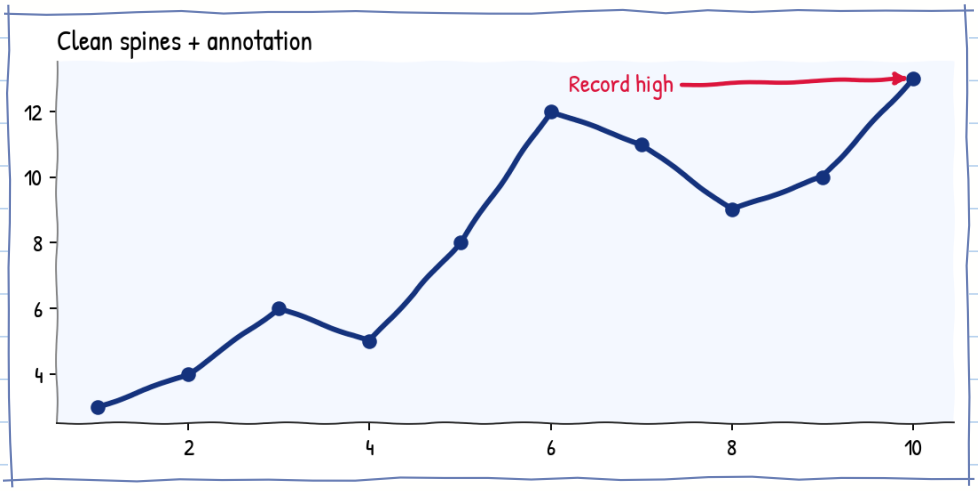
COMMON MISTAKE: Calling `set_xticklabels()` without first setting `set_xticks()` gives a UserWarning and can mislabel ticks. **Always positions first, labels second.**

4.5 Fonts, legends and placement

Item	Code
font size / weight	`fontsize=14, fontweight='bold'`
font family	`fontfamily='serif'` or `plt.rcParams['font.family'] = 'serif'`
legend position	`loc='upper right'` (also `best`, `lower left`, `center`, ...)
legend outside plot	`ax.legend(bbox_to_anchor=(1.02, 1), loc='upper left')`
legend columns / title	`ncol=2, title='Product'`

4.6 Spines, background colour, annotations and arrows

```
import numpy as np, matplotlib.pyplot as plt
x = np.arange(1, 11); y = np.array([3, 4, 6, 5, 8, 12, 11, 9, 10, 13])
fig, ax = plt.subplots(figsize=(6.5, 3.2), facecolor='#fffdf2')
ax.set_facecolor('#f4f8ff')                    # axes background
ax.plot(x, y, color='#14327d', lw=2.5, marker='o')
ax.spines[['top', 'right']].set_visible(False)     # remove two spines
ax.spines['left'].set_color('gray')
ax.annotate('Record high', xy=(10, 13), xytext=(6.2, 12.6), fontsize=12, color='crimson',
            arrowprops=dict(arrowstyle='->', color='crimson', lw=2))
ax.set_title('Clean spines + annotation', loc='left', fontweight='bold')
plt.tight_layout(); plt.show()
```



4.7 Custom colour cycles

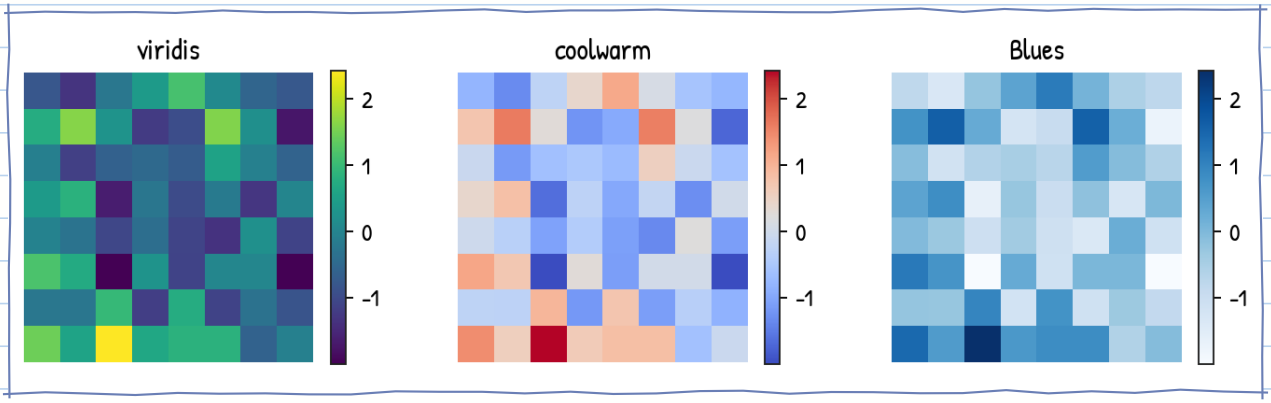
```
import matplotlib.pyplot as plt
from cycler import cycler
plt.rcParams['axes.prop_cycle'] = cycler(color=['#14327d', '#c4262e', '#12803a', '#e6a800'])
# every new line now takes the next colour in this list
```

4.8 Colormaps and colorbars

Concept: A **colormap** converts numbers into colours; a **colorbar** is the legend that explains it. Choose by data type:

Data type	Colormap family	Examples
Sequential (low -> high)	one hue getting darker	viridis, Blues, magma
Diverging (negative <- 0 -> positive)	two hues meeting at a neutral centre	coolwarm, RdBu, PuOr
Categorical (groups)	distinct colours	tab10, Set2

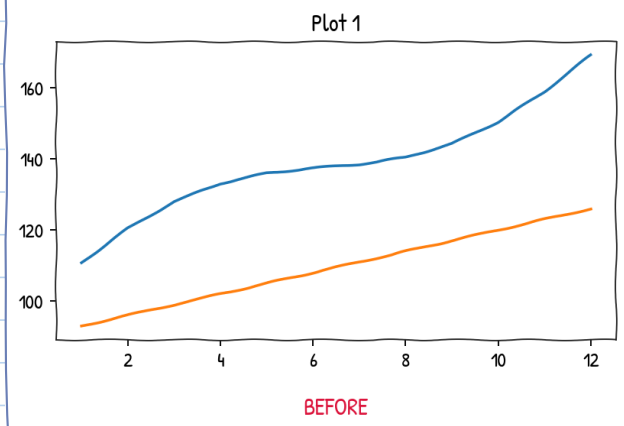
```
import numpy as np, matplotlib.pyplot as plt
rng = np.random.default_rng(5)
z = rng.normal(size=(8, 8))
fig, axs = plt.subplots(1, 3, figsize=(10, 3))
for ax, cm in zip(axs, ['viridis', 'coolwarm', 'Blues']):
    im = ax.imshow(z, cmap=cm); ax.set_title(cm); ax.axis('off')
    fig.colorbar(im, ax=ax, shrink=.8)
plt.show()
```

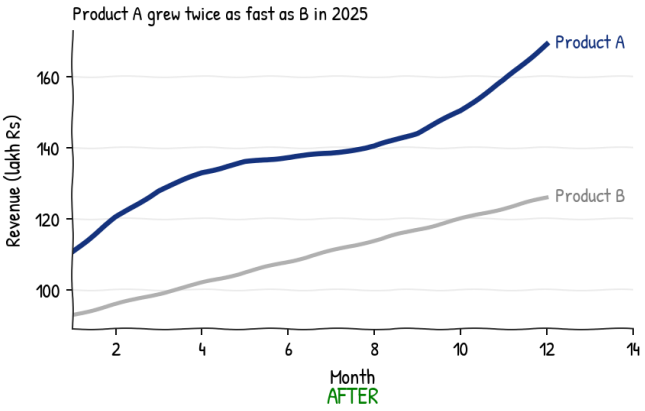


WARNING: Avoid the ***rainbow/jet*** colormap: it is not perceptually uniform, creates false boundaries and is unreadable for colour-blind readers. Prefer *viridis*.

4.9 Before and after: professional styling

```
import numpy as np, matplotlib.pyplot as plt
m = np.arange(1, 13)
a = 100 + 6 * m + 10 * np.sin(m / 2); b = 90 + 3 * m
fig, (l, r) = plt.subplots(1, 2, figsize=(10, 3.6))
# ---- BEFORE: defaults, no labels, boxed
l.plot(m, a); l.plot(m, b); l.set_title('Plot 1')
# ---- AFTER: insight title, direct labels, units, no clutter
r.plot(m, a, color='#14327d', lw=2.8); r.plot(m, b, color='#b0b0b0', lw=2.2)
r.text(12.2, a[-1], 'Product A', color='#14327d', va='center', fontweight='bold')
r.text(12.2, b[-1], 'Product B', color='gray', va='center')
r.set_title('Product A grew twice as fast as B in 2025', loc='left', fontweight='bold', fontsize=11)
r.set_xlabel('Month'); r.set_ylabel('Revenue (lakh Rs)'); r.set_xlim(1, 14)
r.spines[['top', 'right']].set_visible(False); r.grid(axis='y', alpha=.25)
l.text(.5, -.25, 'BEFORE', transform=l.transAxes, ha='center', color='crimson', fontsize=13)
r.text(.5, -.25, 'AFTER', transform=r.transAxes, ha='center', color='green', fontsize=13)
plt.tight_layout(); plt.show()
```





PRO TIP: Pro styling = (1) a title that states the ***insight***, (2) labelled axes with ***units***, (3) direct labels instead of a legend, (4) removed spines/gridline clutter, (5) one highlight colour and the rest grey.

Practice: Take any earlier chart and restyle it: new colour cycle, no top/right spines, rotated tick labels, a legend outside the axes, and one annotated point.

P3 Intermediate & Advanced Matplotlib

multi-axes, time series, 2-D/3-D, layouts, production

Part III turns the basics into professional skills. Each topic is taught with a practical example rather than a bare definition.

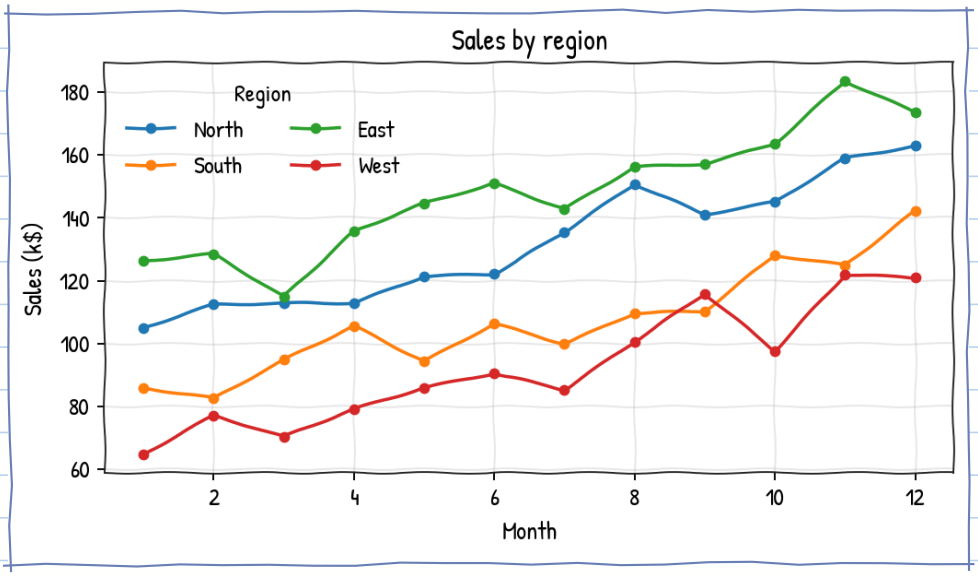
A Intermediate Matplotlib

comparisons, scales, time and uncertainty

A.1 Multiple lines and comparative plots

Concept: Draw several datasets on the same Axes by calling `ax.plot()` repeatedly (or in a loop) and labelling each line. **Why:** direct visual comparison on one common scale.

```
import numpy as np, matplotlib.pyplot as plt
rng = np.random.default_rng(7)
months = np.arange(1, 13)
regions = {'North': 100, 'South': 80, 'East': 120, 'West': 60}
fig, ax = plt.subplots(figsize=(7, 3.4))
for name, base in regions.items():
    sales = base + 5 * months + rng.normal(0, 8, 12)
    ax.plot(months, sales, marker='o', ms=4, label=name)
ax.set_xlabel('Month'); ax.set_ylabel('Sales (k$)'); ax.set_title('Sales by region')
ax.legend(title='Region', ncol=2); ax.grid(alpha=.3)
plt.show()
```

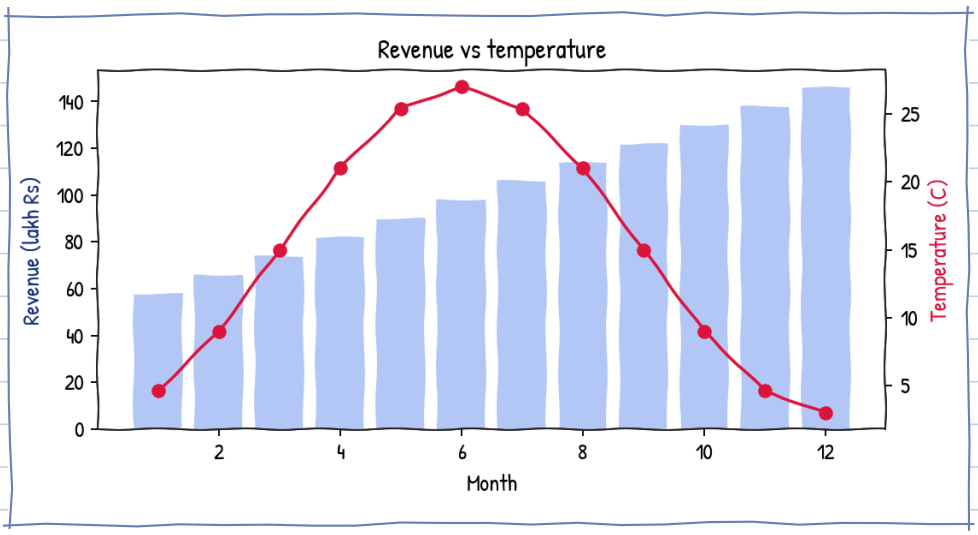


PRO TIP: More than ~5 lines becomes spaghetti. Highlight one line in a strong colour, grey out the rest, or use small multiples (a grid of Axes, *sharey=True*).

A.2 Twin axes: twinx() and twiny()

Concept: *ax.twinx()* adds a second y-axis on the right that shares the same x-axis. **Why:** two quantities with different units over the same x (e.g. revenue in Rs and temperature in C).

```
import numpy as np, matplotlib.pyplot as plt
m = np.arange(1, 13)
rev = 50 + 8 * m
temp = 15 + 12 * np.sin((m - 3) / 12 * 2 * np.pi)
fig, ax1 = plt.subplots(figsize=(7, 3.2))
ax1.bar(m, rev, color='#b3c7f7'); ax1.set_ylabel('Revenue (lakh Rs)', color='#14327d')
ax2 = ax1.twinx()                                  # second y axis
ax2.plot(m, temp, color='crimson', marker='o'); ax2.set_ylabel('Temperature (C)', color='crimson')
ax1.set_xlabel('Month'); ax1.set_title('Revenue vs temperature')
plt.show()
```

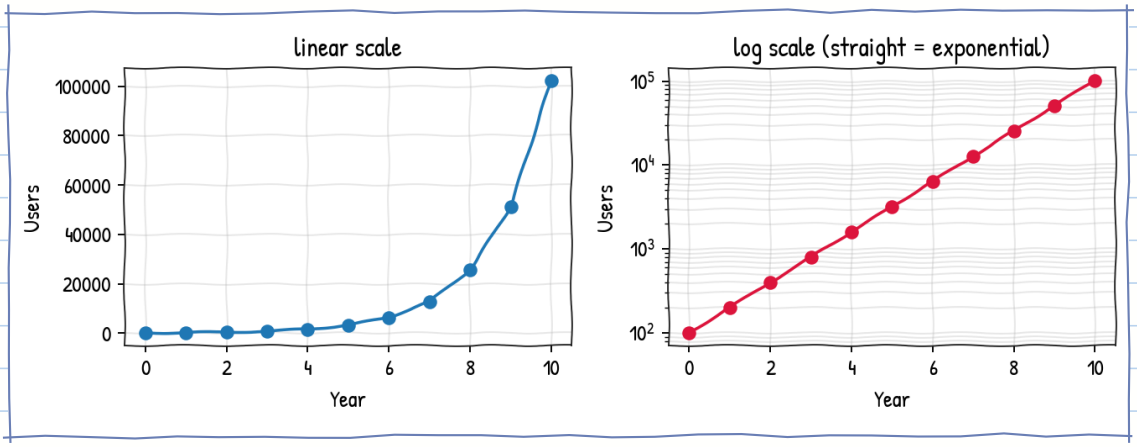


WARNING: Avoid twin axes when you can The two scales are arbitrary, so you can make any two lines look correlated just by stretching an axis. Prefer two stacked panels with *sharex=True*, and always colour-match each axis to its line.

A.3 Logarithmic scales

Concept: A log axis spaces values by ratio (1, 10, 100, 1000 equally apart). **Why:** data spanning several orders of magnitude (populations, prices, growth) or exponential growth that becomes a straight line.
Syntax: *ax.set_yscale('log')*, *ax.set_xscale('log')*, or *plt.semilogy()*, *plt.semilogx()*, *plt.loglog()*

```
import numpy as np, matplotlib.pyplot as plt
t = np.arange(0, 11); users = 100 * 2.0 ** t
fig, (a, b) = plt.subplots(1, 2, figsize=(8, 3))
a.plot(t, users, 'o-'); a.set_title('linear scale')
b.plot(t, users, 'o-', color='crimson'); b.set_yscale('log'); b.set_title('log scale (straight = exponential)')
for ax in (a, b): ax.set_xlabel('Year'); ax.set_ylabel('Users'); ax.grid(alpha=.3, which='both')
plt.tight_layout(); plt.show()
```



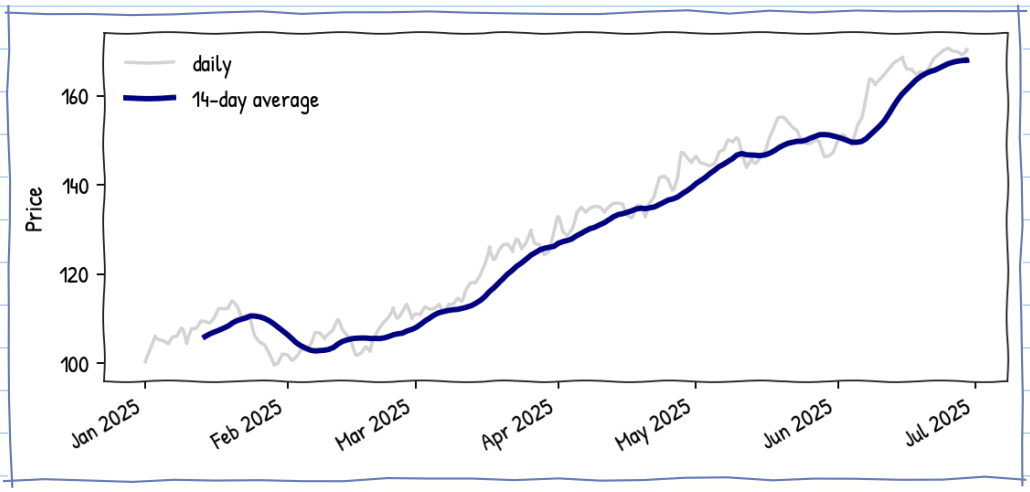
COMMON MISTAKE: Log scales cannot show zero or negative numbers, and non-technical readers misread them. **Say "log scale" in the axis label** when you use one.

A.4 Dates and time-series visualization

Concept: When x-values are real dates (*datetime64*), Matplotlib spaces them by true time and offers date locators/formatters. **Why:** trends, seasonality, and events over time.

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt
import matplotlib.dates as mdates
rng = np.random.default_rng(11)
dates = pd.date_range('2025-01-01', periods=180, freq='D')
price = pd.Series(100 + rng.normal(0.3, 2, 180).cumsum(), index=dates)
fig, ax = plt.subplots(figsize=(7.5, 3.3))
ax.plot(price.index, price, color='lightgray', label='daily')
ax.plot(price.index, price.rolling(14).mean(), color='navy', lw=2.5, label='14-day average')
ax.xaxis.set_major_locator(mdates.MonthLocator())
```

```
ax.xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))
ax.set_ylabel('Price'); ax.legend(); fig.autofmt_xdate()
plt.show()
```

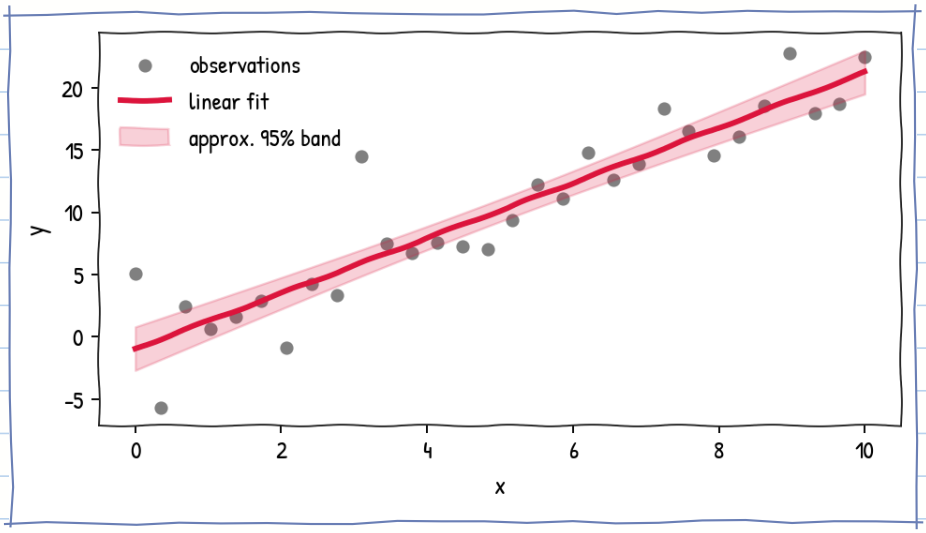


COMMON MISTAKE: Dates stored as ***strings*** are treated as categories (equal spacing, every label drawn). Convert first: `df['date'] = pd.to_datetime(df['date'])`.

A.5 Error bars and uncertainty intervals

Concept: Besides *errorbar*, a shaded band with *fill_between* shows a confidence or prediction interval around a trend line.

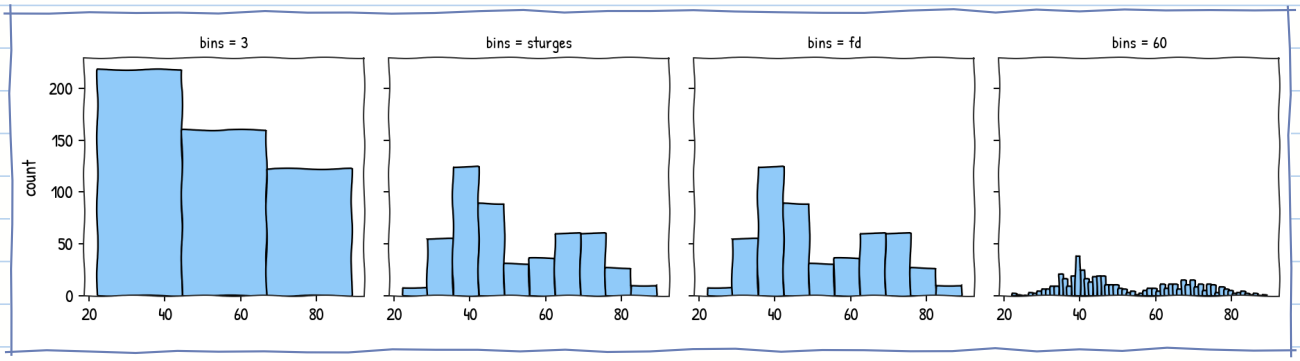
```
import numpy as np, matplotlib.pyplot as plt
rng = np.random.default_rng(3)
x = np.linspace(0, 10, 30)
y = 2 * x + rng.normal(0, 2.5, 30)
coef = np.polyfit(x, y, 1); fit = np.polyval(coef, x)
se = 2.5 * np.sqrt(1 / 30 + (x - x.mean()) ** 2 / ((x - x.mean()) ** 2).sum())
fig, ax = plt.subplots(figsize=(6.5, 3.2))
ax.scatter(x, y, color='gray', s=25, label='observations')
ax.plot(x, fit, color='crimson', lw=2.5, label='linear fit')
ax.fill_between(x, fit - 1.96 * se, fit + 1.96 * se, color='crimson', alpha=.2, label='approx. 95% band')
ax.legend(); ax.set_xlabel('x'); ax.set_ylabel('y'); plt.show()
```



A.6 Histograms: binning strategies and normalization

Concept: Bin count/width changes the story. Too few bins hide structure; too many show noise. `bins` accepts a number, a list of edges or a rule name: `'auto'`, `'sturges'`, `'fd'` (Freedman-Diaconis), `'sqrt'`.

```
import numpy as np, matplotlib.pyplot as plt
rng = np.random.default_rng(4)
d = np.concatenate([rng.normal(40, 6, 300), rng.normal(70, 8, 200)])   # two groups
fig, axs = plt.subplots(1, 4, figsize=(11, 2.8), sharey=True)
for ax, b in zip(axs, [3, 'sturges', 'fd', 60]):
    ax.hist(d, bins=b, color='#90caf9', edgecolor='k'); ax.set_title(f'bins = {b}', fontsize=10)
axs[0].set_ylabel('count')
plt.tight_layout(); plt.show()
```

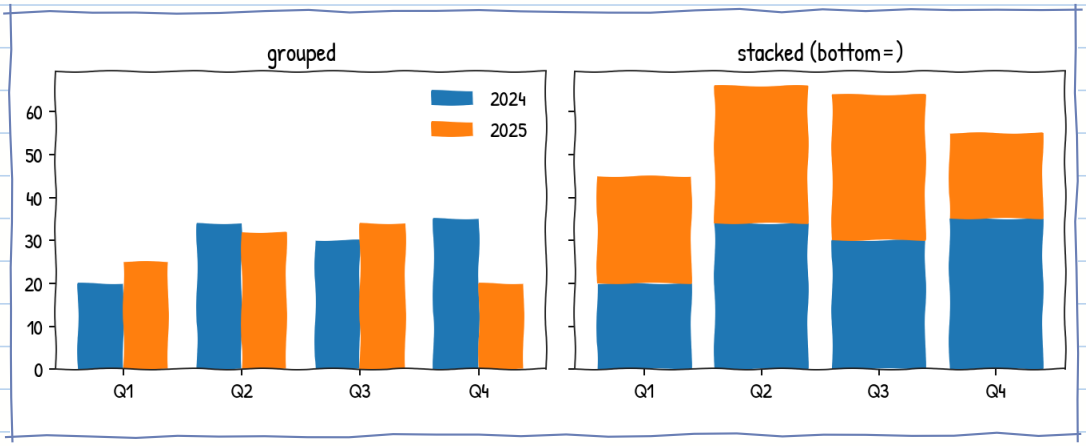


Explanation: Bimodal data (two humps) is invisible with 3 bins. **Density normalization:** `density=True` rescales so the total area is 1 - needed to compare groups of different sizes: `plt.hist(a, density=True, alpha=.5)`. `weights=` and `cumulative=True` give weighted and cumulative histograms.

A.7 Categorical data: grouped and stacked bars

```
import numpy as np, matplotlib.pyplot as plt
cats = ['Q1', 'Q2', 'Q3', 'Q4']; a = [20, 34, 30, 35]; b = [25, 32, 34, 20]
x = np.arange(len(cats)); w = 0.38
fig, (l, r) = plt.subplots(1, 2, figsize=(8, 3), sharey=True)
l.bar(x - w / 2, a, w, label='2024'); l.bar(x + w / 2, b, w, label='2025')
```

```
l.set_xticks(x, cats); l.set_title('grouped'); l.legend()
r.bar(x, a, label='2024'); r.bar(x, b, bottom=a, label='2025')
r.set_xticks(x, cats); r.set_title('stacked (bottom=)')
plt.tight_layout(); plt.show()
```



Practice: Plot two years of monthly sales with a 3-month rolling mean, add a shaded +/-1 SD band, and put one annotation on the best month.

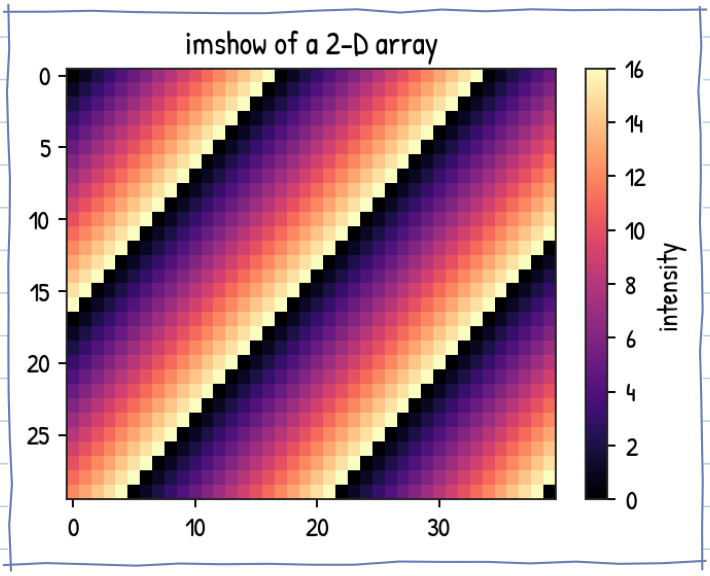
B Images, 2-D Fields, 3-D and Polar

imshow, heatmaps, contours, 3-D, polar

B.1 imshow(): images and matrices as pictures

Concept: `ax.imshow(array)` draws a 2-D array as coloured pixels - photos, matrices, grids. **Why:** the base of heatmaps, confusion matrices and image data.

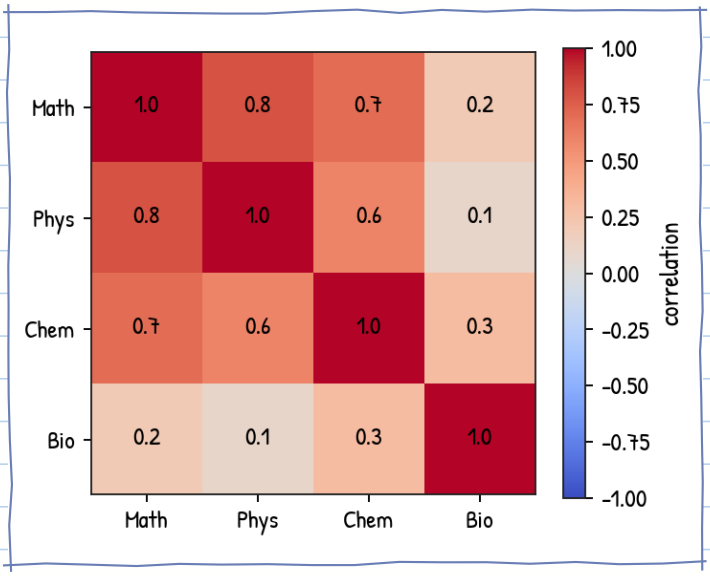
```
import numpy as np, matplotlib.pyplot as plt
img = np.add.outer(np.arange(30), np.arange(40)) % 17          # synthetic 'image'
fig, ax = plt.subplots(figsize=(4.5, 3.2))
im = ax.imshow(img, cmap='magma', origin='upper', aspect='auto')
fig.colorbar(im, ax=ax, label='intensity'); ax.set_title('imshow of a 2-D array')
plt.show()
```



Explanation: `cmap` maps numbers to colour, `origin` sets the row-0 position, `aspect='auto'` stretches pixels to fill the Axes, `vmin/vmax` fix the colour range. For real photos use `plt.imread('photo.png')`.

B.2 Heatmaps in pure Matplotlib

```
import numpy as np, matplotlib.pyplot as plt
labels = ['Math', 'Phys', 'Chem', 'Bio']
corr = np.array([[1, .8, .7, .2], [.8, 1, .6, .1], [.7, .6, 1, .3], [.2, .1, .3, 1]])
fig, ax = plt.subplots(figsize=(4.4, 3.6))
im = ax.imshow(corr, cmap='coolwarm', vmin=-1, vmax=1)
ax.set_xticks(range(4), labels); ax.set_yticks(range(4), labels)
for i in range(4):
    for j in range(4): ax.text(j, i, f'{corr[i, j]:.1f}', ha='center', va='center')
fig.colorbar(im, label='correlation'); plt.show()
```

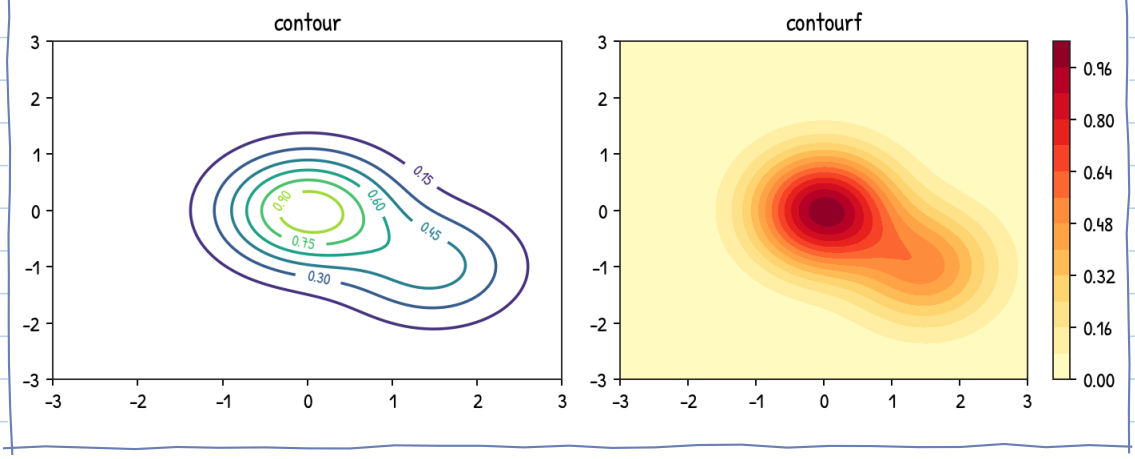


NOTE: Writing text into each cell by hand takes a loop - Seaborn's *sns.heatmap(annot=True)* does it in one line (Part V).

B.3 Contour plots

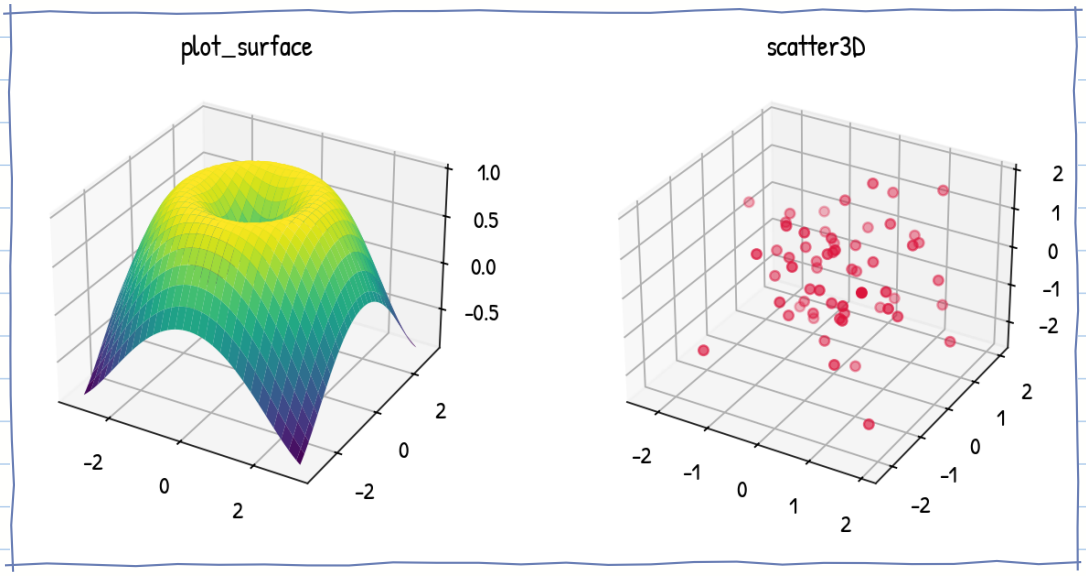
Concept: Contours join points of equal height z on an x-y grid, like a topographic map. *contour* draws lines, *contourf* fills the regions. **Why:** 3-variable relationships, loss surfaces, density maps.

```
import numpy as np, matplotlib.pyplot as plt
x = np.linspace(-3, 3, 120); y = np.linspace(-3, 3, 120)
X, Y = np.meshgrid(x, y)
Z = np.exp(-(X ** 2 + Y ** 2)) + 0.5 * np.exp(-((X - 1.5) ** 2 + (Y + 1) ** 2))
fig, (a, b) = plt.subplots(1, 2, figsize=(8.5, 3.3))
cs = a.contour(X, Y, Z, levels=8, cmap='viridis'); a.clabel(cs, fontsize=8); a.set_title('contour')
cf = b.contourf(X, Y, Z, levels=12, cmap='YlOrRd'); fig.colorbar(cf, ax=b); b.set_title('contourf')
plt.tight_layout(); plt.show()
```



B.4 3-D plotting with mpl_toolkits.mplot3d

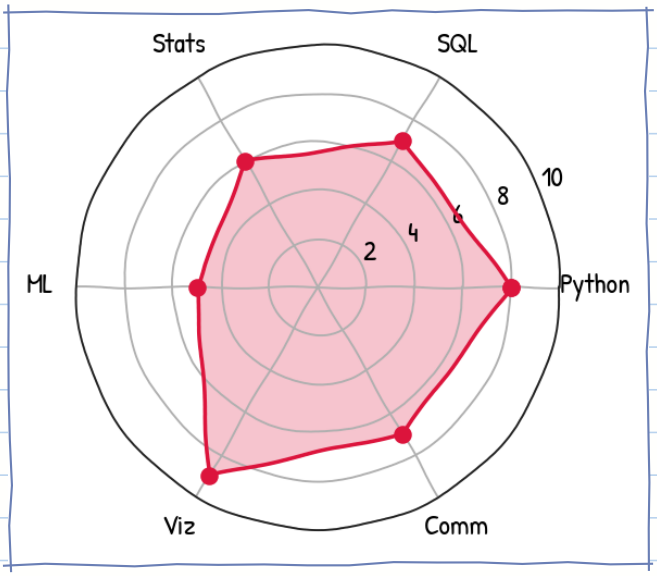
```
import numpy as np, matplotlib.pyplot as plt
x = np.linspace(-3, 3, 60); X, Y = np.meshgrid(x, x)
Z = np.sin(np.sqrt(X ** 2 + Y ** 2))
fig = plt.figure(figsize=(7.5, 3.4))
a = fig.add_subplot(1, 2, 1, projection='3d')
a.plot_surface(X, Y, Z, cmap='viridis', edgecolor='none'); a.set_title('plot_surface')
rng = np.random.default_rng(0)
b = fig.add_subplot(1, 2, 2, projection='3d')
b.scatter(rng.normal(size=60), rng.normal(size=60), rng.normal(size=60), c='crimson'); b.set_title('scatter3D')
plt.tight_layout(); plt.show()
```



WARNING: Avoid 3-D charts for ordinary data. Perspective distorts size and hides values. Use 3-D only for true 3-variable surfaces; for bars/pies it is chart-junk.

B.5 Polar plots

```
import numpy as np, matplotlib.pyplot as plt
skills = ['Python', 'SQL', 'Stats', 'ML', 'Viz', 'Comm']
vals = [8, 7, 6, 5, 9, 7]
ang = np.linspace(0, 2 * np.pi, len(skills), endpoint=False)
fig = plt.figure(figsize=(4, 3.4)); ax = fig.add_subplot(projection='polar')
ax.plot(np.r_[ang, ang[0]], vals + vals[:1], 'o-', color='crimson')
ax.fill(np.r_[ang, ang[0]], vals + vals[:1], alpha=.25, color='crimson')
ax.set_xticks(ang, skills); ax.set_ylim(0, 10); plt.show()
```



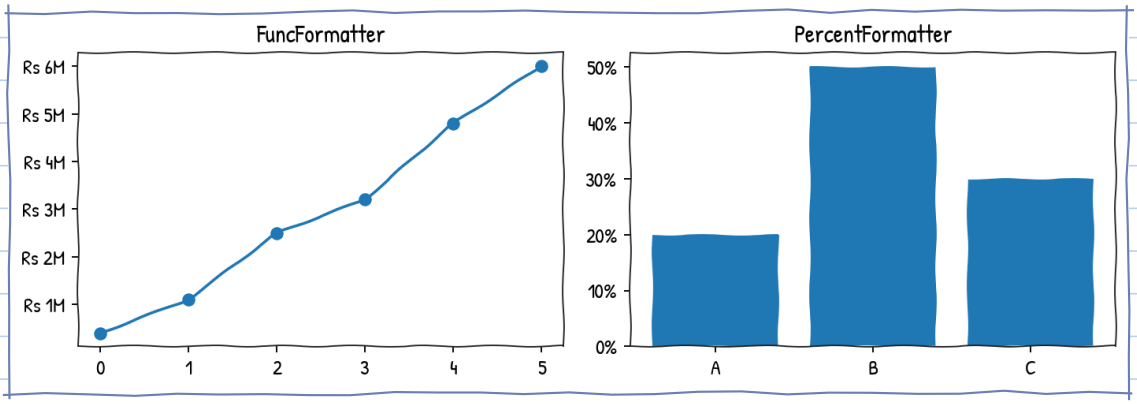
PRO TIP: Polar/radar charts suit cyclic data (hours of day, wind direction, months). For skill comparison a sorted bar is easier to read.

C Layouts, Formatting & Production

ticks, LaTeX, GridSpec, insets, export, performance, accessibility

C.1 Custom ticks and formatters

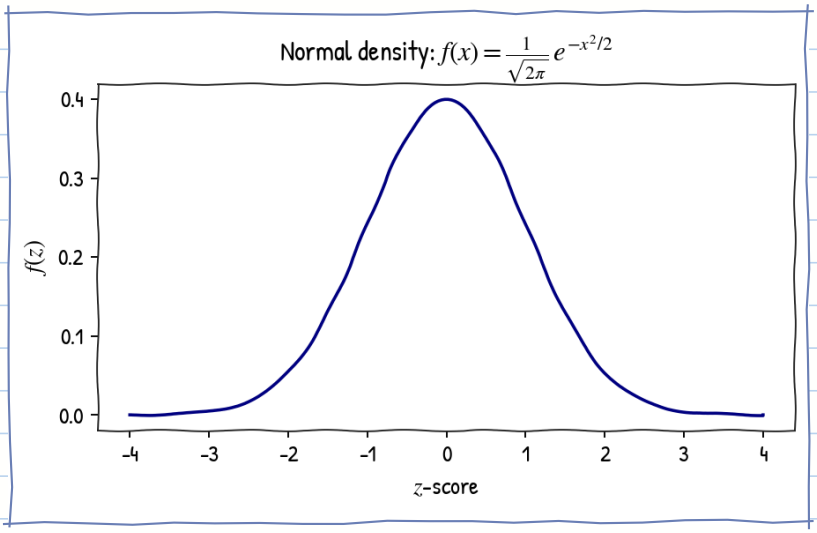
```
import numpy as np, matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter, MultipleLocator, PercentFormatter
x = np.arange(0, 6); rev = np.array([0.4e6, 1.1e6, 2.5e6, 3.2e6, 4.8e6, 6.0e6])
fig, (a, b) = plt.subplots(1, 2, figsize=(8.5, 3))
a.plot(x, rev, 'o-'); a.yaxis.set_major_formatter(FuncFormatter(lambda v, p: f'Rs {v / 1e6:.0f}M'))
a.xaxis.set_major_locator(MultipleLocator(1)); a.set_title('FuncFormatter')
b.bar(['A', 'B', 'C'], [0.2, 0.5, 0.3]); b.set_title('PercentFormatter')
b.yaxis.set_major_formatter(PercentFormatter(1.0))
plt.tight_layout(); plt.show()
```



C.2 Mathematical notation (LaTeX-style labels)

Syntax: Wrap maths in dollar signs inside a raw string: `r'$\alpha^2 + \beta_i$'`. Matplotlib's built-in mathtext needs no LaTeX installation.

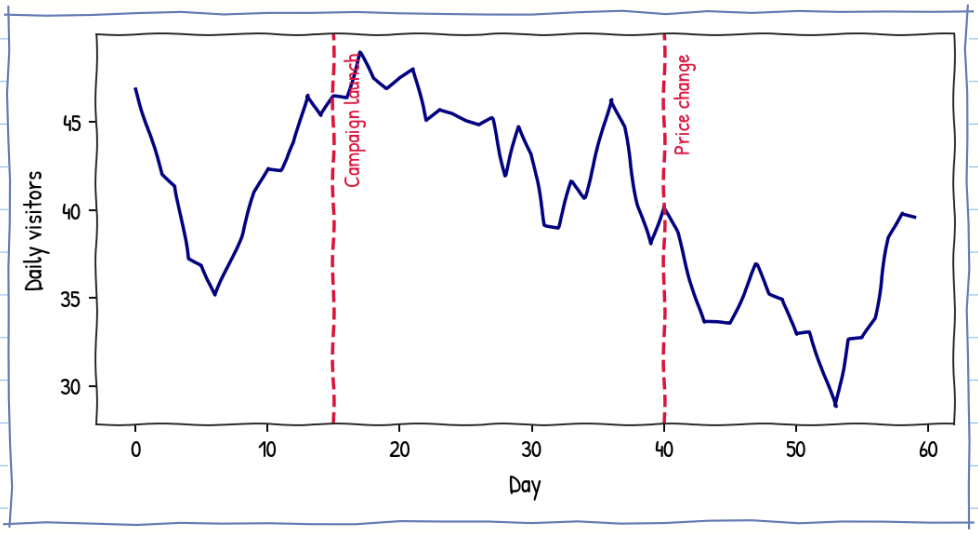
```
import numpy as np, matplotlib.pyplot as plt
x = np.linspace(-4, 4, 200)
plt.figure(figsize=(6, 3))
plt.plot(x, np.exp(-x ** 2 / 2) / np.sqrt(2 * np.pi), color='navy')
plt.title(r'Normal density: $f(x)=\frac{1}{\sqrt{2\pi}}\,e^{-x^2/2}$', fontsize=13)
plt.xlabel(r'$z$-score'); plt.ylabel(r'$f(z)$'); plt.show()
```



COMMON MISTAKE: Forgetting the leading `r` (raw string) turns `\f` or `\t` into form-feed/tab characters and garbles the formula.

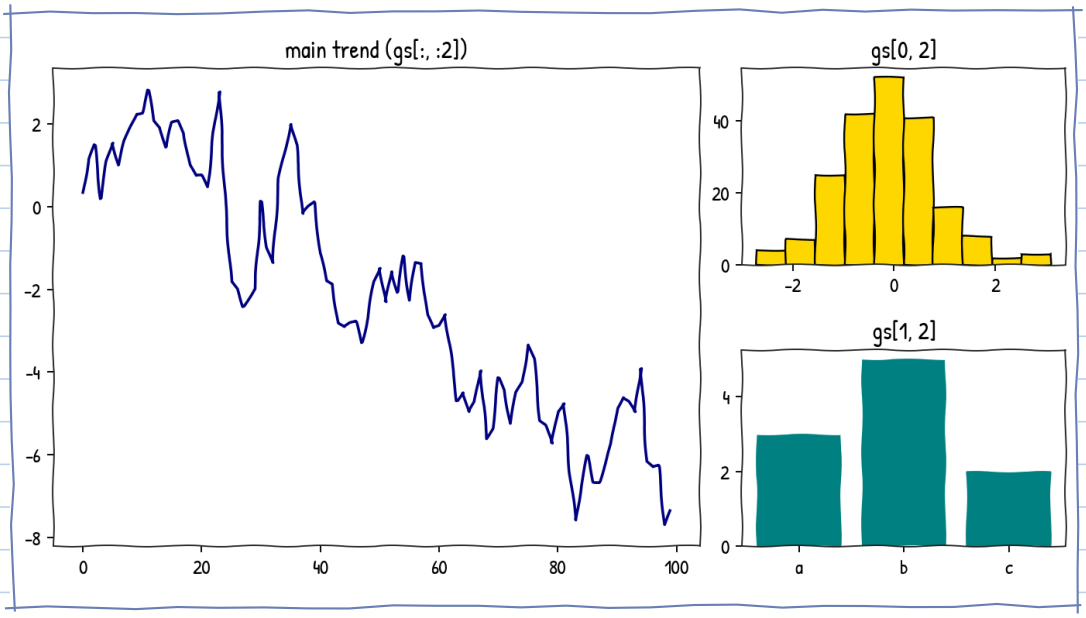
C.3 Event markers and annotations

```
import numpy as np, matplotlib.pyplot as plt
rng = np.random.default_rng(8); d = np.arange(60); v = 50 + rng.normal(0, 3, 60).cumsum() * .6
fig, ax = plt.subplots(figsize=(7, 3.2))
ax.plot(d, v, color='navy')
for day, txt in [(15, 'Campaign launch'), (40, 'Price change')]:
    ax.axvline(day, color='crimson', ls='--')
    ax.text(day + .8, v.max(), txt, color='crimson', rotation=90, va='top', fontsize=10)
ax.set_xlabel('Day'); ax.set_ylabel('Daily visitors'); plt.show()
```



C.4 Complex layouts with GridSpec

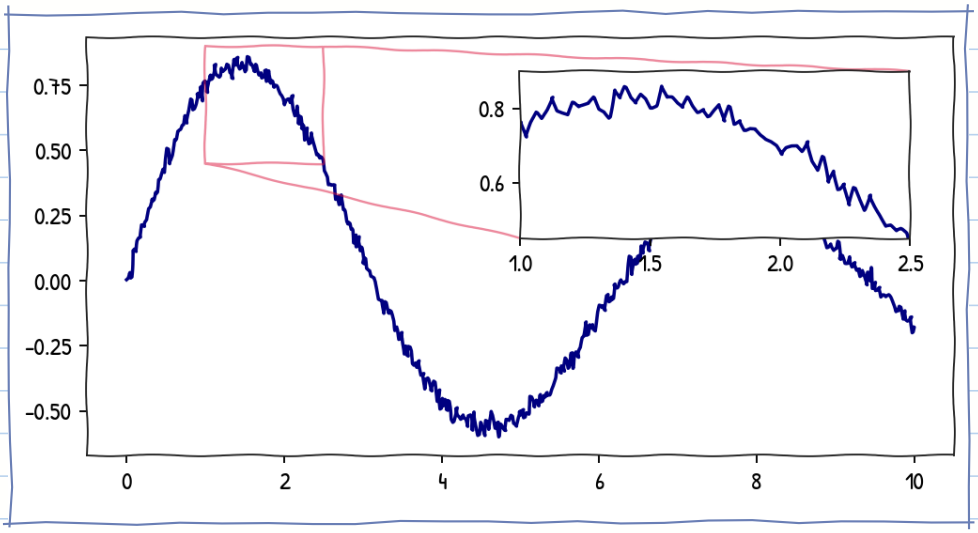
```
import numpy as np, matplotlib.pyplot as plt
rng = np.random.default_rng(1)
fig = plt.figure(figsize=(8, 4.2), layout='constrained')
gs = fig.add_gridspec(2, 3)                    # 2 rows x 3 columns
big = fig.add_subplot(gs[:, :2])               # spans both rows, first 2 columns
big.plot(np.cumsum(rng.normal(size=100)), color='navy'); big.set_title('main trend (gs[:, :2])')
t = fig.add_subplot(gs[0, 2]); t.hist(rng.normal(size=200), color='gold', edgecolor='k'); t.set_title('gs[0, 2]')
b = fig.add_subplot(gs[1, 2]); b.bar(['a', 'b', 'c'], [3, 5, 2], color='teal'); b.set_title('gs[1, 2]')
plt.show()
```



Explanation: GridSpec slices work like NumPy slices: `gs[:, :2]` = all rows, first two columns. Alternatives: `plt.subplot_mosaic('AAB;AAC')` draws the same layout from a text picture.

C.5 Insets and zoomed views

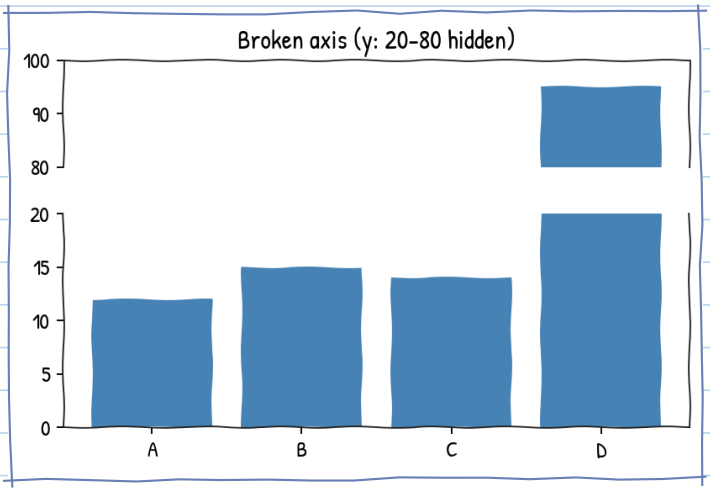
```
import numpy as np, matplotlib.pyplot as plt
rng = np.random.default_rng(2)
x = np.linspace(0, 10, 500); y = np.sin(x) * np.exp(-x / 8) + rng.normal(0, .02, 500)
fig, ax = plt.subplots(figsize=(7, 3.4))
ax.plot(x, y, color='navy')
ins = ax.inset_axes([0.5, 0.52, 0.45, 0.4])
ins.plot(x, y, color='navy'); ins.set_xlim(1, 2.5); ins.set_ylim(0.45, 0.9)
ax.indicate_inset_zoom(ins, edgecolor='crimson')
plt.show()
```



C.6 Broken axes - and why to be careful

Concept: A broken axis skips a range so tiny and huge values can share one chart. It is built from two Axes with hidden spines.

```
import matplotlib.pyplot as plt
names = ['A', 'B', 'C', 'D']; vals = [12, 15, 14, 95]
fig, (top, bot) = plt.subplots(2, 1, sharex=True, figsize=(5, 3.4), gridspec_kw={'height_ratios': [1, 2]})
for ax in (top, bot): ax.bar(names, vals, color='steelblue')
top.set_ylim(80, 100); bot.set_ylim(0, 20)
top.spines.bottom.set_visible(False); bot.spines.top.set_visible(False)
top.tick_params(bottom=False)
top.set_title('Broken axis (y: 20-80 hidden)')
plt.tight_layout(); plt.show()
```



WARNING: Misleading-scale alert: broken axes hide how big the difference really is (D looks only ~5x bigger than C but is ~7x). Mark the break clearly, or better, use a log scale or show D separately.

C.7 Object-oriented best practices and reusable functions

```
import numpy as np, matplotlib.pyplot as plt

def line_panel(ax, x, series, title, xlabel, ylabel, colors=None):
    """Draw several named series on a given Axes with house styling."""
    for i, (name, y) in enumerate(series.items()):
        ax.plot(x, y, lw=2.2, label=name, color=None if colors is None else colors[i])
    ax.set(title=title, xlabel=xlabel, ylabel=ylabel)        # set many properties at once
    ax.spines[['top', 'right']].set_visible(False)
    ax.grid(axis='y', alpha=.25); ax.legend(frameon=False)
    return ax

x = np.arange(12)
fig, (a, b) = plt.subplots(1, 2, figsize=(9, 3.2))
line_panel(a, x, {'A': x * 2, 'B': x * 1.5}, 'Sales', 'Month', 'k$')
line_panel(b, x, {'C': x ** 1.3}, 'Costs', 'Month', 'k$')
fig.tight_layout()
```

PRO TIP: Pass the **Axes in** and **return it** from helper functions. That makes them reusable in any layout, and keeps one place to change your styling.

C.8 Exporting high-quality figures

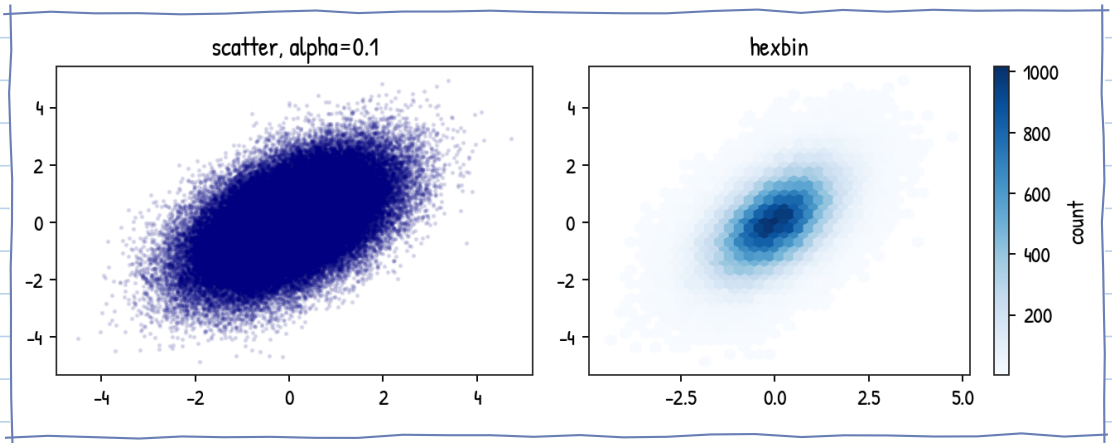
Destination	Format	Settings
Research paper / LaTeX	PDF or SVG (vector)	`figsize` = journal column width (about 3.5 in); fonts >= 8 pt
PowerPoint / Word	PNG	`dpi=200-300`, `bbox_inches='tight'`, white background
Website	SVG or PNG	`dpi=150`; compress; give alt-text
Dark slides	PNG transparent	`transparent=True` (check text is readable)

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(3.5, 2.4))        # journal single-column width
ax.plot([1, 2, 3], [3, 1, 2]); ax.set_xlabel('x'); ax.set_ylabel('y')
fig.savefig('figure1.pdf', bbox_inches='tight')              # vector for papers
fig.savefig('figure1.png', dpi=300, bbox_inches='tight')     # raster for slides
```

C.9 Plotting large datasets and performance

Problem	Fix
200,000 scatter points = a solid blob, slow	`ax.hexbin()`, `plt.hist2d()`, or `alpha=.05, s=2`
Huge line (millions of points)	downsample / resample, `plt.rcParams['path.simplify_threshold']`
Slow PDF with many marks	`rasterized=True` on the heavy artist
Many figures in a loop use RAM	`plt.close(fig)` after saving
Interactive zoom needed	Plotly / Bokeh / Datashader (beyond Matplotlib)

```
import numpy as np, matplotlib.pyplot as plt
rng = np.random.default_rng(0)
x = rng.normal(size=100_000); y = x * .6 + rng.normal(size=100_000)
fig, (a, b) = plt.subplots(1, 2, figsize=(8, 3))
a.scatter(x, y, s=2, alpha=.1, color='navy', rasterized=True); a.set_title('scatter, alpha=0.1')
hb = b.hexbin(x, y, gridsize=35, cmap='Blues', mincnt=1); b.set_title('hexbin')
fig.colorbar(hb, ax=b, label='count')
plt.tight_layout(); plt.show()
```

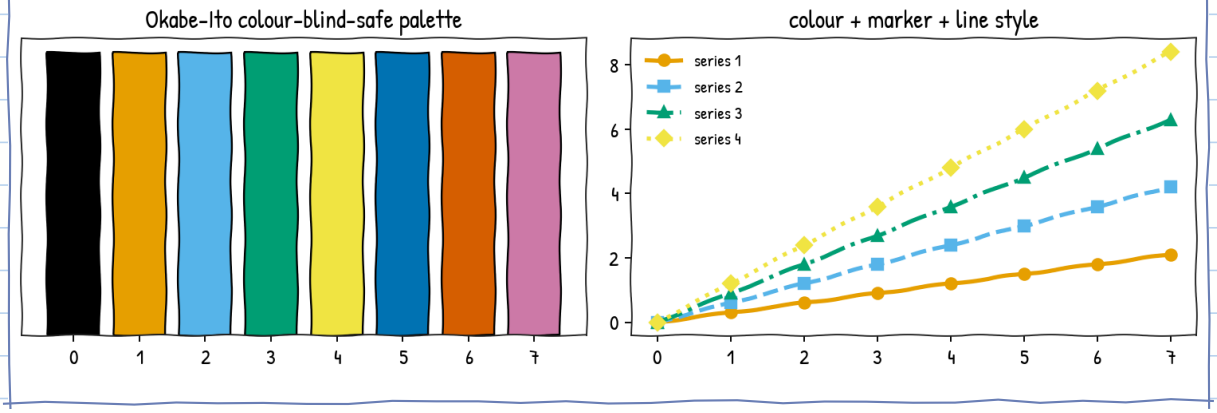


C.10 Accessibility, contrast and colour-blind-friendly palettes

Concept: About 1 in 12 men has colour-vision deficiency. Never rely on colour alone: add **markers, line styles or direct labels**, and ensure strong contrast with the background.

```
import numpy as np, matplotlib.pyplot as plt
okabe_ito = ['#000000', '#E69F00', '#56B4E9', '#009E73', '#F0E442', '#0072B2', '#D55E00', '#CC79A7']
x = np.arange(8); mk = ['o', 's', '^', 'D']; ls = ['-', '--', '-.', ':']
fig, (a, b) = plt.subplots(1, 2, figsize=(9, 3))
a.bar(x, np.ones(8), color=okabe_ito, edgecolor='k'); a.set_yticks([])
a.set_title('Okabe-Ito colour-blind-safe palette')
for i in range(4):
    b.plot(x, x * (i + 1) * .3, ls[i], marker=mk[i], color=okabe_ito[i + 1], lw=2.2, label=f'series {i + 1}')
```

```
b.legend(fontsize=9); b.set_title('colour + marker + line style')
plt.tight_layout(); plt.show()
```



PRO TIP: Safe built-in options: colormap *viridis*, Seaborn palette '*colorblind*', and the Okabe-Ito list above. Test your chart in greyscale - if it is unreadable there, it is not accessible.

Practice: Build a 2 x 2 figure with GridSpec: a time series (with date axis), a log-scale plot, a hexbin of 50k points, and an inset zoom. Export it as PDF and PNG.

P4 Seaborn Fundamentals

themes, relational, distribution and categorical plots

Seaborn sits on top of Matplotlib. You give it a tidy Pandas DataFrame and column names; it handles grouping, aggregation, colours, legends and (often) uncertainty for you.

S Seaborn Foundations

install, data, themes, figure- vs axes-level

S.1 Purpose, installation and built-in datasets

Concept: Seaborn = ***statistical*** plots + attractive defaults + DataFrame awareness. Install with `pip install seaborn`; import as `sns`.

```
import seaborn as sns
tips = sns.load_dataset('tips')        # downloads a CSV from GitHub on first use
print(sns.get_dataset_names()[:6])
print(tips.head())
```

WARNING: !!`sns.load_dataset()` needs internet!! (it fetches from the seaborn-data repository, then caches). Offline, in an exam or behind a firewall it raises a connection error. **Solution:** build your own sample DataFrame.

Example: These notes use two synthetic datasets with the same column names as Seaborn's `tips` and `penguins`. Save this once as `sample_data.py` next to your notebook and import it:

```
import numpy as np
import pandas as pd

def make_tips(n=244, seed=42):
    """Synthetic restaurant bills (shaped like seaborn's 'tips')."""
    rng = np.random.default_rng(seed)
    day = rng.choice(['Thur', 'Fri', 'Sat', 'Sun'], n, p=[.25, .08, .37, .30])
    late = rng.choice(['Lunch', 'Dinner'], n, p=[.8, .2])
    time = np.where(np.isin(day, ['Sat', 'Sun']), 'Dinner', late)
    size = rng.choice([1, 2, 3, 4, 5, 6], n, p=[.02, .62, .15, .15, .04, .02])
    wiggle = rng.uniform(.7, 1.3, n)
    bill = (5 + 6.5 * size * wiggle + rng.gamma(2, 2, n)).round(2)
    tip = (bill * rng.normal(.16, .04, n)).clip(1).round(2)
```

```
    return pd.DataFrame({
        'total_bill': bill, 'tip': tip,
        'sex': rng.choice(['Male', 'Female'], n, p=[.64, .36]),
        'smoker': rng.choice(['Yes', 'No'], n, p=[.38, .62]),
        'day': pd.Categorical(day, ['Thur', 'Fri', 'Sat', 'Sun']),
        'time': time, 'size': size})

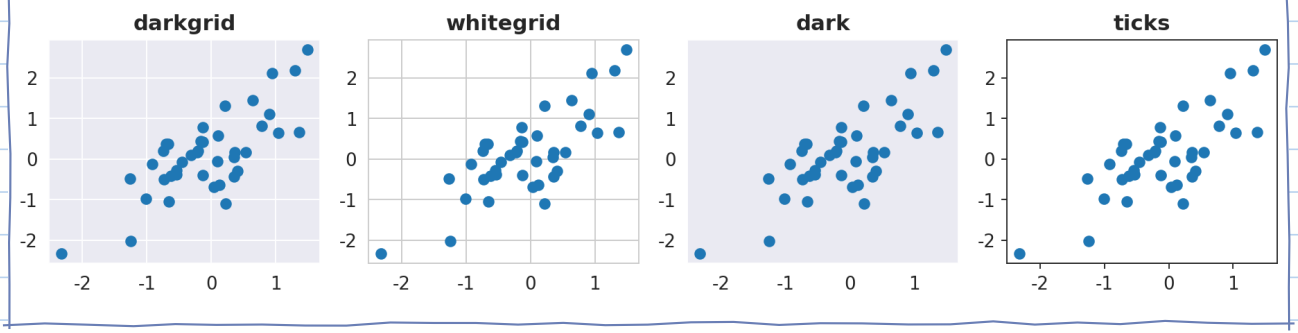
def make_penguins(n=300, seed=7):
    """Synthetic penguins: 3 species, a few missing values."""
    rng = np.random.default_rng(seed)
    sp = rng.choice(['Adelie', 'Chinstrap', 'Gentoo'], n, p=[.44, .2, .36])
    P = {'Adelie': (38.8, 18.3, 190, 3700),
         'Chinstrap': (48.8, 18.4, 196, 3730),
         'Gentoo': (47.5, 15.0, 217, 5075)}
    m = np.array([P[s] for s in sp], dtype=float)
    df = pd.DataFrame({
        'species': sp,
        'island': rng.choice(['Biscoe', 'Dream', 'Torgersen'], n),
        'bill_length_mm': m[:, 0] + rng.normal(0, 2.7, n),
        'bill_depth_mm': m[:, 1] + rng.normal(0, 1.1, n),
        'flipper_length_mm': m[:, 2] + rng.normal(0, 6.5, n),
        'body_mass_g': m[:, 3] + rng.normal(0, 380, n),
        'sex': rng.choice(['Male', 'Female'], n)}).round(1)
    for c in ['bill_length_mm', 'body_mass_g']:
        df.loc[rng.choice(n, 8, replace=False), c] = np.nan
    return df
```

NOTE: Real *tips* has 244 rows; ours mimics it. The numbers differ from the real dataset, so your charts will not be pixel-identical to textbook screenshots - but every pattern and every API call is the same.

5.2 set_theme: styles, contexts and palettes

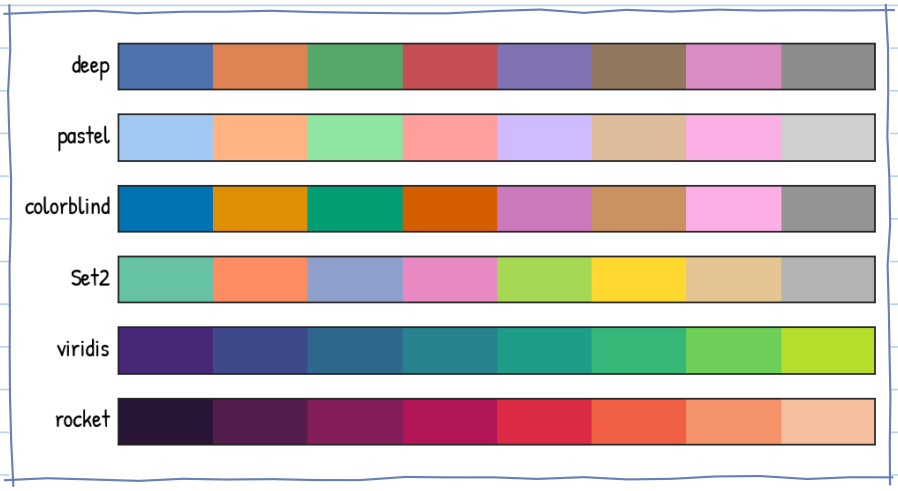
Syntax: `sns.set_theme(context='notebook', style='darkgrid', palette='deep', font_scale=1)`

```
import numpy as np, seaborn as sns, matplotlib.pyplot as plt
rng = np.random.default_rng(0); x = rng.normal(size=40); y = x + rng.normal(0, .6, 40)
fig = plt.figure(figsize=(11, 2.7))
for i, st in enumerate(['darkgrid', 'whitegrid', 'dark', 'ticks'], 1):
    with sns.axes_style(st):                    # temporary style
        ax = fig.add_subplot(1, 4, i)
    ax.scatter(x, y, color='C0'); ax.set_title(st)
plt.tight_layout(); plt.show()
```



Setting	Options	Effect
style	darkgrid whitegrid dark white ticks	background, grid, spines
context	paper < notebook < talk < poster	scales fonts and line widths for the medium
palette	deep muted pastel bright dark colorblind or any matplotlib cmap	default colours
font_scale	1.2, 1.5 ...	multiplies all font sizes

```
import seaborn as sns, matplotlib.pyplot as plt
names = ['deep', 'pastel', 'colorblind', 'Set2', 'viridis', 'rocket']
fig, axs = plt.subplots(len(names), 1, figsize=(6, 3))
for ax, n in zip(axs, names):
    ax.imshow([sns.color_palette(n, 8)], aspect='auto'); ax.set_yticks([]); ax.set_xticks([])
    ax.set_ylabel(n, rotation=0, ha='right', va='center')
plt.tight_layout(); plt.show()
```



PRO TIP: `sns.set_theme()` is **global**: it changes every later Matplotlib chart too. Use `with sns.axes_style('white'):` for a temporary change. `sns.despine()` removes the top/right spines.

S.3 Figure-level vs axes-level functions

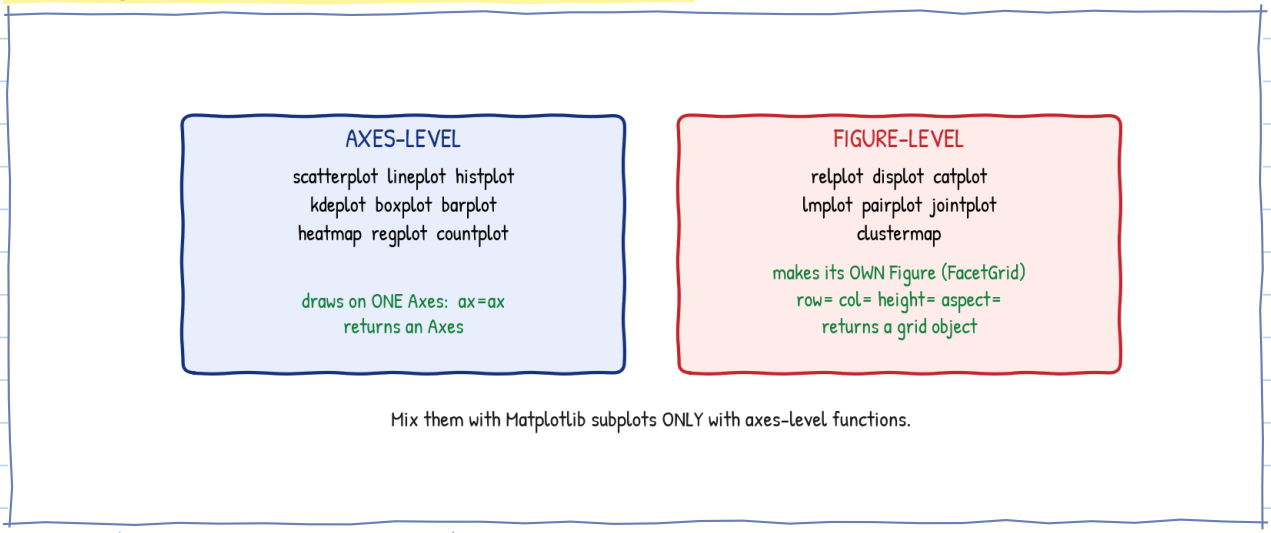


Fig S.1 - the most important Seaborn distinction.

COMMON MISTAKE: Passing *ax=ax* to a figure-level function (*sns.displot(..., ax=ax)*) is an error / ignored. Use the axes-level twin (*histplot*) inside your own subplots, or accept the grid and use *row*/*col*.

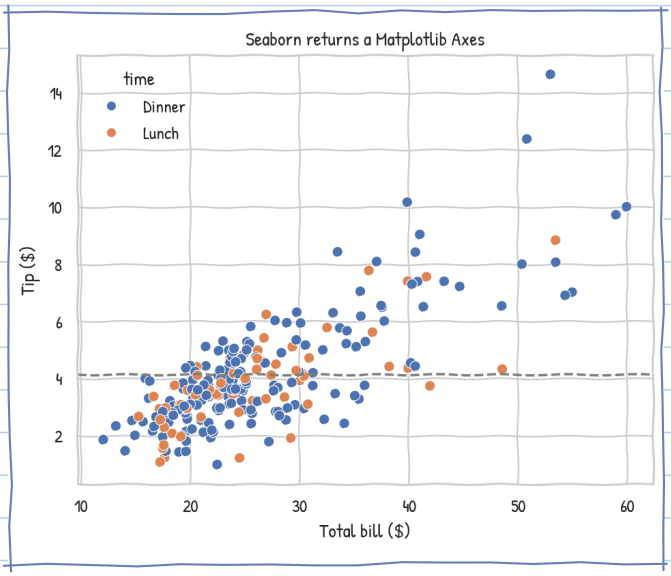
S.4 The roles of data, x, y, hue, style, size, row and col

Argument	Role	Example
data	the DataFrame (tidy: 1 row = 1 observation)	*data=tips*
x, y	columns for the axes	*x='total_bill', y='tip'*
hue	colour by a column	*hue='time'*
style	marker / line style by a column	*style='smoker'*
size	marker size / line width by a column	*size='size'*
row, col	split into small multiples (figure-level only)	*col='day'*

KEY IDEA: *Tidy data* = each variable is a column, each observation a row. If your data is "wide" (one column per year), reshape it with *df.melt(...)* before using *hue*.

```
import seaborn as sns
import matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins
tips = make_tips()
sns.set_theme(style='whitegrid')
ax = sns.scatterplot(data=tips, x='total_bill', y='tip', hue='time')
ax.set_title('Seaborn returns a Matplotlib Axes')   # ...so use Matplotlib to refine
ax.axhline(tips['tip'].mean(), color='gray', ls='--')
```

```
ax.set(xlabel='Total bill ($)', ylabel='Tip ($)')
plt.show()
```



5 Relational Plots

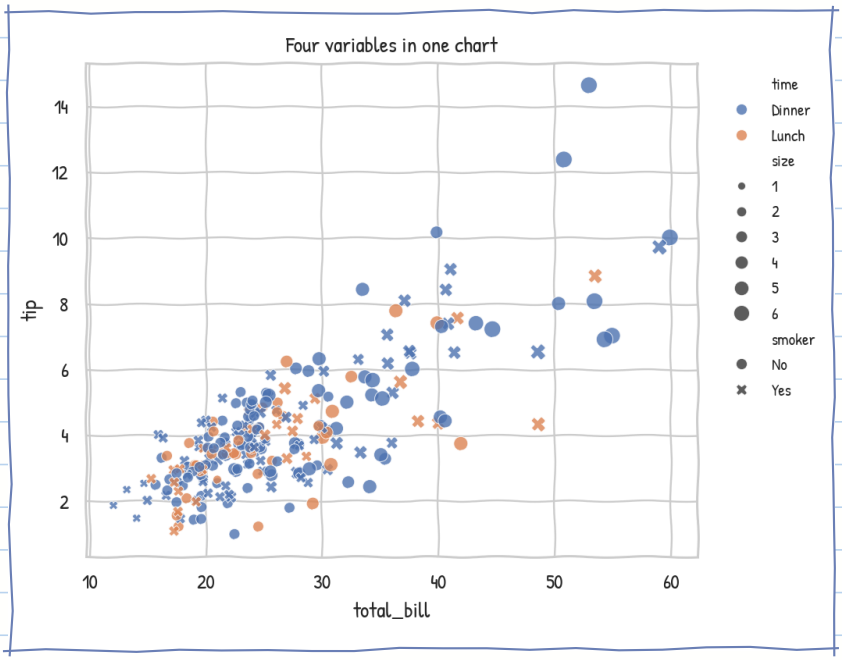
scatterplot, lineplot, relplot

5.1 scatterplot()

Concept: Dots for two numeric variables, with up to three more variables encoded by *hue*, *size* and *style*. **Why:** discover relationships and group differences.

Syntax: `sns.scatterplot(data=df, x=, y=, hue=, size=, style=, palette=, alpha=, ax=)`

```
import seaborn as sns
import matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins
tips = make_tips()
sns.set_theme(style='whitegrid')
ax = sns.scatterplot(data=tips, x='total_bill', y='tip',
                     hue='time', size='size', style='smoker', alpha=.8)
ax.legend(bbox_to_anchor=(1.02, 1), loc='upper left', fontsize=9)
ax.set_title('Four variables in one chart'); plt.tight_layout(); plt.show()
```

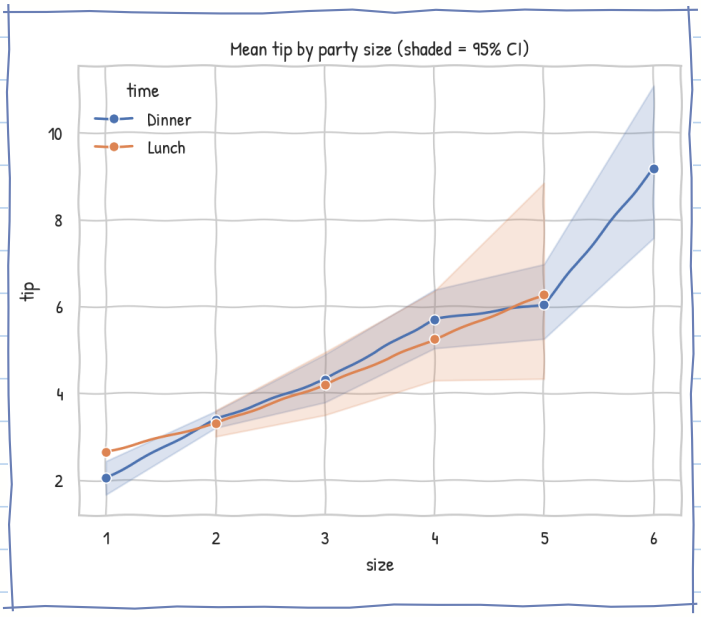


WARNING: Encoding too many variables makes charts unreadable. One chart should show **at most 2-3** mappings; split the rest into facets.

5.2 lineplot(): aggregation and confidence intervals

Concept: When there are several y values per x, *lineplot* plots their **mean** and shades an **uncertainty interval** (default: 95% bootstrap CI). It also sorts x for you.

```
import seaborn as sns
import matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins
tips = make_tips()
sns.set_theme(style='whitegrid')
ax = sns.lineplot(data=tips, x='size', y='tip', hue='time', marker='o', errorbar=('ci', 95))
ax.set_title('Mean tip by party size (shaded = 95% CI)')
plt.show()
```



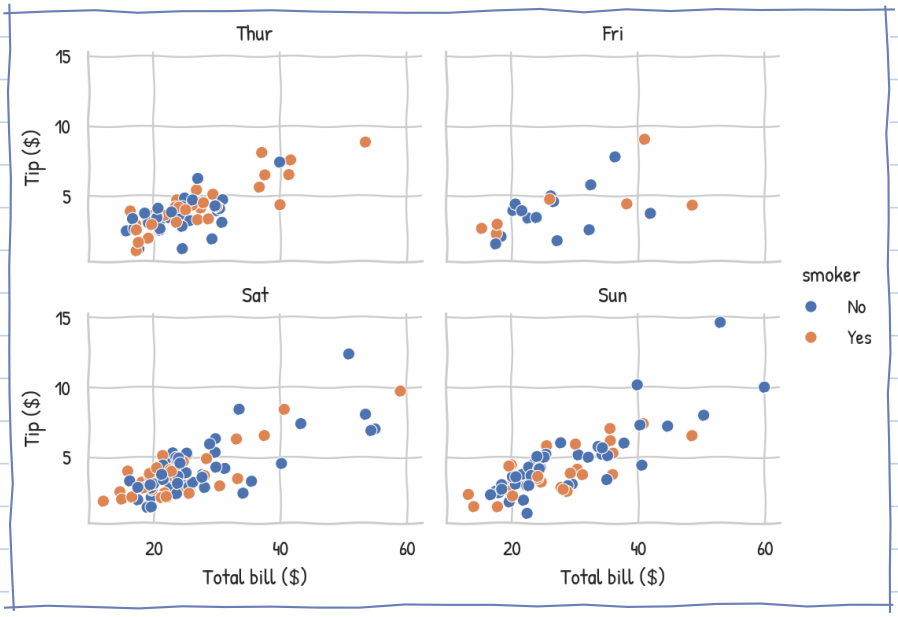
Parameter	Meaning
errorbar=('ci', 95)	bootstrap confidence interval (default); *('ci', 99)* wider
errorbar='sd' / *'se'*	standard deviation / standard error band
errorbar=None	no band (faster)
estimator='median' / *None*	other aggregate / plot every row raw
sort=False	keep data order (for paths/trajectories)

NOTE: Version trap: before Seaborn 0.12 the argument was *ci=95* (and *ci='sd'*). In 0.13 *ci=* is deprecated - use *errorbar=*. Likewise *sns.distplot*, *sns.factorplot* and *sns.lvplot* are removed/deprecated.

5.3 relplot(), faceting by row and col

Concept: relplot(kind='scatter'|'line') is the figure-level version. col= and row= create **small multiples**: one panel per category on identical axes.

```
import seaborn as sns
import matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins
tips = make_tips()
sns.set_theme(style='whitegrid')
g = sns.relplot(data=tips, x='total_bill', y='tip', hue='smoker',
                col='day', col_wrap=2, height=2.3, aspect=1.3)
g.set_axis_labels('Total bill ($)', 'Tip ($)'); g.set_titles('{col_name}')
plt.show()
```



Choose	when
scatterplot	each row is an independent observation; x is NOT ordered
lineplot	x is ordered (time, dose, size) and you want a trend / mean
relplot	you need facets (row/col) or an own figure of fixed height/aspect

Practice: Using make_tips(): (1) scatter total_bill vs tip coloured by day; (2) line plot of mean tip by size with errorbar='sd'; (3) a relplot faceted by sex x time.

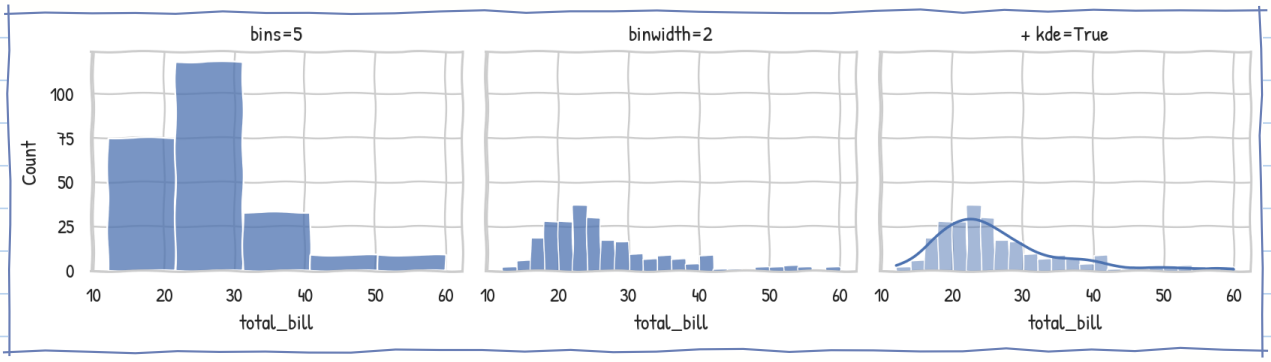
6 Distribution Plots

histplot, kdeplot, ecdfplot, displot, rugplot

Distribution plots answer: what values does this variable take, and how often? Always look at distributions ***before*** computing means or fitting models.

6.1 histplot(): bins, bin widths and stat

```
import seaborn as sns
import matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins
tips = make_tips()
sns.set_theme(style='whitegrid')
fig, axs = plt.subplots(1, 3, figsize=(10, 2.8), sharey=True)
sns.histplot(tips, x='total_bill', bins=5, ax=axs[0]);  axs[0].set_title('bins=5')
sns.histplot(tips, x='total_bill', binwidth=2, ax=axs[1]); axs[1].set_title('binwidth=2')
sns.histplot(tips, x='total_bill', binwidth=2, kde=True, ax=axs[2]); axs[2].set_title('+ kde=True')
plt.tight_layout(); plt.show()
```

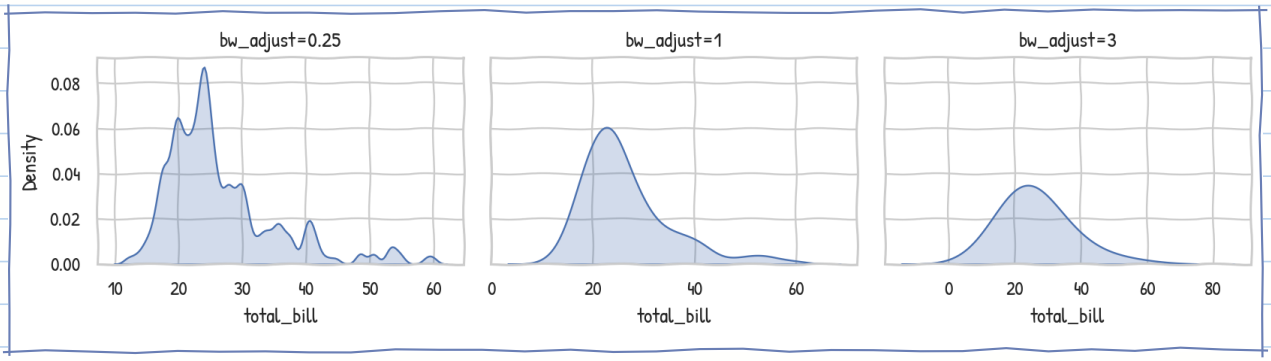


stat=	y-axis shows
'count' (default)	number of rows per bin
'frequency'	count / bin width
'probability' / 'percent'	share of all data (sums to 1 / 100)
'density'	area of all bars = 1 (use to compare unequal group sizes)

6.2 kdeplot(): smooth densities and bandwidth

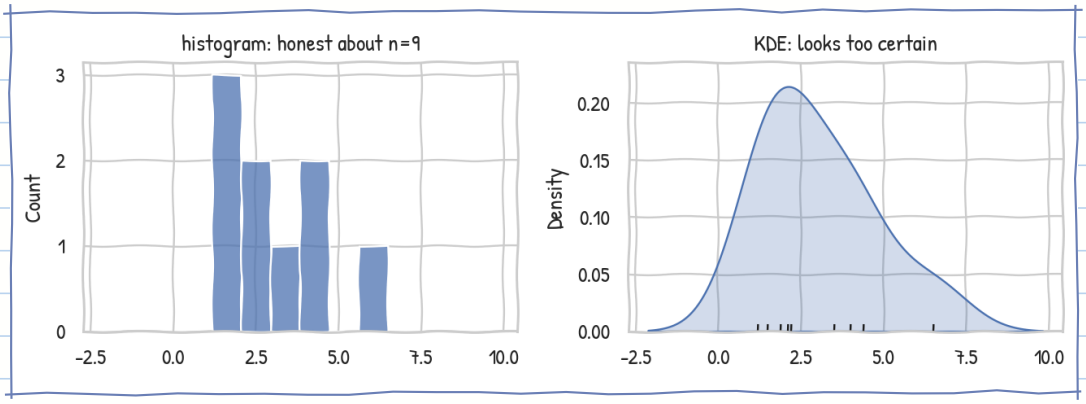
Concept: A ***kernel density estimate*** places a small smooth bump on every observation and adds them up. The ***bandwidth*** controls smoothness: `bw_adjust` < 1 = wiggly, > 1 = smoother.

```
import seaborn as sns
import matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins
tips = make_tips()
sns.set_theme(style='whitegrid')
fig, axs = plt.subplots(1, 3, figsize=(10, 2.7), sharey=True)
for ax, bw in zip(axs, [0.25, 1, 3]):
    sns.kdeplot(tips, x='total_bill', bw_adjust=bw, fill=True, ax=ax); ax.set_title(f'bw_adjust={bw}')
plt.tight_layout(); plt.show()
```



WARNING: When KDE misleads: with very few points, KDE invents smooth structure; it extends past the data range (negative prices, ages below 0) unless you set `cut=0`; and on bounded or discrete data it blurs. ***Use a histogram*** (or ECDF) for small samples, counts and bounded variables.

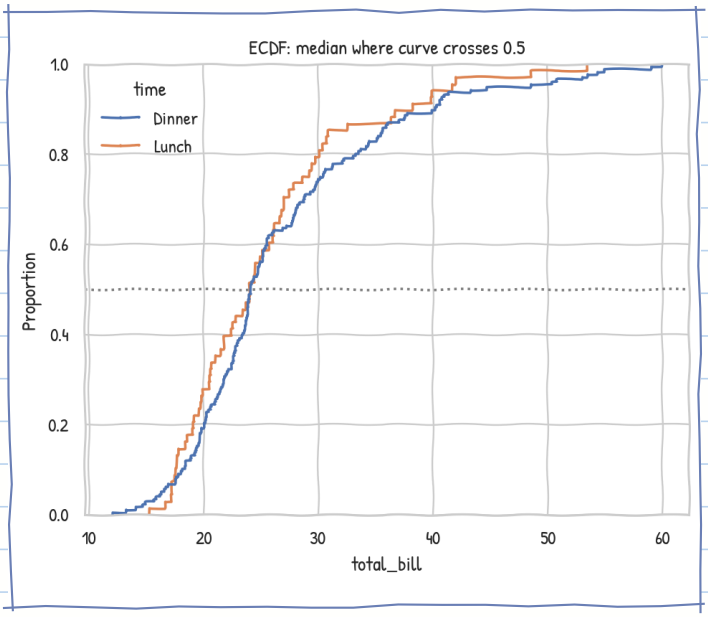
```
import seaborn as sns
import matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins
tips = make_tips()
sns.set_theme(style='whitegrid')
import numpy as np
vals = np.array([1.2, 1.5, 1.9, 2.1, 2.2, 3.5, 4.0, 4.4, 6.5])      # only 9 points
fig, (a, b) = plt.subplots(1, 2, figsize=(8, 2.8), sharex=True)
sns.histplot(x=vals, bins=6, ax=a); a.set_title('histogram: honest about n=9')
sns.kdeplot(x=vals, fill=True, ax=b); sns.rugplot(x=vals, ax=b, color='k'); b.set_title('KDE: looks too certain')
plt.tight_layout(); plt.show()
```



6.3 ecdfplot() and cumulative distributions

Concept: The **empirical CDF** shows, for each value x, the share of observations <= x. It needs no bins or bandwidth, shows every data point and makes percentiles easy to read: where the curve crosses 0.5 is the median.

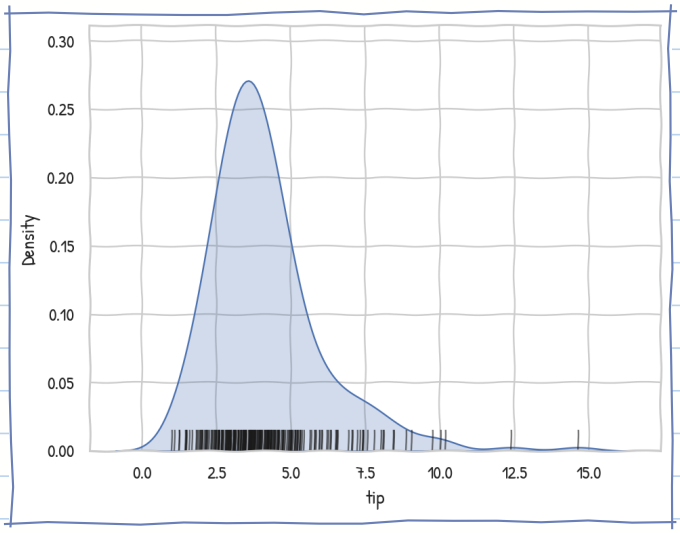
```
import seaborn as sns
import matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins
tips = make_tips()
sns.set_theme(style='whitegrid')
ax = sns.ecdfplot(data=tips, x='total_bill', hue='time')
ax.axhline(.5, color='gray', ls=':'); ax.set_title('ECDF: median where curve crosses 0.5')
plt.show()
```



6.4 rugplot(): marginal ticks

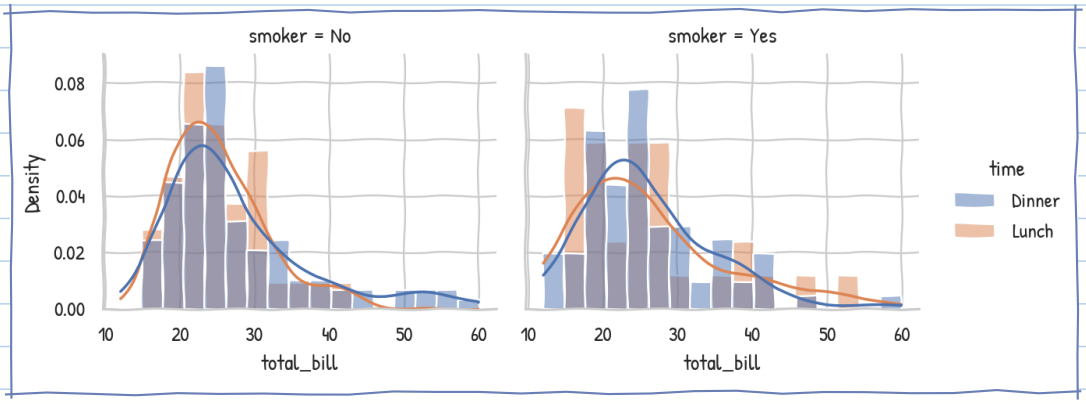
Concept: A **rug** draws a small tick for every observation along an axis. Add it under a KDE or histogram to show where the real data points are (and how sparse they are).

```
import seaborn as sns
import matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins
tips = make_tips()
sns.set_theme(style='whitegrid')
ax = sns.kdeplot(data=tips, x='tip', fill=True)
sns.rugplot(data=tips, x='tip', height=.05, color='k', alpha=.6, ax=ax)
plt.show()
```



6.5 displot(): the figure-level distribution function

```
import seaborn as sns
import matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins
tips = make_tips()
sns.set_theme(style='whitegrid')
g = sns.displot(data=tips, x='total_bill', hue='time', col='smoker', kind='hist',
                kde=True, height=3, aspect=1.2, stat='density', common_norm=False)
plt.show()
```

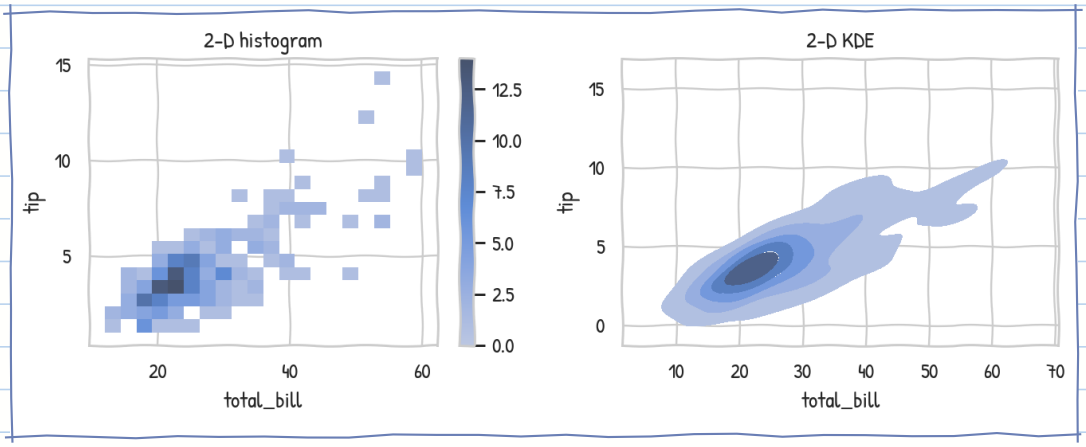


Explanation: `kind='hist' | 'kde' | 'ecdf'`. `common_norm=False` normalises each hue group separately - essential when groups have different sizes. `multiple='layer' | 'stack' | 'dodge' | 'fill'` controls how hue groups overlap; `element='step'` draws outlines only.

6.6 Univariate vs bivariate distributions

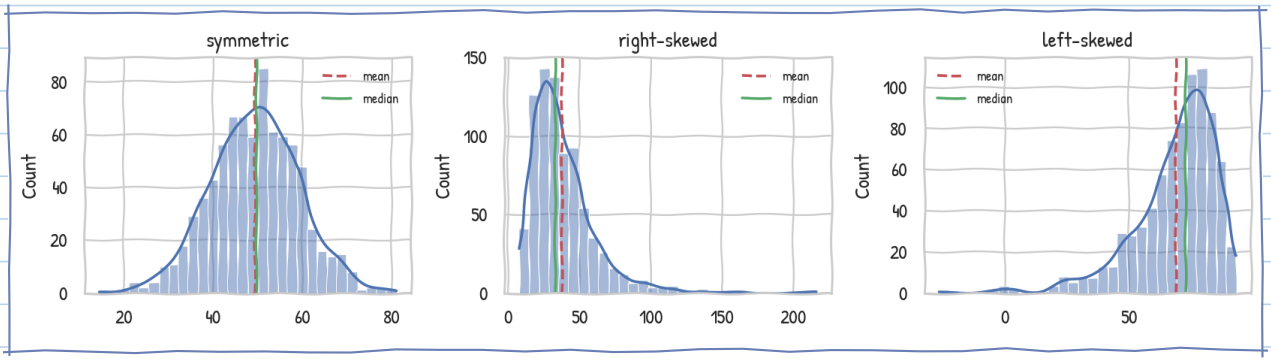
```
import seaborn as sns
import matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins
tips = make_tips()
sns.set_theme(style='whitegrid')
fig, axs = plt.subplots(1, 2, figsize=(8, 3.2))
```

```
sns.histplot(tips, x='total_bill', y='tip', bins=20, cbar=True, ax=axs[0]); axs[0].set_title('2-D histogram')
sns.kdeplot(tips, x='total_bill', y='tip', fill=True, levels=6, thresh=.05, ax=axs[1]); axs[1].set_title('2-D KDE')
plt.tight_layout(); plt.show()
```



6.7 Reading skewness, spread and outliers

```
import seaborn as sns
import matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins
tips = make_tips()
sns.set_theme(style='whitegrid')
import numpy as np
rng = np.random.default_rng(1)
d = {'symmetric': rng.normal(50, 10, 800), 'right-skewed': rng.lognormal(3.5, .5, 800),
     'left-skewed': 100 - rng.lognormal(3.3, .5, 800)}
fig, axs = plt.subplots(1, 3, figsize=(10, 2.7))
for ax, (k, v) in zip(axs, d.items()):
    sns.histplot(v, bins=30, kde=True, ax=ax); ax.axvline(np.mean(v), color='r', ls='--', label='mean')
    ax.axvline(np.median(v), color='g', ls='-', label='median'); ax.set_title(k); ax.legend(fontsize=8)
plt.tight_layout(); plt.show()
```



KEY IDEA: Right skew: long tail to the right, mean > median (income, prices). **Left skew:** mean < median. Heavy skew -> report the median or take logs. **Outliers:** isolated bars/points far from the bulk - investigate, do not delete blindly.

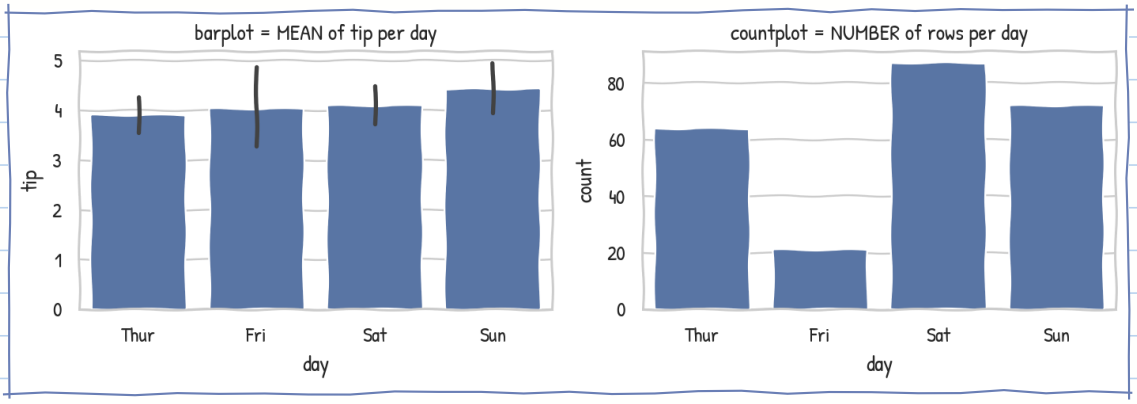
7 Categorical Plots

bar, count, box, violin, strip, swarm, boxen, point, catplot

Categorical plots compare a numeric variable (or counts) across the levels of a category. Seaborn groups the data for you.

7.1 barplot() vs countplot() - the crucial difference

```
import seaborn as sns
import matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins
tips = make_tips()
sns.set_theme(style='whitegrid')
fig, (a, b) = plt.subplots(1, 2, figsize=(8.5, 3))
sns.barplot(data=tips, x='day', y='tip', errorbar=('ci', 95), ax=a); a.set_title('barplot = MEAN of tip per day')
sns.countplot(data=tips, x='day', ax=b); b.set_title('countplot = NUMBER of rows per day')
plt.tight_layout(); plt.show()
```

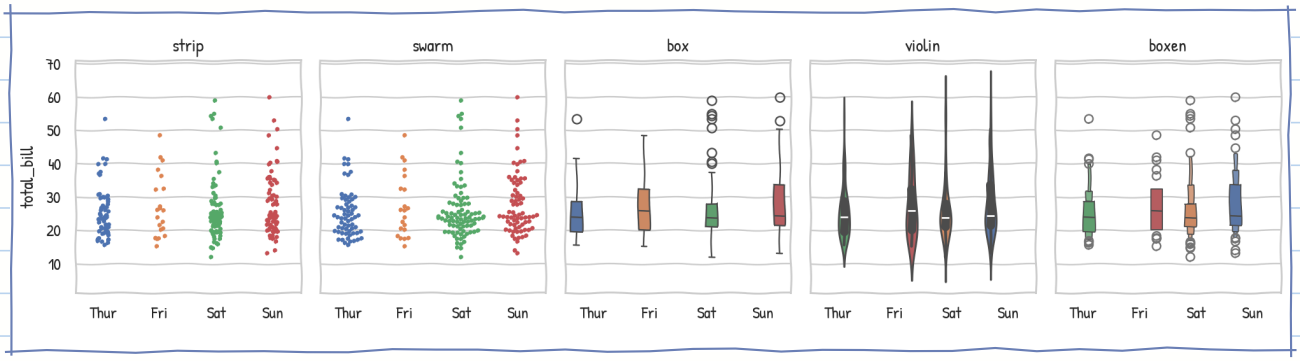


	barplot	countplot
Needs	x and numeric y	only x (or only y)
Height shows	an **aggregate** (mean by default) of y	**count of observations**
Error bar	95% CI of the aggregate (errorbar=)	none
Change aggregate	estimator='median', 'sum'	-

COMMON MISTAKE: A bar of means can hide everything: the same mean can come from tight or wild data, and the bar says nothing about how many observations back it. Show the **raw points** (strip/swarm) or a **box/violin** as well, and report n.

7.2 Five ways to show a distribution per category

```
import seaborn as sns
import matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins
tips = make_tips()
sns.set_theme(style='whitegrid')
fig, axs = plt.subplots(1, 5, figsize=(12, 3), sharey=True)
kinds = [('stripplot', 'strip'), ('swarmplot', 'swarm'), ('boxplot', 'box'),
         ('violinplot', 'violin'), ('boxenplot', 'boxen')]
for ax, (fn, name) in zip(axs, kinds):
    kw = dict(size=2.5) if fn in ('stripplot', 'swarmplot') else {}
    getattr(sns, fn)(data=tips, x='day', y='total_bill', hue='day', legend=False, ax=ax, **kw)
    ax.set_title(name); ax.set_xlabel('')
plt.tight_layout(); plt.show()
```

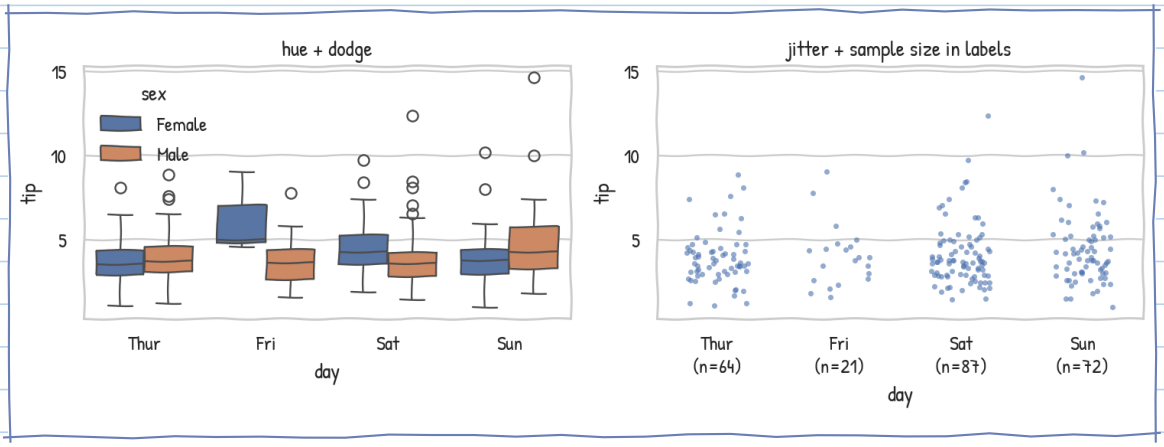


Plot	Shows	Best when	Watch out
stripplot	every point (jittered)	small/medium n	overlap on big n
swarmplot	every point, non-overlapping	n < ~300 per group	slow & wide for big n
boxplot	median, IQR, whiskers, outliers	compare many groups	hides multimodality
violinplot	KDE shape + box	shape matters	KDE caveats (6.2); use *cut=0*
boxenplot	many quantiles	large n, heavy tails	less familiar to readers

7.3 Order, hue, dodging, jitter and sample sizes

```
import seaborn as sns
import matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins
tips = make_tips()
sns.set_theme(style='whitegrid')
order = ['Thur', 'Fri', 'Sat', 'Sun']
fig, (a, b) = plt.subplots(1, 2, figsize=(9, 3.2))
```

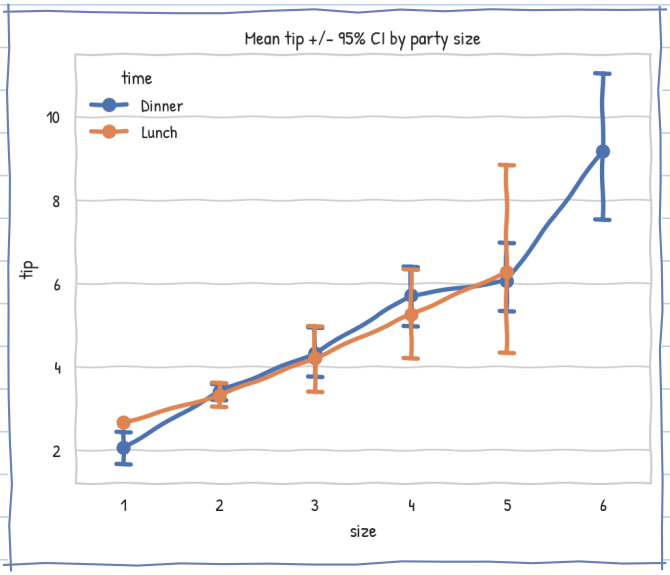
```
sns.boxplot(data=tips, x='day', y='tip', hue='sex', order=order, dodge=True, ax=a)   # side by side
a.set_title('hue + dodge')
sns.stripplot(data=tips, x='day', y='tip', order=order, jitter=.25, size=3, alpha=.6, ax=b)
n = tips['day'].value_counts()
b.set_xticks(range(4), [f'{d}\n(n={n[d]})' for d in order]); b.set_title('jitter + sample size in labels')
plt.tight_layout(); plt.show()
```



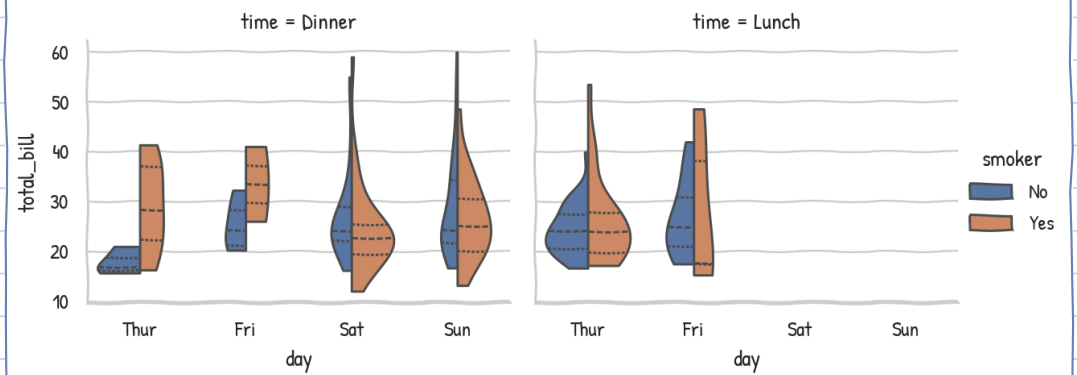
Explanation: *order=* fixes category order (otherwise order of appearance); *hue=* splits each category by a second variable; *dodge=True* places hue groups side by side; *jitter* spreads points sideways to reduce overlap. ***Outliers*** in a box are points beyond 1.5 x IQR - they are real data, not errors, until proven otherwise.

7.4 pointplot() and catplot()

```
import seaborn as sns
import matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins
tips = make_tips()
sns.set_theme(style='whitegrid')
ax = sns.pointplot(data=tips, x='size', y='tip', hue='time', errorbar=('ci', 95), capsize=.15)
ax.set_title('Mean tip +/- 95% CI by party size'); plt.show()
```



```
import seaborn as sns
import matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins
tips = make_tips()
sns.set_theme(style='whitegrid')
g = sns.catplot(data=tips, x='day', y='total_bill', hue='smoker', kind='violin',
                split=True, col='time', height=3, aspect=1.2, inner='quart', cut=0)
plt.show()
```



Explanation: *pointplot* shows means with CIs joined by lines - ideal for comparing trends across categories (only if categories are ordered). *catplot(kind=...)* is the figure-level entry point to all categorical plots: *kind='strip','swarm','box','violin','boxen','point','bar','count'*.

> ***NOTE:*** Seaborn 0.13: passing *palette=* ***without*** *hue=* is deprecated. Write *hue='day', palette='Set2', legend=False* instead.

Practice: Show *total_bill* by *day* with: a boxplot with raw points overlaid; a countplot of *smoker* split by *sex*; a violin per *time*. State one thing the boxplot hides.

P5 Advanced Seaborn & Statistical Visualization

heatmaps, grids, regression, correlation, uncertainty

Part V covers multi-variable and model-oriented plots. Keep one rule in mind throughout: a chart DESCRIBES data; it does not PROVE a cause or statistical significance.

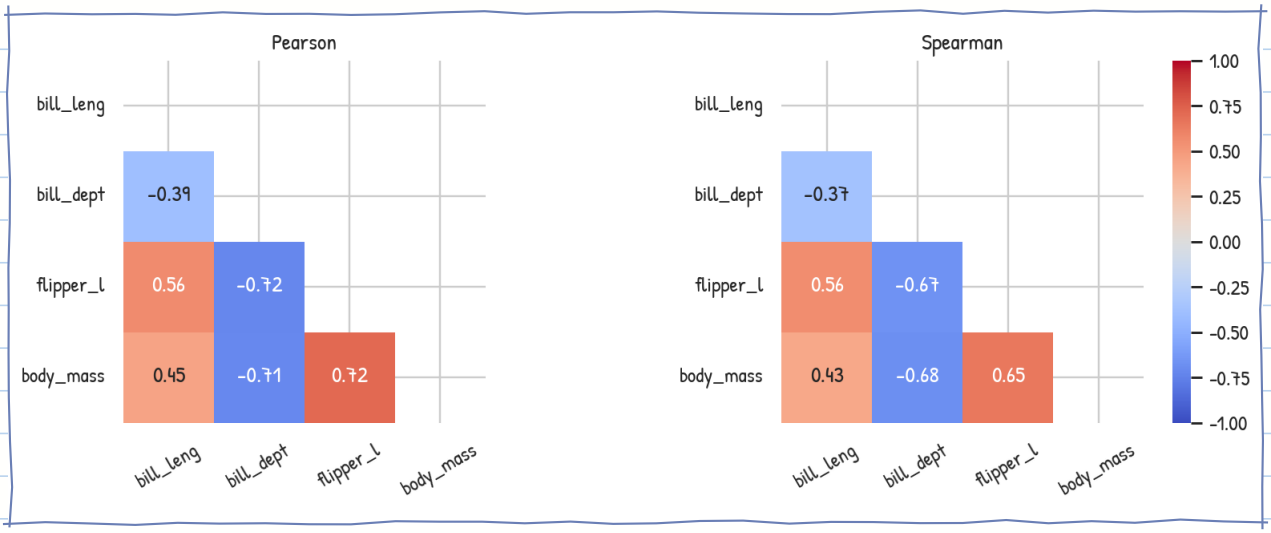
D Matrix and Grid Plots

heatmap, clustermap, pairplot, jointplot, PairGrid, FacetGrid

D.1 Correlation heatmaps, Pearson vs Spearman

Concept: A **heatmap** colours a matrix; with *df.corr()* it shows how strongly every pair of numeric columns moves together. **Pearson** r measures linear association; **Spearman** rho uses ranks, so it catches monotonic non-linear links and resists outliers. Both range -1 to +1.

```
import seaborn as sns, numpy as np, pandas as pd
import matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins
tips, pg = make_tips(), make_penguins()
num = ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']
sns.set_theme(style='whitegrid')
fig, axs = plt.subplots(1, 2, figsize=(10, 3.8))
for ax, m in zip(axs, ['pearson', 'spearman']):
    c = pg[num].corr(method=m)                    # NaN rows are dropped pairwise
    mask = np.triu(np.ones_like(c, dtype=bool))
    sns.heatmap(c, mask=mask, annot=True, fmt='.2f', cmap='coolwarm', center=0,
                vmin=-1, vmax=1, square=True, cbar=ax is axs[1], ax=ax)
    ax.set_title(m.capitalize()); ax.set_xticklabels([t[:9] for t in num], rotation=30)
    ax.set_yticklabels([t[:9] for t in num], rotation=0)
plt.tight_layout(); plt.show()
```



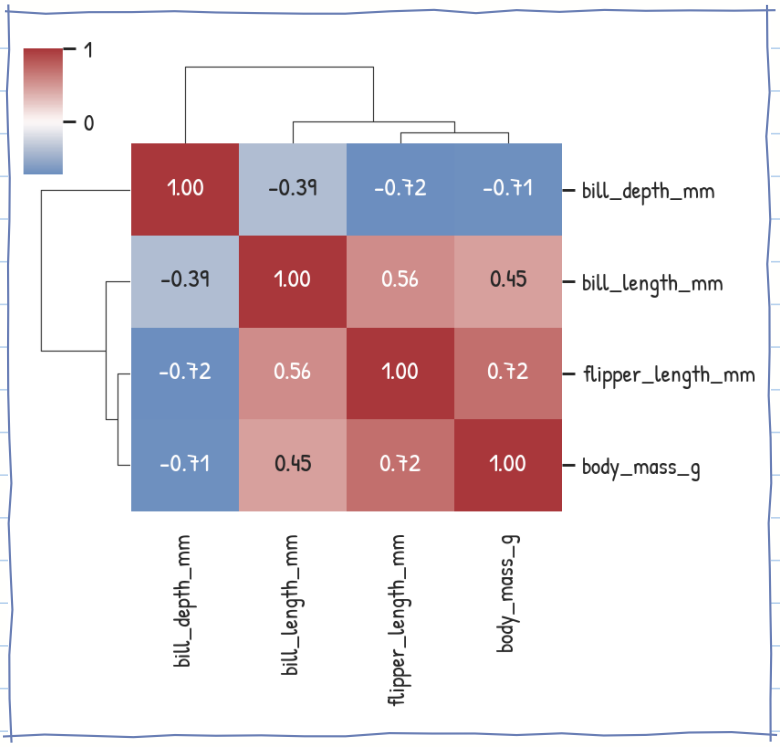
WARNING: Correlation is not causation. Here body mass and flipper length correlate partly because species differs in both (a confounder). Also: `corr()` ignores NaNs pairwise - say how many rows were used. Use `center=0` with diverging maps so 0 is neutral.

COMMON MISTAKE: Pearson r near 0 does NOT mean no relationship - a perfect U-shape has r = 0. Always look at the scatter plot as well.

D.2 clustermap(): hierarchical clustering

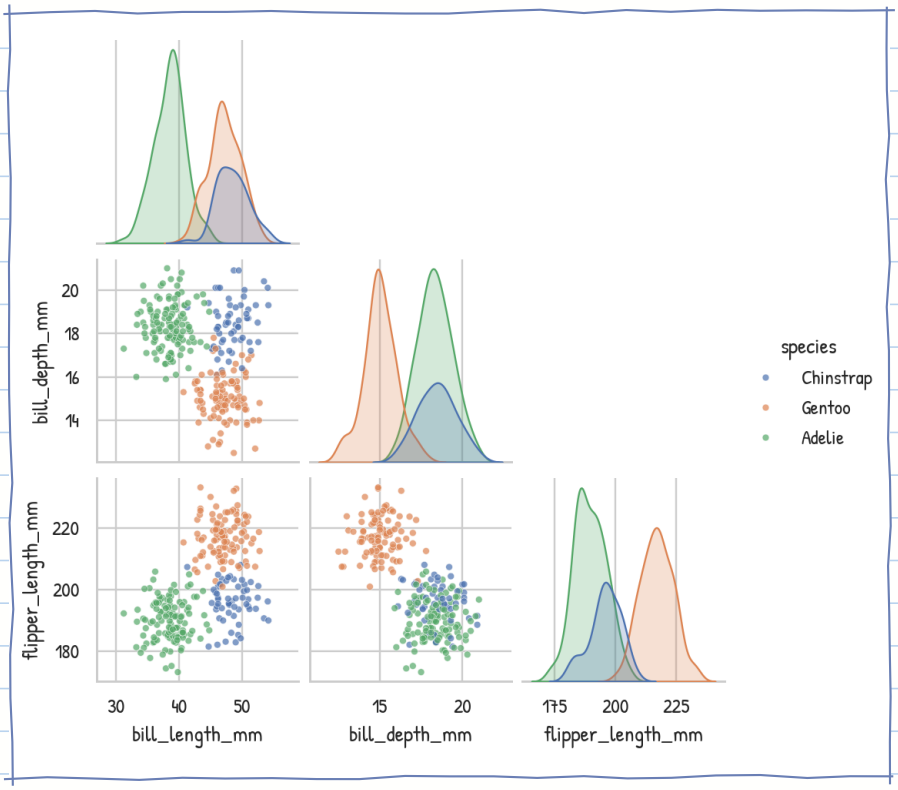
Concept: `clustermap` reorders rows and columns so similar ones sit together, and draws dendrograms (trees) showing the merge order. Needs SciPy.

```
import seaborn as sns, numpy as np, pandas as pd
import matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins
tips, pg = make_tips(), make_penguins()
num = ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']
sns.set_theme(style='whitegrid')
g = sns.clustermap(pg[num].corr(), annot=True, fmt='.2f', cmap='vlag', center=0,
                   figsize=(5, 4.5), dendrogram_ratio=.18)
plt.show()
```



D.3 pairplot(): all pairs at once

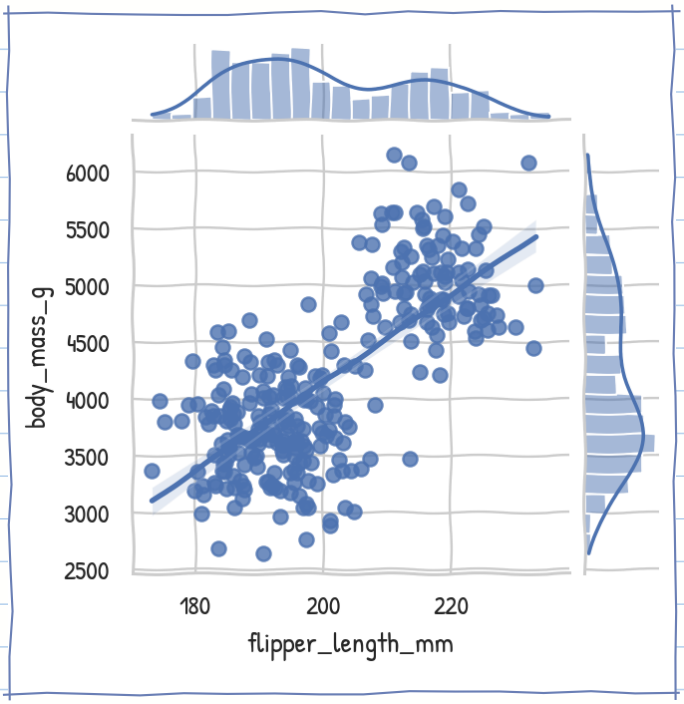
```
import seaborn as sns, numpy as np, pandas as pd
import matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins
tips, pg = make_tips(), make_penguins()
num = ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']
sns.set_theme(style='whitegrid')
g = sns.pairplot(pg.dropna(), vars=num[:3], hue='species', corner=True,
                 diag_kind='kde', plot_kws=dict(s=14, alpha=.7), height=1.9)
plt.show()
```



NOTE: *pairplot* draws n x n panels, so 10 variables = 100 plots: slow and unreadable. Pick *vars=* (3-6), use *corner=True*, lower *s/alpha*, or sample rows (*df.sample(2000)*) for big data.

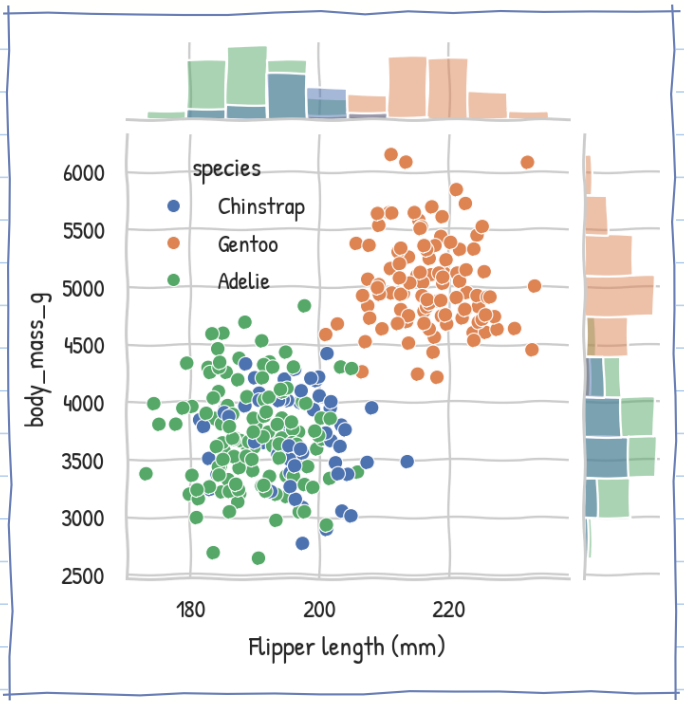
D.4 jointplot() and JointGrid

```
import seaborn as sns, numpy as np, pandas as pd
import matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins
tips, pg = make_tips(), make_penguins()
num = ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']
sns.set_theme(style='whitegrid')
fig = plt.figure()
for k in ['scatter', 'hex', 'kde', 'reg']:
    pass
g = sns.jointplot(data=pg, x='flipper_length_mm', y='body_mass_g', kind='reg',
                  height=4, marginal_kws=dict(bins=20))
plt.show()
```



Explanation: `kind=` can be `'scatter'` (default), `'kde'`, `'hist'`, `'hex'`, `'reg'` or `'resid'`. The side plots are ***marginal distributions***. For full control build it yourself:

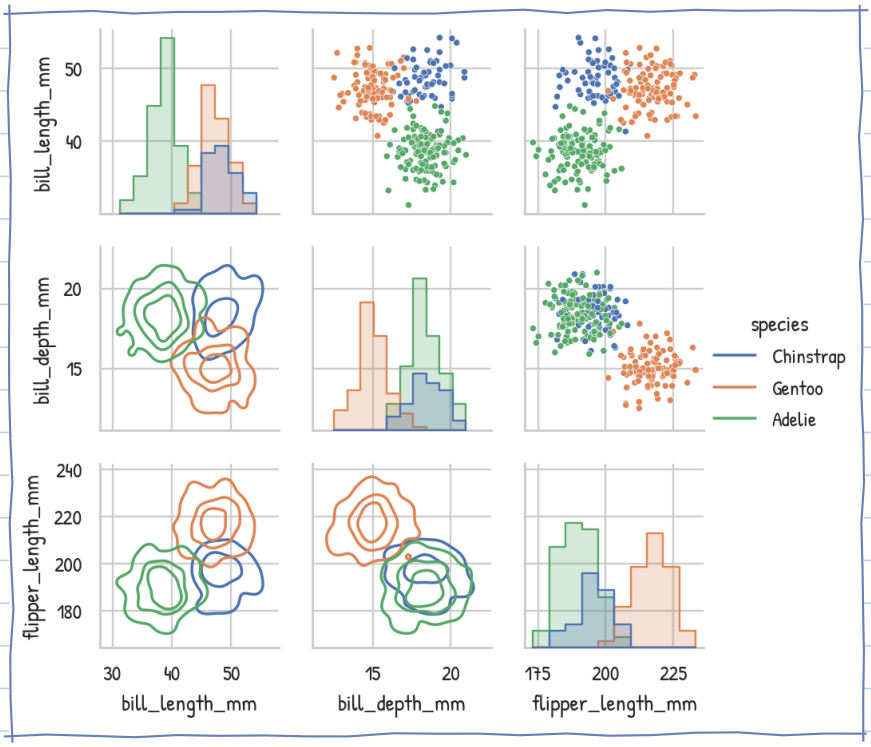
```
import seaborn as sns, numpy as np, pandas as pd
import matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins
tips, pg = make_tips(), make_penguins()
num = ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']
sns.set_theme(style='whitegrid')
g = sns.JointGrid(data=pg, x='flipper_length_mm', y='body_mass_g', hue='species', height=4.2)
g.plot(sns.scatterplot, sns.histplot)
g.ax_joint.set_xlabel('Flipper length (mm)')
plt.show()
```



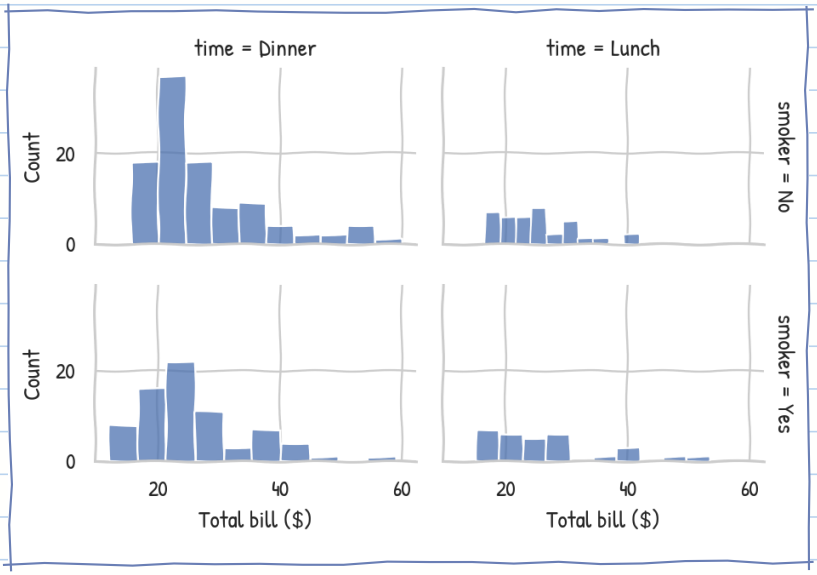
D.5 PairGrid and FacetGrid

Concept: *PairGrid* is the customizable engine under *pairplot* (choose different plots for diagonal / upper / lower). *FacetGrid* splits ANY plotting function into rows and columns of subsets.

```
import seaborn as sns, numpy as np, pandas as pd
import matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins
tips, pg = make_tips(), make_penguins()
num = ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']
sns.set_theme(style='whitegrid')
g = sns.PairGrid(pg.dropna(), vars=num[:3], hue='species', height=1.8)
g.map_upper(sns.scatterplot, s=12); g.map_lower(sns.kdeplot, levels=4)
g.map_diag(sns.histplot, element='step')
g.add_legend(); plt.show()
```




```
import seaborn as sns, numpy as np, pandas as pd
import matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins
tips, pg = make_tips(), make_penguins()
num = ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']
sns.set_theme(style='whitegrid')
g = sns.FacetGrid(tips, col='time', row='smoker', height=1.9, aspect=1.5, margin_titles=True)
g.map_dataframe(sns.histplot, x='total_bill', bins=10)
g.set_axis_labels('Total bill ($)', 'Count'); plt.show()
```



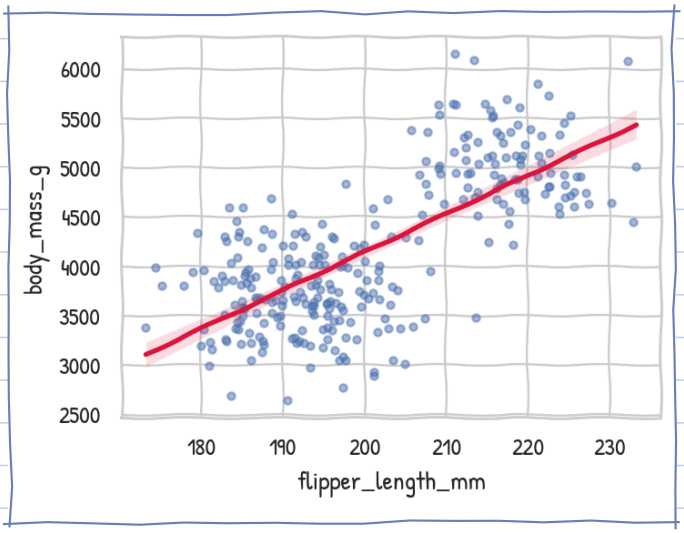
E Regression, Residuals and Uncertainty

regplot, lmplot, residplot, bootstrap

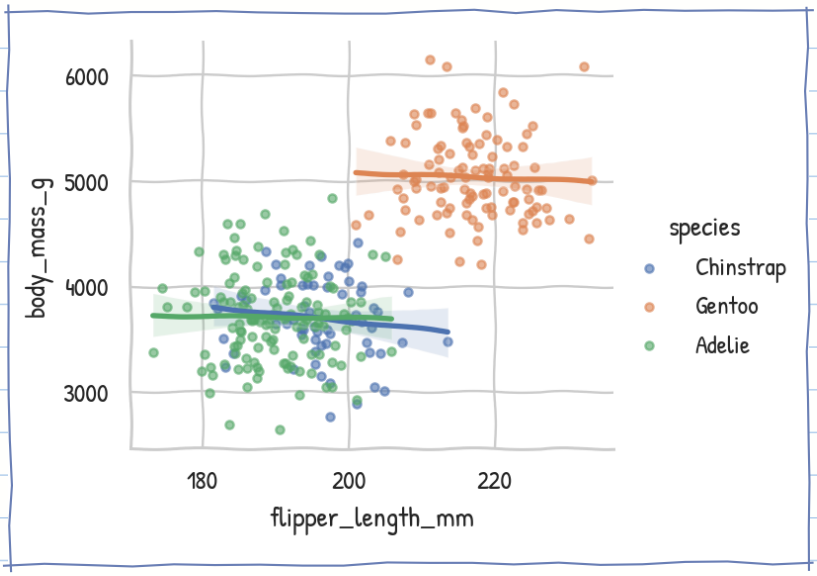
E.1 regplot() vs lmplot()

Concept: Both draw a scatter plus a fitted regression line with a shaded 95% confidence band (the uncertainty of the line, from bootstrapping). *regplot* is axes-level (works with *ax=*). *lmplot* is figure-level (adds *hue*, *col*, *row*).

```
import seaborn as sns, numpy as np, pandas as pd
import matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins
tips, pg = make_tips(), make_penguins()
num = ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']
sns.set_theme(style='whitegrid')
fig, ax = plt.subplots(figsize=(4.5, 3.2))
sns.regplot(data=pg, x='flipper_length_mm', y='body_mass_g', scatter_kws=dict(s=12, alpha=.5),
            line_kws=dict(color='crimson'), ax=ax)
plt.show()
```




```
import seaborn as sns, numpy as np, pandas as pd
import matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins
tips, pg = make_tips(), make_penguins()
num = ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']
sns.set_theme(style='whitegrid')
g = sns.lmplot(data=pg, x='flipper_length_mm', y='body_mass_g', hue='species',
               height=3.3, aspect=1.2, scatter_kws=dict(s=12, alpha=.6))
plt.show()
```



KEY IDEA: One pooled line over all species suggests one slope; `lmplot(hue='species')` shows each group has its own line. ***Simpson's paradox***: a trend can reverse after you split by group.

	regplot	*lmplot*
Level	axes	figure (FacetGrid)
Facets / hue	no (one fit)	yes *hue*, *col*, *row*
Place in subplots	*ax=ax*	no - own figure, use *height*/*aspect*
Options	*order=2*, *logx=True*, *lowess=True*, *ci=95*	same, via the grid

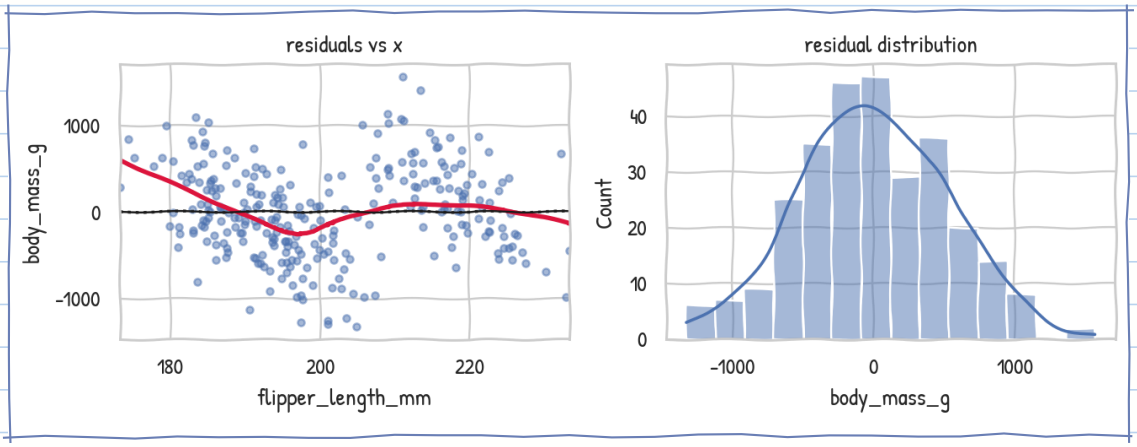
E.2 residplot(): residual analysis

NOTE: *lowess=True* needs the optional package *statsmodels* (*pip install statsmodels*).

Concept: A ***residual*** = observed - predicted. A good linear model leaves residuals scattered randomly around 0. Curves, funnels (growing spread) or clusters mean the model is missing something.

```
import seaborn as sns, numpy as np, pandas as pd
import matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins
tips, pg = make_tips(), make_penguins()
num = ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']
sns.set_theme(style='whitegrid')
d = pg.dropna()
fig, axs = plt.subplots(1, 2, figsize=(8, 3))
sns.residplot(data=d, x='flipper_length_mm', y='body_mass_g', lowess=True,
              scatter_kws=dict(s=12, alpha=.5), line_kws=dict(color='crimson'), ax=axs[0])
axs[0].axhline(0, color='k', lw=1); axs[0].set_title('residuals vs x')
b = np.polyfit(d.flipper_length_mm, d.body_mass_g, 1)
resid = d.body_mass_g - np.polyval(b, d.flipper_length_mm)
```

```
sns.histplot(x=resid, kde=True, ax=axs[1]); axs[1].set_title('residual distribution')
plt.tight_layout(); plt.show()
```

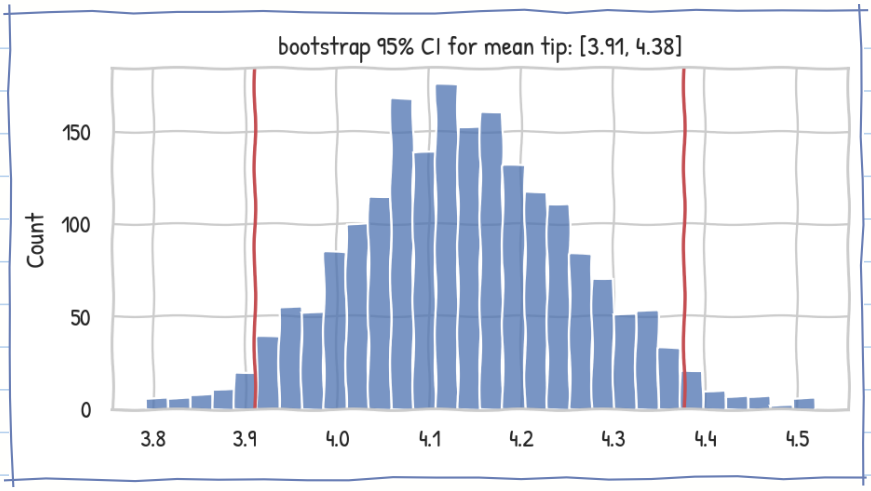


Explanation: Check: (1) no pattern vs x (linearity), (2) similar spread everywhere (constant variance), (3) roughly bell-shaped histogram (normal errors), (4) no extreme points. Residuals here show structure left by species.

E.3 Bootstrap confidence intervals and sample size

Concept: Seaborn builds its error bars by ***bootstrapping***: resample the data with replacement thousands of times, compute the statistic each time, and take the middle 95%. It reflects sampling uncertainty - not measurement error or bias.

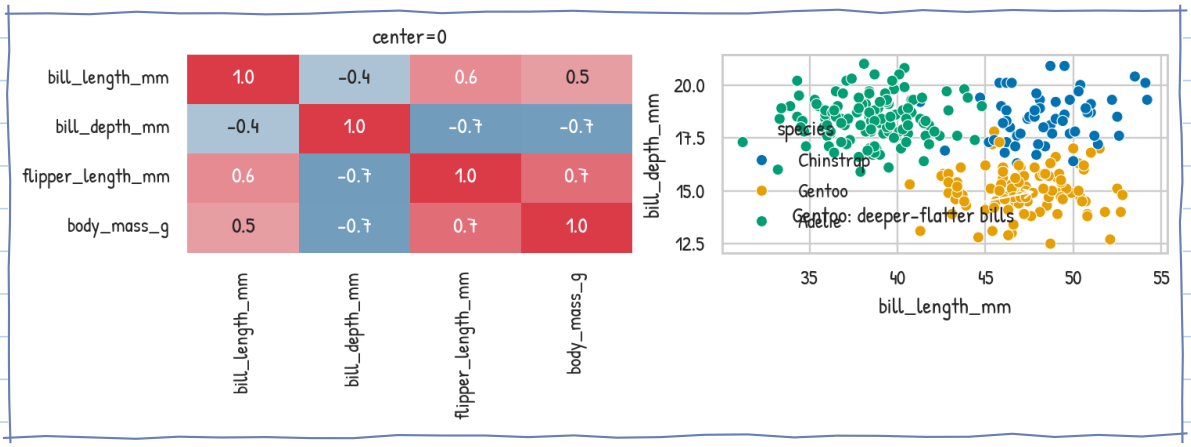
```
import seaborn as sns, numpy as np, pandas as pd
import matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins
tips, pg = make_tips(), make_penguins()
num = ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']
sns.set_theme(style='whitegrid')
rng = np.random.default_rng(0)
x = tips['tip'].to_numpy()
boots = [rng.choice(x, len(x)).mean() for _ in range(2000)]
lo, hi = np.percentile(boots, [2.5, 97.5])
fig, ax = plt.subplots(figsize=(6, 2.8))
sns.histplot(boots, bins=30, ax=ax); ax.axvline(lo, color='r'); ax.axvline(hi, color='r')
ax.set_title(f'bootstrap 95% CI for mean tip: [{lo:.2f}, {hi:.2f}]'); plt.show()
```



EXAM TIP: Descriptive vs inferential: a chart with a CI band is descriptive evidence. Overlapping or non-overlapping bars are NOT a significance test; use a proper test (t-test, ANOVA, permutation test) and report effect sizes. Small n gives wide, unstable intervals - always print n.

E.4 Palettes, normalization and combining with Matplotlib

```
import seaborn as sns, numpy as np, pandas as pd
import matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins
tips, pg = make_tips(), make_penguins()
num = ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']
sns.set_theme(style='whitegrid')
pal = sns.diverging_palette(240, 10, as_cmap=True)          # blue - white - red
fig, axs = plt.subplots(1, 2, figsize=(9, 3.3))
c = pg[num].corr()
sns.heatmap(c, cmap=pal, center=0, annot=True, fmt='.1f', ax=axs[0], cbar=False); axs[0].set_title('center=0')
ax = sns.scatterplot(data=pg, x='bill_length_mm', y='bill_depth_mm', hue='species',
                     palette=['#0072B2', '#E69F00', '#009E73'], ax=axs[1])
ax.annotate('Gentoo: deeper-flatter bills', xy=(48, 15), xytext=(34, 13.5), arrowprops=dict(arrowstyle='->'))
plt.tight_layout(); plt.show()
```



PRO TIP: Workflow: draw with Seaborn (quick, statistical) -> receive the *Axes*/*Figure* -> polish with Matplotlib (*ax.set_title*, *ax.annotate*, *fig.savefig*).

E.5 Big data and choosing the right statistical plot

Question	Plot
Shape of one variable	*histplot*, *kdeplot*, *ecdfplot*
Compare a variable across groups	*boxplot*, *violinplot*, *swarmplot* (+ n)
Counts / proportions	*countplot*, stacked bar
Relationship of two numerics	*scatterplot*, *regplot*, *jointplot*
Many variables	*pairplot* (few), *heatmap* of corr, *clustermap*
Trend with uncertainty	*lineplot* (CI), *pointplot*
Model fit quality	*residplot*, predicted-vs-actual
500k+ rows	sample, *histplot(bins=)*, *jointplot(kind='hex')*, aggregate first

Practice: On *make_penguins()*: draw a Spearman heatmap, a pairplot by species with *corner=True*, a residual plot, and write two sentences separating what the charts show from what they cannot prove.

P6

Professional Chart Design

publication-quality and business-ready visualization

8

Designing Charts That Communicate

choose, colour, simplify, annotate, export

8.1 Choosing the right chart: decision tree

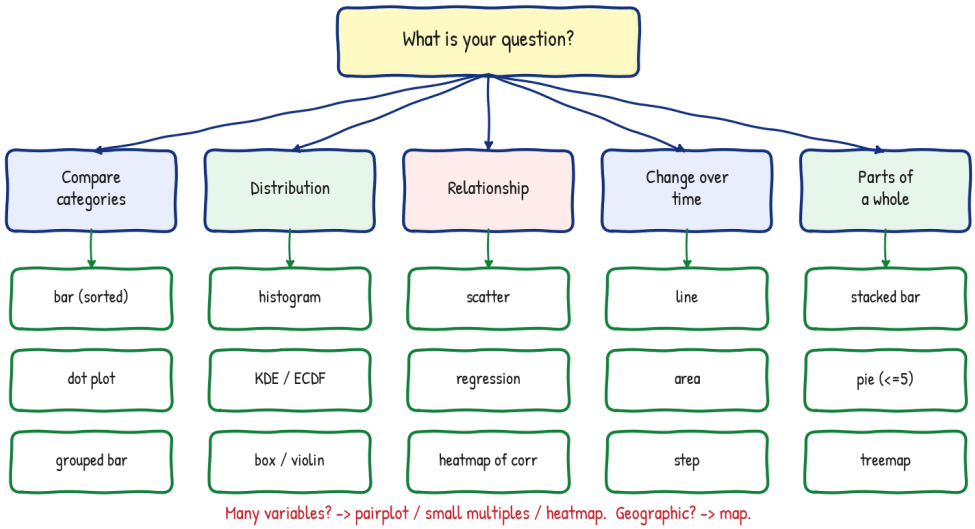
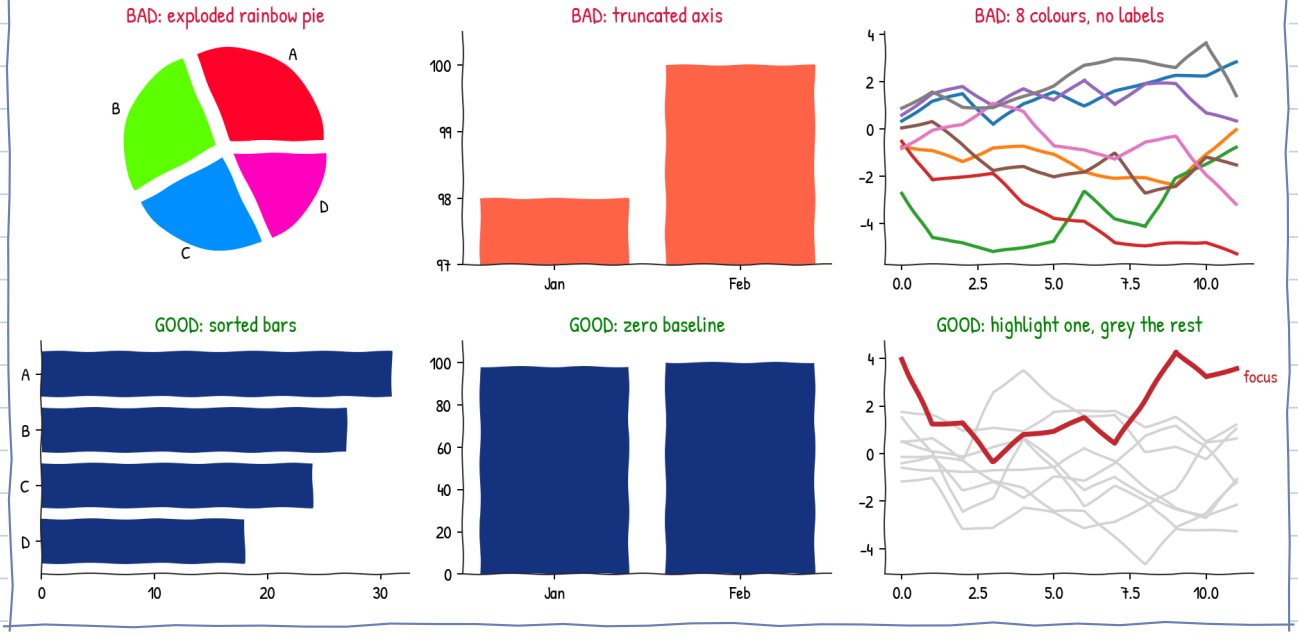


Fig 8.1 - chart-selection decision tree: start from the question, not from the chart.

8.2 Bad charts vs improved charts

```
import numpy as np, matplotlib.pyplot as plt
rng = np.random.default_rng(1)
fig, axs = plt.subplots(2, 3, figsize=(11, 5.4))
n = ['A', 'B', 'C', 'D']; v = [31, 27, 24, 18]
axs[0, 0].pie(v, labels=n, colors=plt.cm.gist_rainbow(np.linspace(0, 1, 4)), explode=[.1] * 4)
axs[0, 0].set_title('BAD: exploded rainbow pie', color='crimson')
```

```
axs[1, 0].barh(n[::-1], v[::-1], color='#14327d'); axs[1, 0].set_title('GOOD: sorted bars', color='green')
axs[0, 1].bar(['Jan', 'Feb'], [98, 100], color='tomato'); axs[0, 1].set_ylim(97, 100.5)
axs[0, 1].set_title('BAD: truncated axis', color='crimson')
axs[1, 1].bar(['Jan', 'Feb'], [98, 100], color='#14327d'); axs[1, 1].set_ylim(0, 110)
axs[1, 1].set_title('GOOD: zero baseline', color='green')
x = np.arange(12)
for i in range(8): axs[0, 2].plot(x, rng.normal(0, 1, 12).cumsum(), lw=2)
axs[0, 2].set_title('BAD: 8 colours, no labels', color='crimson')
for i in range(8): axs[1, 2].plot(x, rng.normal(0, 1, 12).cumsum(), color='lightgray')
axs[1, 2].plot(x, rng.normal(0, 1, 12).cumsum() + 3, color='#c4262e', lw=3)
axs[1, 2].text(11.2, 3, 'focus', color='#c4262e')
axs[1, 2].set_title('GOOD: highlight one, grey the rest', color='green')
for a in axs.flat[1:]: a.spines[['top', 'right']].set_visible(False)
plt.tight_layout(); plt.show()
```



Bad habit	Why it hurts	Fix
3-D / exploded pie or bars	perspective distorts size	flat sorted bars
Truncated bar axis	exaggerates differences	start bars at 0 (lines may zoom, but say so)
Rainbow / jet colours	no order, not colour-blind safe	*viridis*, one accent + grey
Dual axes	arbitrary scales fake correlation	two stacked panels
Spaghetti lines	unreadable	highlight, facet or label directly
Default title *Plot 1*	reader must decode	insight title: Sales doubled after March
Missing units / source	cannot trust or interpret	label units, add *Source:* note

8.3 Colour, labels, annotation and outliers

- **Meaningful colour:** use colour to encode information (a group, a highlight), not decoration. Sequential maps for magnitude, diverging for +/-, categorical for groups, max ~6 hues.
- **Colour-blind safe:** *viridis*, Seaborn *'colorblind'*, Okabe-Ito; add markers/direct labels.
- **Remove clutter:** drop top/right spines, heavy grids, redundant legends, background fills, 3-D.
- **Label everything:** axis names with units, data source, date range, sample size, and what error bars mean.
- **Aspect ratio:** time series look best wide (about 2:1 to 3:1); scatter plots near square so slopes are not distorted.
- **Overlapping labels:** rotate 30-45 degrees, switch to horizontal bars, abbreviate, or use *tight_layout*/*constrained*.
- **Outliers:** do not silently clip them. Show them, annotate them, or use a log scale and say so; if you remove one, state why.

PRO TIP: Write the title as the **takeaway** ("Online orders overtook retail in Q3") and put the descriptive detail in a smaller subtitle.

8.4 Dashboards, report-ready sets and exporting

```
import matplotlib.pyplot as plt, numpy as np
rng = np.random.default_rng(0)
fig, axs = plt.subplots(2, 2, figsize=(9, 5.5), layout='constrained')
axs[0, 0].plot(np.cumsum(rng.normal(size=60)), color='#14327d'); axs[0, 0].set_title('Revenue trend')
axs[0, 1].bar(list('ABCD'), [30, 25, 20, 10], color='#12803a'); axs[0, 1].set_title('By region')
axs[1, 0].hist(rng.normal(size=300), bins=20, color='#e6a800'); axs[1, 0].set_title('Order value')
axs[1, 1].scatter(rng.normal(size=80), rng.normal(size=80), s=15); axs[1, 1].set_title('Spend vs visits')
fig.suptitle('Monthly KPI dashboard', fontsize=15, fontweight='bold')
fig.text(0.01, 0.005, 'Source: sales database, synthetic data', fontsize=8, color='gray')
fig.savefig('dashboard.png', dpi=200)       # slides / Word
fig.savefig('dashboard.pdf')                # print / papers
```

Target	Do this
PowerPoint / Word	PNG 200 dpi, width about 10 in (16:9) or 6 in (Word column), fonts >= 12 pt
PDF report / paper	vector PDF/SVG, fonts embedded (*pdf.fonttype=42*), consistent size
Website	SVG or compressed PNG, alt text, responsive width
Interactive dashboard	Plotly Dash / Streamlit / Power BI (beyond Matplotlib)

8.5 Professional visualization checklist

[ ] Does the chart answer ONE clear question?
[ ] Is the chart type right for the data type?
[ ] Do bars start at zero?
[ ] Are axes labelled with units?
[ ] Is there a takeaway title?

[ ] Is colour meaningful and colour-blind safe?
[ ] Is clutter removed and text readable at final size?
[ ] Are n, source and uncertainty shown?
[ ] Would it work in greyscale?

P7 Real-World Data Analysis Projects

ten complete, reproducible workflows

Each project: problem -> data -> cleaning -> charts -> interpretation -> insight -> limitations. All data are synthetic and generated in code with a fixed seed, so results are reproducible and the 'insights' describe the patterns built into that data, not real companies.

9 Business & Operations Projects

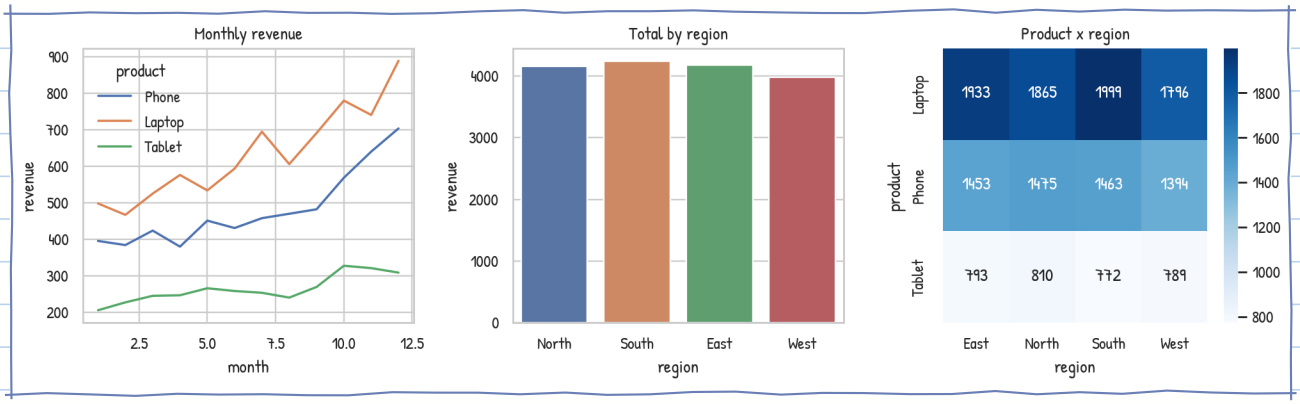
sales, e-commerce, marketing, churn, employees

Project 1: Sales analysis

Problem: Which months, products and regions drive revenue?

Dataset: 360 rows: 12 months x 3 products x 10 region-weighted records; columns month, product, region, revenue.

```
import numpy as np, pandas as pd, seaborn as sns, matplotlib.pyplot as plt
rng = np.random.default_rng(1)
df = pd.DataFrame([(m, p, r) for m in range(1, 13) for p in ['Phone', 'Laptop', 'Tablet']
                   for r in ['North', 'South', 'East', 'West']], columns=['month', 'product', 'region'])
base = df['product'].map({'Phone': 90, 'Laptop': 120, 'Tablet': 50})
season = 1 + 0.25 * (df['month'] >= 10) + 0.04 * df['month']
df['revenue'] = (base * season * rng.uniform(.8, 1.2, len(df))).round(1)
df = df.dropna().drop_duplicates()                          # cleaning
sns.set_theme(style='whitegrid')
fig, axs = plt.subplots(1, 3, figsize=(12, 3.6))
sns.lineplot(df, x='month', y='revenue', hue='product', estimator='sum', errorbar=None, ax=axs[0])
sns.barplot(df, x='region', y='revenue', estimator='sum', errorbar=None, hue='region', legend=False, ax=axs[1])
sns.heatmap(df.pivot_table('revenue', 'product', 'region', 'sum'), annot=True, fmt='.0f', cmap='Blues', ax=axs[2])
axs[0].set_title('Monthly revenue'); axs[1].set_title('Total by region'); axs[2].set_title('Product x region')
plt.tight_layout(); plt.show()
```



Interpretation: Revenue climbs through the year with a jump in Q4 (months 10–12). Laptops lead every month; the heatmap shows product-by-region totals at a glance.

> ***KEY IDEA:*** Seasonality matters: Q4 lifts all products, so plan inventory and campaigns before October.

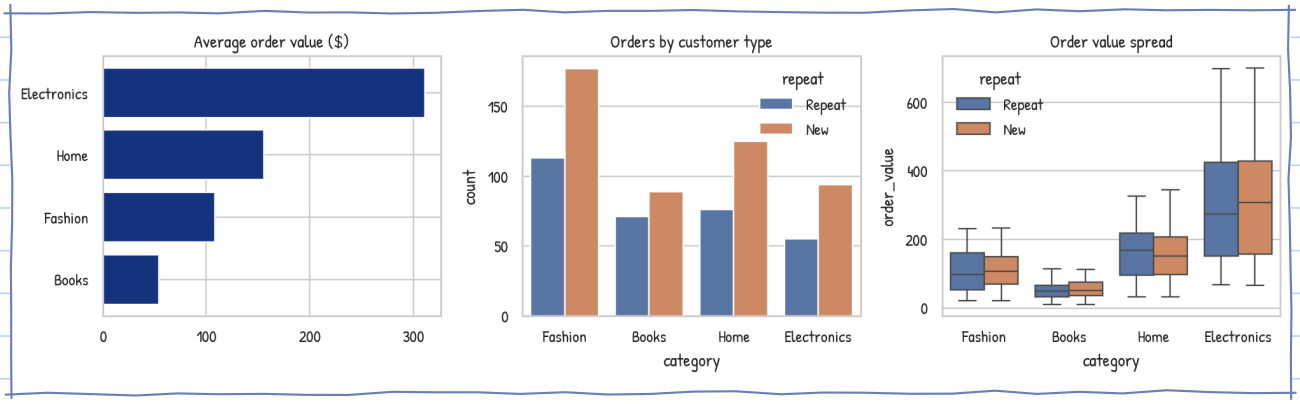
Limitations / next steps: Synthetic data, no costs/margins, no year-on-year view. Next: add profit, compare to last year, forecast with a rolling mean.

Project 2: E-commerce analysis

Problem: How do order value, category and customer behaviour relate?

Dataset: 800 orders with category, items, order value and a repeat-customer flag.

```
import numpy as np, pandas as pd, seaborn as sns, matplotlib.pyplot as plt
rng = np.random.default_rng(2)
n = 800
d = pd.DataFrame({'category': rng.choice(['Fashion', 'Electronics', 'Home', 'Books'], n, p=[.35, .2, .25, .2]),
                  'items': rng.integers(1, 7, n), 'repeat': rng.choice(['New', 'Repeat'], n, p=[.6, .4])})
price = d['category'].map({'Fashion': 30, 'Electronics': 90, 'Home': 45, 'Books': 15})
d['order_value'] = (price * d['items'] * rng.uniform(.7, 1.3, n)).round(2)
aov = d.groupby('category')['order_value'].mean().sort_values()          # average order value
sns.set_theme(style='whitegrid')
fig, axs = plt.subplots(1, 3, figsize=(12, 3.5))
axs[0].barh(aov.index, aov.values, color='#14327d'); axs[0].set_title('Average order value ($)')
sns.countplot(d, x='category', hue='repeat', ax=axs[1]); axs[1].set_title('Orders by customer type')
sns.boxplot(d, x='category', y='order_value', hue='repeat', ax=axs[2]); axs[2].set_title('Order value spread')
plt.tight_layout(); plt.show()
```



Interpretation: Electronics has the highest average order value but fewer orders; Fashion drives volume. Boxes show right-skewed order values with high-value outliers.

> ***KEY IDEA:*** High volume and high value come from different categories: protect Fashion traffic and upsell Electronics.

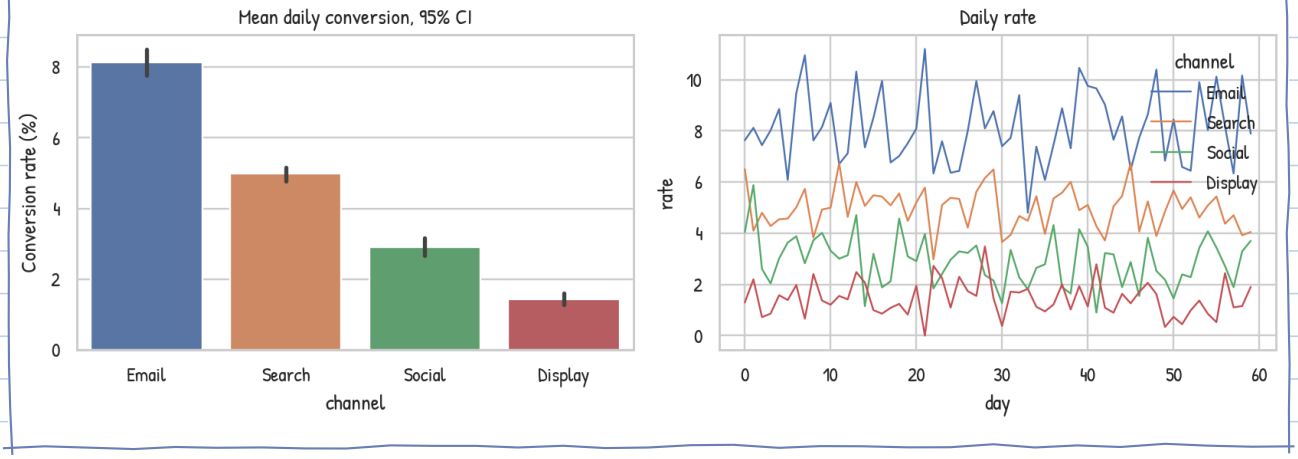
Limitations / next steps: Average order value is sensitive to outliers (consider the median). No time dimension or customer IDs. Next: cohort and funnel analysis.

Project 3: Marketing campaign performance

Problem: Which channel converts best, and is the difference reliable?

Dataset: Daily results for 4 channels over 60 days: visitors and conversions.

```
import numpy as np, pandas as pd, seaborn as sns, matplotlib.pyplot as plt
rng = np.random.default_rng(3)
rows = []
for ch, rate in {'Email': .08, 'Search': .05, 'Social': .03, 'Display': .015}.items():
    for day in range(60):
        v = rng.integers(200, 600)
        rows.append((ch, day, v, rng.binomial(v, rate)))
m = pd.DataFrame(rows, columns=['channel', 'day', 'visitors', 'conversions'])
m['rate'] = m['conversions'] / m['visitors'] * 100
sns.set_theme(style='whitegrid')
fig, axs = plt.subplots(1, 2, figsize=(10, 3.5))
sns.barplot(m, x='channel', y='rate', errorbar=('ci', 95), hue='channel', legend=False, ax=axs[0])
axs[0].set_ylabel('Conversion rate (%)'); axs[0].set_title('Mean daily conversion, 95% CI')
sns.lineplot(m, x='day', y='rate', hue='channel', errorbar=None, lw=1, ax=axs[1]); axs[1].set_title('Daily rate')
plt.tight_layout(); plt.show()
```



Interpretation: Email and Search sit clearly above Social and Display; their confidence intervals do not overlap with the low channels. Daily rates are noisy, so a trend line needs smoothing.

> ***KEY IDEA:*** Shift budget toward channels with reliably higher conversion, but compare ***cost per conversion***, not rate alone.

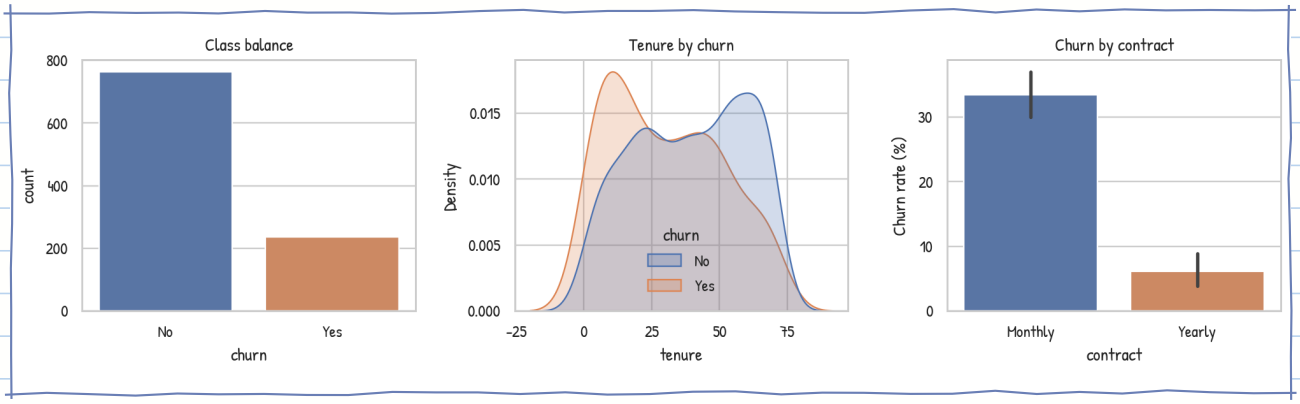
Limitations / next steps: Rate ignores cost and customer quality. Non-overlapping CIs hint at a difference but a formal test is needed. Next: A/B test, rolling averages.

Project 4: Customer churn exploration

Problem: Who leaves, and how imbalanced is the target?

Dataset: 1,000 customers: tenure (months), monthly charge, contract type, churn flag (about 25% churn).

```
import numpy as np, pandas as pd, seaborn as sns, matplotlib.pyplot as plt
rng = np.random.default_rng(4)
n = 1000
c = pd.DataFrame({'tenure': rng.integers(1, 72, n), 'charge': rng.normal(65, 20, n).clip(20),
                  'contract': rng.choice(['Monthly', 'Yearly'], n, p=[.65, .35])})
p = 0.55 - 0.006 * c['tenure'] + 0.003 * (c['charge'] - 65) - 0.3 * (c['contract'] == 'Yearly')
c['churn'] = np.where(rng.random(n) < p.clip(.02, .9), 'Yes', 'No')
sns.set_theme(style='whitegrid')
fig, axs = plt.subplots(1, 3, figsize=(12, 3.4))
sns.countplot(c, x='churn', hue='churn', legend=False, ax=axs[0]); axs[0].set_title('Class balance')
sns.kdeplot(c, x='tenure', hue='churn', common_norm=False,
            fill=True, ax=axs[1]); axs[1].set_title('Tenure by churn')
sns.barplot(c.assign(y=(c.churn == 'Yes') * 100), x='contract', y='y', hue='contract', legend=False, ax=axs[2])
axs[2].set_ylabel('Churn rate (%)'); axs[2].set_title('Churn by contract')
plt.tight_layout(); plt.show()
```



Interpretation: Churners are the minority class (imbalance), concentrated at short tenure; monthly contracts churn far more than yearly ones.

KEY IDEA: Early-life customers on monthly plans are the risk group: target them with onboarding and yearly-plan offers. Imbalance means plain accuracy is misleading for a model - use recall/precision/AUC.

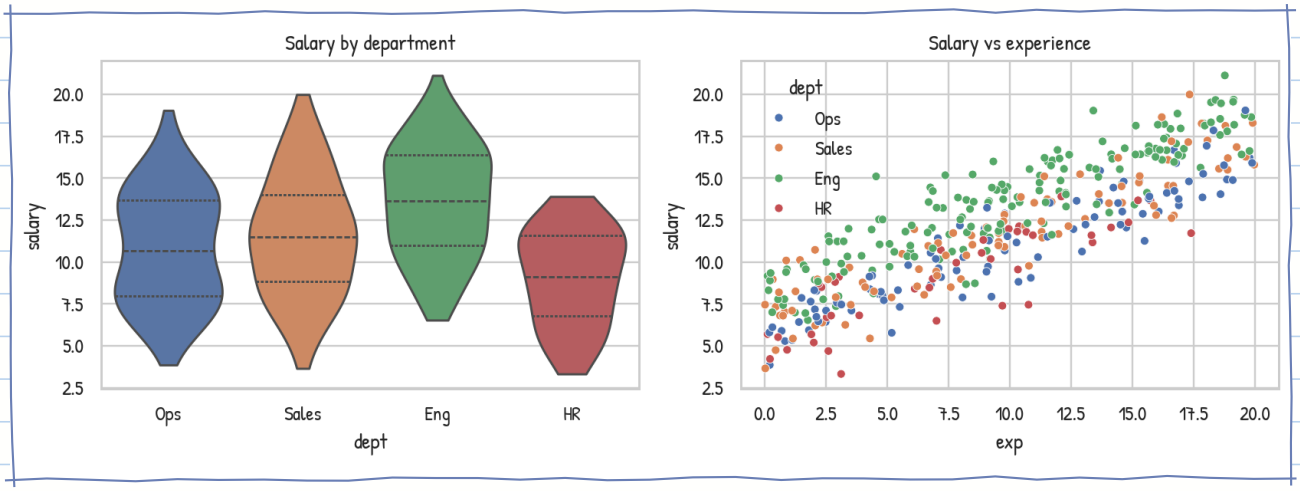
Limitations / next steps: Association, not cause: contract type may proxy for other traits. Next: correlation with more features, then a baseline classifier.

Project 5: Employee analytics

Problem: How do salary distributions differ by department, and how does experience relate?

Dataset: 400 employees: department, years of experience, salary (Rs lakh).

```
import numpy as np, pandas as pd, seaborn as sns, matplotlib.pyplot as plt
rng = np.random.default_rng(5)
n = 400
e = pd.DataFrame({'dept': rng.choice(['Eng', 'Sales', 'HR', 'Ops'], n, p=[.4, .25, .1, .25]),
                  'exp': rng.uniform(0, 20, n)})
base = e['dept'].map({'Eng': 8, 'Sales': 6, 'HR': 5, 'Ops': 5.5})
e['salary'] = base + .55 * e['exp'] + rng.normal(0, 1.5, n)
sns.set_theme(style='whitegrid')
fig, axs = plt.subplots(1, 2, figsize=(10, 3.6))
sns.violinplot(e, x='dept', y='salary', hue='dept', inner='quart', cut=0, legend=False, ax=axs[0])
sns.scatterplot(e, x='exp', y='salary', hue='dept', s=22, ax=axs[1])
axs[0].set_title('Salary by department'); axs[1].set_title('Salary vs experience')
plt.tight_layout(); plt.show()
```



Interpretation: Engineering has the highest salaries; every department shows an upward slope with experience.

KEY IDEA: Experience explains a lot of pay; department shifts the whole line up or down. Compare people at equal experience, not raw averages.

Limitations / next steps: No role level, gender or location, so no fairness conclusions. Next: regression with department as a factor.

10 Statistical & Modelling Projects

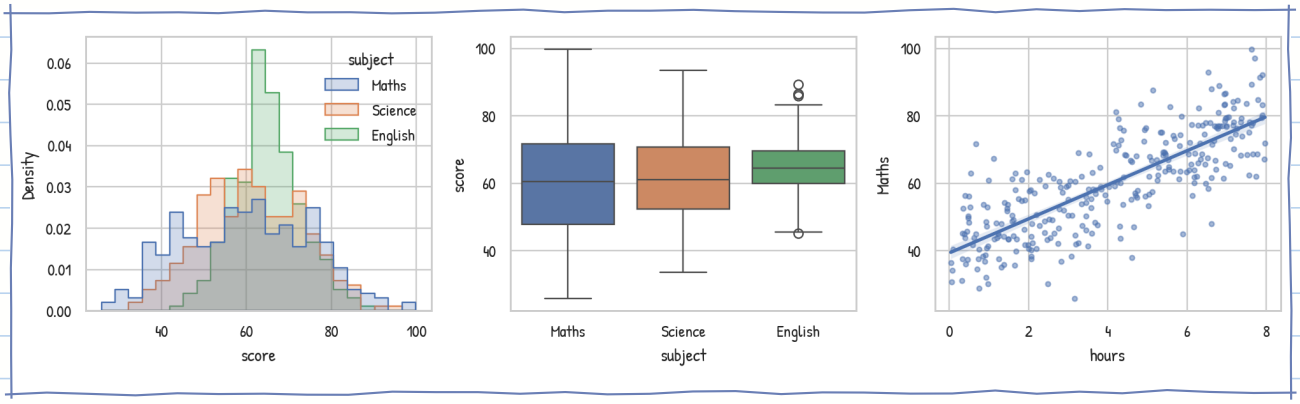
students, finance, EDA, correlation, regression

Project 6: Student performance

Problem: How do subject scores distribute and compare?

Dataset: 300 students, scores in Maths, Science, English and study hours.

```
import numpy as np, pandas as pd, seaborn as sns, matplotlib.pyplot as plt
rng = np.random.default_rng(6)
n = 300
hours = rng.uniform(0, 8, n)
s = pd.DataFrame({'hours': hours,
                  'Maths': 40 + 5 * hours + rng.normal(0, 9, n),
                  'Science': 45 + 4 * hours + rng.normal(0, 8, n),
                  'English': 60 + 1.2 * hours + rng.normal(0, 7, n)}).clip(upper=100)
long = s.melt(id_vars='hours', var_name='subject', value_name='score')   # wide -> tidy
sns.set_theme(style='whitegrid')
fig, axs = plt.subplots(1, 3, figsize=(12, 3.4))
sns.histplot(long, x='score', hue='subject', element='step', stat='density', common_norm=False, ax=axs[0])
sns.boxplot(long, x='subject', y='score', hue='subject', legend=False, ax=axs[1])
sns.regplot(s, x='hours', y='Maths', scatter_kws=dict(s=10, alpha=.5), ax=axs[2])
plt.tight_layout(); plt.show()
```



Interpretation: English scores are higher and tighter; Maths is the most spread out and rises most steeply with study hours.

KEY IDEA: Study hours help Maths most; consider extra support for low-hours students. Use *melt* to turn wide columns into the tidy format Seaborn wants.

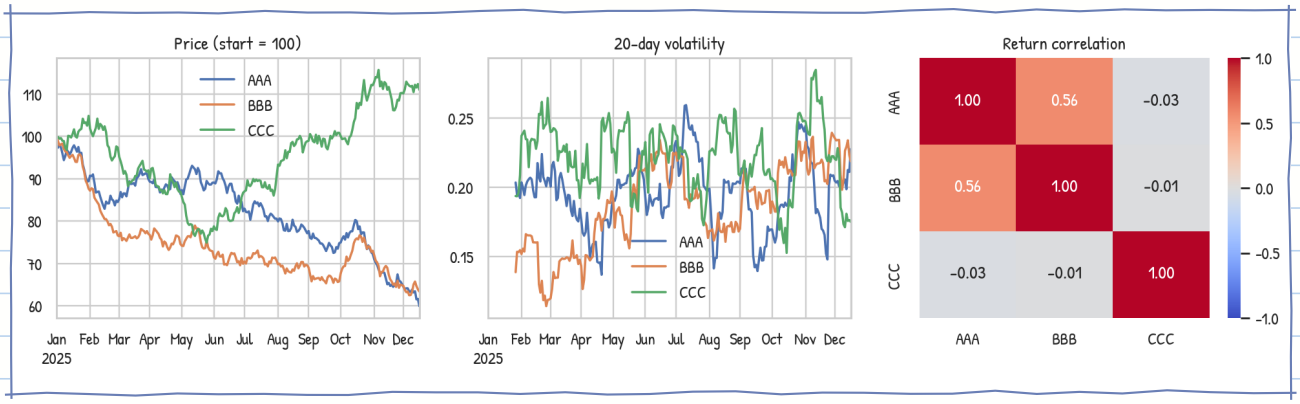
Limitations / next steps: Hours is self-reported in real life; correlation is not proof that hours cause marks. Next: add attendance and test with regression.

Project 7: Financial analysis

Problem: How do prices, returns, volatility and correlation behave?

Dataset: 250 trading days of three synthetic stocks (random walks, one correlated pair).

```
import numpy as np, pandas as pd, seaborn as sns, matplotlib.pyplot as plt
rng = np.random.default_rng(7)
idx = pd.bdate_range('2025-01-01', periods=250)
common = rng.normal(0, .01, 250)
r = pd.DataFrame({'AAA': common + rng.normal(.0004, .008, 250),
                  'BBB': common + rng.normal(.0002, .008, 250),
                  'CCC': rng.normal(.0003, .015, 250)}, index=idx)
price = 100 * (1 + r).cumprod()
vol = r.rolling(20).std() * np.sqrt(252)                    # annualised volatility
sns.set_theme(style='whitegrid')
fig, axs = plt.subplots(1, 3, figsize=(12, 3.4))
price.plot(ax=axs[0], title='Price (start = 100)'); vol.plot(ax=axs[1], title='20-day volatility')
sns.heatmap(r.corr(), annot=True, fmt='.2f', cmap='coolwarm', vmin=-1, vmax=1, ax=axs[2])
axs[2].set_title('Return correlation'); plt.tight_layout(); plt.show()
```



Interpretation: AAA and BBB move together (high correlation); CCC is independent and visibly more volatile.

> **KEY IDEA:** Diversification helps only when assets are not strongly correlated: pairing AAA with CCC reduces risk more than AAA with BBB.

Limitations / next steps: Synthetic, no transaction costs; past volatility does not predict the future. This is education, not investment advice. Next: drawdown, Sharpe ratio.

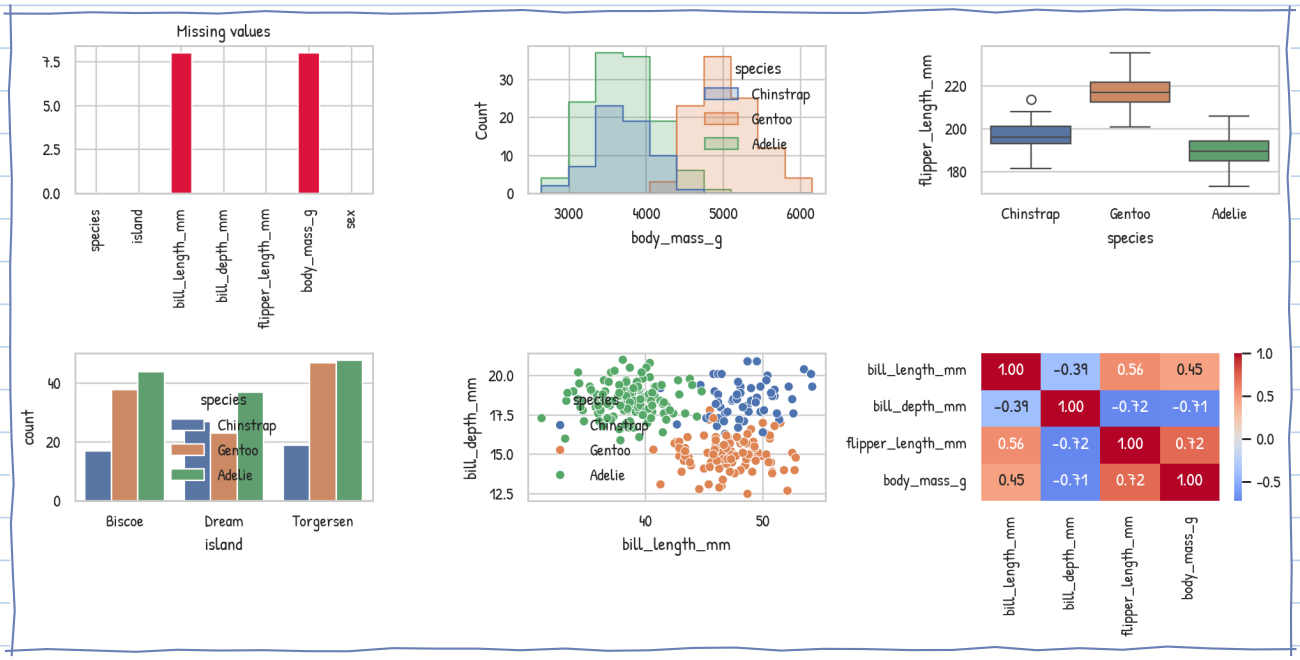
Project 8: End-to-end EDA notebook

Problem: Understand a new dataset from scratch before modelling.

Dataset: Synthetic penguins (300 rows, 7 columns, 16 missing values).

```
import numpy as np, seaborn as sns, matplotlib.pyplot as plt
from sample_data import make_penguins
df = make_penguins()
print(df.shape); print(df.dtypes.to_string()); print(df.isna().sum().to_string())   # 1 structure + missing
print(df.describe().round(1).to_string())                                          # 2 summary stats
```

```
sns.set_theme(style='whitegrid')
fig, axs = plt.subplots(2, 3, figsize=(12, 6))
df.isna().sum().plot.bar(ax=axs[0, 0], color='crimson', title='Missing values')
sns.histplot(df, x='body_mass_g', hue='species', element='step', ax=axs[0, 1])
sns.boxplot(df, x='species', y='flipper_length_mm', hue='species', legend=False, ax=axs[0, 2])
sns.countplot(df, x='island', hue='species', ax=axs[1, 0])
sns.scatterplot(df, x='bill_length_mm', y='bill_depth_mm', hue='species', ax=axs[1, 1])
sns.heatmap(df.corr(numeric_only=True), annot=True, fmt='.2f', cmap='coolwarm', center=0, ax=axs[1, 2])
plt.tight_layout(); plt.show()
```



Interpretation: Missing values are few and limited to two columns; species separate cleanly in mass, flipper and bill measurements; correlations mix a species effect with a within-species effect.

KEY IDEA: EDA order: structure -> missing -> single variables -> groups -> pairs -> correlations. Decide cleaning (drop/impute) only after looking.

Limitations / next steps: Descriptive only. Next: impute or drop NaNs explicitly and test a classifier for species.

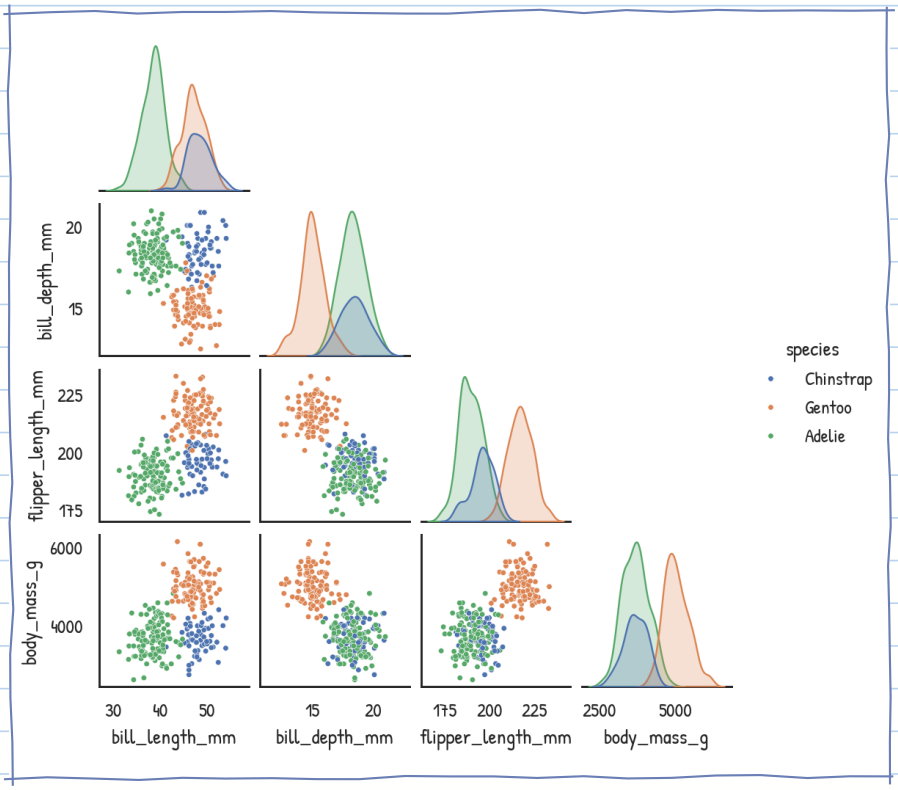
Project 9: Correlation analysis: heatmap and pairplot

Problem: Which numeric variables move together, and is it linear?

Dataset: Same penguin data, four numeric columns.

```
import seaborn as sns, matplotlib.pyplot as plt
from sample_data import make_penguins
df = make_penguins().dropna()
cols = ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']
sns.set_theme(style='white')
fig, ax = plt.subplots(figsize=(4.6, 3.6))
sns.heatmap(df[cols].corr(method='spearman'), annot=True, fmt='.2f', cmap='vlag', center=0, ax=ax)
plt.tight_layout()
```

```
g = sns.pairplot(df, vars=cols, hue='species', corner=True, height=1.5, plot_kws=dict(s=10))
plt.show()
```



Interpretation: Flipper length and body mass correlate strongly. The pairplot shows these links come mostly from three separate species clusters; inside each species the relationships are weak.

KEY IDEA: Always pair a correlation number with its scatter plot: pooled correlation can hide or reverse within-group trends (Simpson's paradox).

Limitations / next steps: Pairwise deletion of NaNs; correlation is not causation. Next: partial correlation controlling for species.

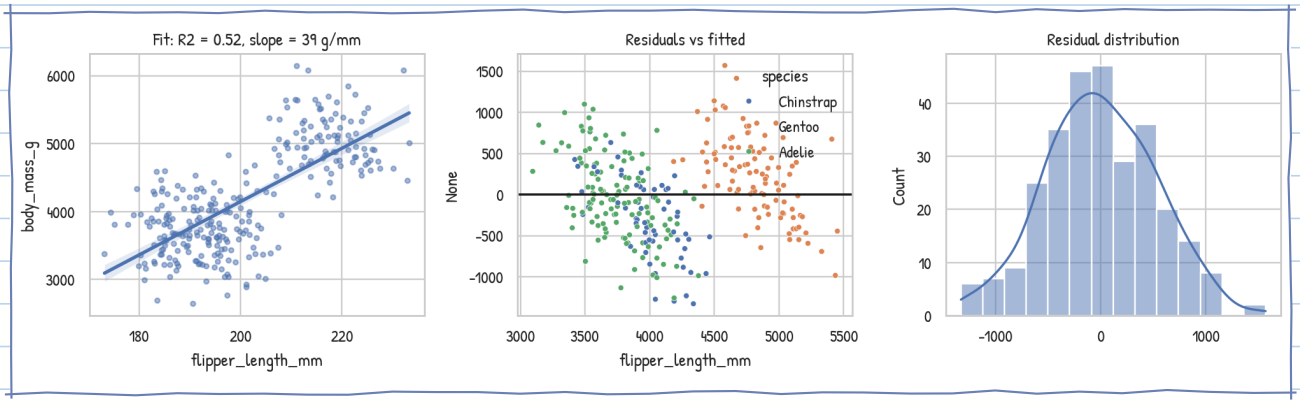
Project 10: Regression visualization with residual diagnostics

Problem: Model body mass from flipper length and judge the fit.

Dataset: Penguin data, complete cases.

```
import numpy as np, seaborn as sns, matplotlib.pyplot as plt
from sample_data import make_penguins
d = make_penguins().dropna()
b1, b0 = np.polyfit(d.flipper_length_mm, d.body_mass_g, 1)
fit = b0 + b1 * d.flipper_length_mm
res = d.body_mass_g - fit
r2 = 1 - (res ** 2).sum() / ((d.body_mass_g - d.body_mass_g.mean()) ** 2).sum()
sns.set_theme(style='whitegrid')
fig, axs = plt.subplots(1, 3, figsize=(12, 3.5))
sns.regplot(d, x='flipper_length_mm', y='body_mass_g', scatter_kws=dict(s=10, alpha=.5), ax=axs[0])
axs[0].set_title(f'Fit: R2 = {r2:.2f}, slope = {b1:.0f} g/mm')
```

```
sns.scatterplot(x=fit, y=res, hue=d.species, s=14, ax=axs[1])
axs[1].axhline(0, color='k'); axs[1].set_title('Residuals vs fitted')
sns.histplot(res, kde=True, ax=axs[2]); axs[2].set_title('Residual distribution')
plt.tight_layout(); plt.show()
```



Interpretation: The line fits well overall (high R2), but residuals cluster by species: the model ignores species, and the band shows uncertainty of the line, not of individual penguins.

KEY IDEA: A high R2 does not prove a good model; the residual plot shows what is missing. Add species as a predictor.

Limitations / next steps: Linear, single predictor, no train/test split. Next: multiple regression, cross-validation.

P8

Matplotlib versus Seaborn

when to use which, and how to combine them

11

Matplotlib vs Seaborn

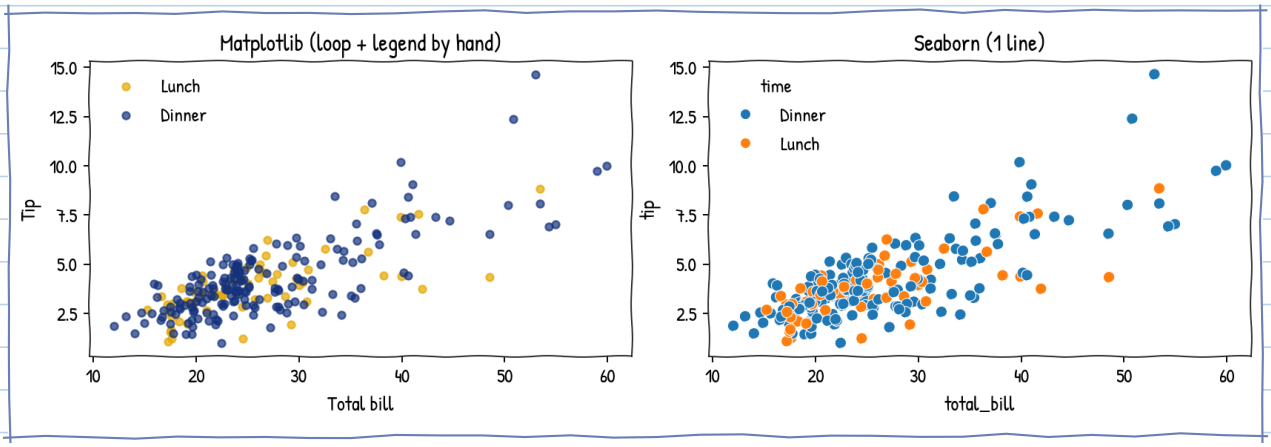
same data, two libraries

Aspect	Matplotlib	Seaborn
Purpose / philosophy	general drawing toolkit; you build every part	statistical graphics; describe what to show, it decides how
Ease of use	more lines, more decisions	one call for complex plots
Customization	unlimited, every artist	good via themes; deep tweaks need Matplotlib
Statistical plotting	manual (you compute CI, KDE)	built in: CI, KDE, regression, bootstrap
DataFrame integration	pass columns yourself	*data=df, x='col'*, groupby built in
Default aesthetics	plain	polished themes and palettes
Complex subplot layouts	GridSpec, mosaic, full control	FacetGrid / PairGrid; axes-level can sit in Matplotlib layouts
Learning curve	steeper at first, then flat	gentle, but needs tidy data + Matplotlib for tweaks
Typical use	papers, custom dashboards, anything unusual	EDA, group comparisons, distributions
Performance	fast for lines/images; fine to ~100k points	slower (aggregation + bootstrap); sample big data

11.1 The same chart in both libraries

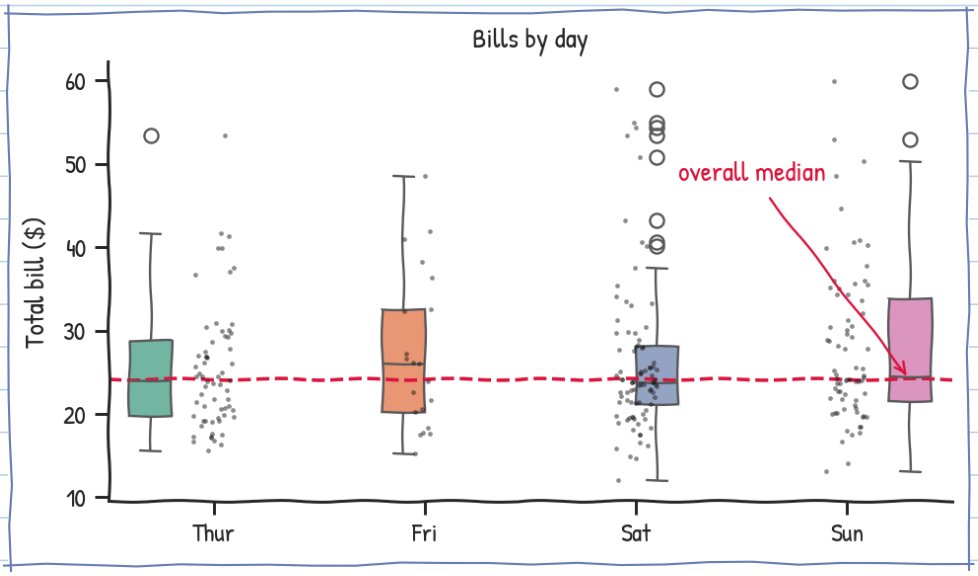
```
import numpy as np, matplotlib.pyplot as plt, seaborn as sns
from sample_data import make_tips
tips = make_tips()
fig, (a, b) = plt.subplots(1, 2, figsize=(10, 3.4))
# --- Matplotlib: you do the grouping, the colours and the legend
for t, c in zip(['Lunch', 'Dinner'], ['#e6a800', '#14327d']):
    s = tips[tips.time == t]
```

```
    a.scatter(s.total_bill, s.tip, color=c, s=18, alpha=.7, label=t)
a.set_xlabel('Total bill'); a.set_ylabel('Tip'); a.legend(); a.set_title('Matplotlib (loop + legend by hand)')
# --- Seaborn: one call
sns.scatterplot(data=tips, x='total_bill', y='tip', hue='time', ax=b); b.set_title('Seaborn (1 line)')
plt.tight_layout(); plt.show()
```



11.2 Seaborn for the statistics, Matplotlib for the finish

```
import matplotlib.pyplot as plt, seaborn as sns
from sample_data import make_tips
tips = make_tips()
sns.set_theme(style='ticks')
fig, ax = plt.subplots(figsize=(6.5, 3.4))
sns.boxplot(data=tips, x='day', y='total_bill', hue='day', legend=False, palette='Set2', ax=ax)  # statistics
sns.stripplot(data=tips, x='day', y='total_bill', color='k', size=2, alpha=.5, ax=ax)
ax.axhline(tips.total_bill.median(), color='crimson', ls='--')                  # Matplotlib touches
ax.annotate('overall median', xy=(3.3, tips.total_bill.median()), xytext=(2.2, 48),
            arrowprops=dict(arrowstyle='->', color='crimson'), color='crimson')
ax.set(xlabel='', ylabel='Total bill ($)', title='Bills by day'); sns.despine(ax=ax)
fig.savefig('bills.png', dpi=300, bbox_inches='tight'); plt.show()
```



KEY IDEA: ***Use Seaborn*** for fast statistical exploration of DataFrames. ***Use Matplotlib*** for exact layout, unusual charts and final polish. In practice you use ***both in the same figure***.

P9 Common Mistakes and Debugging

problem, why, fix, expected result

12 Debugging Visualizations

18 frequent errors

1. Forgetting an import

Why: Python only knows names you import.

```
# WRONG
plt.plot([1, 2, 3])          # NameError: name 'plt' is not defined
# RIGHT
import matplotlib.pyplot as plt
plt.plot([1, 2, 3])
```

Result: the chart appears.

2. Mismatched x and y lengths

Why: every x needs exactly one y.

```
# WRONG
plt.plot([1, 2, 3], [4, 5])   # ValueError: x and y must have same first dimension
# RIGHT
plt.plot([1, 2, 3], [4, 5, 6])       # check len(x) == len(y)
```

Result: a 3-point line.

3. Wrong column name

Why: names are case- and space-sensitive.

```
# WRONG
sns.scatterplot(data=tips, x='Total_Bill', y='tip')   # ValueError: could not interpret value
# RIGHT
print(tips.columns.tolist())
sns.scatterplot(data=tips, x='total_bill', y='tip')
```

Result: scatter plot of the right column.

4. Numbers stored as text

Why: a column of strings is plotted as categories (equal spacing, wrong order).

```
# WRONG
df['price'] = ['10', '9', '100']; plt.plot(df['price'])     # strange axis
# RIGHT
df['price'] = pd.to_numeric(df['price'], errors='coerce')
```

Result: a numeric axis; bad entries become NaN you can inspect.

5. Unexpected missing values

Why: NaNs create gaps, empty bars or dropped rows silently.

```
# WRONG
plt.plot(s)            # gap, no warning
# RIGHT
print(s.isna().sum()); s = s.interpolate()       # or dropna(), and SAY which
```

Result: a continuous line with a documented choice.

6. Bar chart vs histogram confusion

Why: `bar` plots one bar per supplied value; `hist` bins raw data.

```
# WRONG
plt.bar(range(len(marks)), marks)      # one bar per student - not a distribution
# RIGHT
plt.hist(marks, bins=15)
```

Result: bars show how many students fall in each range.

7. Overlapping tick labels

Why: long labels collide on a horizontal axis.

```
# WRONG
plt.bar(long_names, vals)        # unreadable
# RIGHT
plt.barh(long_names, vals)        # or plt.xticks(rotation=45, ha='right')
plt.tight_layout()
```

Result: readable labels.

8. Subplot indexing

Why: `plt.subplot` is 1-based; `axs[r, c]` is 0-based; 1-row grids are 1-D.

```
# WRONG
fig, axs = plt.subplots(1, 3); axs[0, 1].plot(x)    # IndexError
# RIGHT
fig, axs = plt.subplots(1, 3); axs[1].plot(x)
# or: plt.subplots(2, 2, squeeze=False)
```

Result: plot lands in the second panel.

9. figure() vs subplots()

Why: `subplots` inside a loop makes a new figure each time.

```
# WRONG
for k in range(3):
    fig, ax = plt.subplots(); ax.plot(data[k])      # 3 separate figures
```

```
# RIGHT
fig, axs = plt.subplots(1, 3)
for k, ax in enumerate(axs): ax.plot(data[k])
```

Result: one figure with three panels.

10. Misplaced or missing legend

Why: no `label=` means no legend entry; legends can cover data.

```
# WRONG
plt.plot(x, y); plt.legend()       # 'No artists with labels found'
# RIGHT
plt.plot(x, y, label='sales'); plt.legend(loc='upper left', bbox_to_anchor=(1, 1))
```

Result: legend appears outside the data.

11. Misusing hue

Why: `hue` needs a column name; numeric hue gives a continuous colour scale.

```
# WRONG
sns.scatterplot(data=tips, x='total_bill', y='tip', hue='size')   # sequential colours, not groups
# RIGHT
tips['size'] = tips['size'].astype(str)    # or palette='tab10' with hue_order
sns.scatterplot(data=tips, x='total_bill', y='tip', hue='size')
```

Result: discrete groups with distinct colours.

12. Misleading colour scales

Why: rainbow maps invent boundaries; uncentred diverging maps mislead.

```
# WRONG
sns.heatmap(corr, cmap='jet')
# RIGHT
sns.heatmap(corr, cmap='coolwarm', center=0, vmin=-1, vmax=1)
```

Result: 0 is neutral, +/- are symmetric.

13. Reading correlation as causation

Why: r only measures co-movement; confounders, reverse causation and chance exist.

```
# WRONG
# 'ice-cream sales correlate with drownings, so ice-cream causes drowning'
# RIGHT
# say: associated; look for a third factor (hot weather); test with an experiment
```

Result: a claim the data actually supports.

14. Axes-level vs figure-level confusion

Why: `displot`/`catplot`/`relplot` make their own figure and ignore `ax=`.

```
# WRONG
fig, ax = plt.subplots(); sns.displot(data=tips, x='tip', ax=ax)      # empty ax
# RIGHT
fig, ax = plt.subplots(); sns.histplot(data=tips, x='tip', ax=ax)
# or g = sns.displot(...); g.set_axis_labels(...)
```

Result: the histogram sits inside your Axes.

15. Saving the wrong or a blank figure

Why: `show()` closes the figure; `plt.savefig` saves the current one.

```
# WRONG
plt.plot(x, y); plt.show(); plt.savefig('a.png')     # blank image
# RIGHT
fig, ax = plt.subplots(); ax.plot(x, y)
fig.savefig('a.png', dpi=300, bbox_inches='tight'); plt.show()
```

Result: the file contains your chart.

16. Dates plotted as text

Why: string dates become categories.

```
# WRONG
plt.plot(df['date'], df['v'])       # every label drawn, uneven gaps
# RIGHT
df['date'] = pd.to_datetime(df['date']); plt.plot(df['date'], df['v']); plt.gcf().autofmt_xdate()
```

Result: true time spacing with tidy date ticks.

17. Categorical x treated as numeric

Why: integer codes (1,2,3) draw as a continuous line.

```
# WRONG
plt.plot(df['grade_code'], df['score'])        # implies order and distance
# RIGHT
sns.boxplot(data=df, x='grade', y='score')    # treat as category
```

Result: separate boxes, no false trend.

18. Deprecated or removed API

Why: libraries evolve; old tutorials break.

```
# WRONG
sns.distplot(x)            # removed
plt.style.use('seaborn-whitegrid')    # OSError in Matplotlib >= 3.6
sns.barplot(x='a', y='b', data=df, ci=95)    # ci deprecated
# RIGHT
sns.histplot(x, kde=True)
plt.style.use('seaborn-v0_8-whitegrid')
sns.barplot(data=df, x='a', y='b', errorbar=('ci', 95))
```

Result: no warnings, future-proof code.

PRO TIP: Debug routine: (1) read the LAST line of the traceback, (2) `print(df.shape, df.dtypes, df.isna().sum())`, (3) plot a tiny hand-made sample, (4) check library versions, (5) search the exact error text.

P10 Cheat Sheets & Revision Notes

quick reference for exams and interviews

13 Cheat Sheets

one page per topic

Matplotlib function cheat sheet

Task	pyplot	Axes method
line / scatter / bar	plot scatter bar barh	ax.plot ax.scatter ax.bar
distribution	hist boxplot violinplot	ax.hist ax.boxplot
share / area	pie fill_between stackplot	ax.pie ax.fill_between
2-D	imshow contour contourf hexbin	ax.imshow ax.contourf
titles / labels	title xlabel ylabel	set_title set_xlabel set_ylabel
limits / scale	xlim ylim xscale	set_xlim set_ylim set_yscale
ticks	xticks yticks	set_xticks set_xticklabels tick_params
decor	grid legend text annotate	ax.grid ax.legend ax.annotate
reference	axhline axvline axhspan	same on ax
layout	subplot tight_layout	plt.subplots fig.tight_layout GridSpec
output	savefig show close	fig.savefig

Seaborn function cheat sheet

Goal	Axes-level	Figure-level
relationship	scatterplot lineplot regplot	relplot lmplot
distribution	histplot kdeplot ecdfplot rugplot	displot
categorical	barplot countplot boxplot violinplot stripplot swarmplot boxenplot pointplot	catplot

Goal	Axes-level	Figure-level
matrix	*heatmap*	*clustermap*
multi-variable	-	*pairplot jointplot PairGrid JointGrid FacetGrid*
residuals	*residplot*	-
themes	*set_theme axes_style despine color_palette*	-

Chart-selection guide and distribution / categorical comparison

Question	Chart
compare categories	bar (sorted, zero baseline)
trend over time	line
shape of one variable	histogram / KDE / ECDF
groups' distributions	box / violin / strip + n
two numeric variables	scatter (+ fit)
parts of whole	stacked bar, pie only if <= 5
many variables	pairplot, corr heatmap, facets

Distribution plot	Best for	Caution
histplot	shape, counts, small n	bin choice
kdeplot	smooth comparison	bandwidth, tails past data
ecdfplot	percentiles, group comparison	less familiar
rugplot	show individual points	only an aid

Categorical plot	Shows	Use when
barplot	mean (or other estimator) + CI	few groups, plus raw data
countplot	number of rows	frequencies
boxplot	quartiles + outliers	many groups
violinplot	density shape	shape matters
stripplot / *swarmplot*	every point	small n
boxenplot	many quantiles	large n
pointplot	means + CI joined	ordered categories

Parameters, colours, line styles, markers

Parameter	Values
color / c	'r' 'tab:blue' '#14327d' (r,g,b)
linestyle	'-' '--' '-.' ':'
marker	'o' 's' '^' 'D' '*' 'x' '+'
alpha	0 - 1
lw / ms	line width / marker size
cmap	viridis magma coolwarm Blues tab10
loc	best upper left lower right center
figsize / dpi	(8, 4) / 100-300

Figure & Axes, subplots, Seaborn mappings

Concept	Remember
Figure	whole canvas: suptitle, legend, savefig
Axes	one plot: set_title, plot, legend
Axis	x or y number line with ticks
subplots(r, c)	axs[row, col], 0-based; sharex, sharey
GridSpec	gs[:, :2] spans; subplot_mosaic('AAB;AAC')
hue	colour by column
style	marker / line style by column
size	marker size by column
row / col	facets (figure-level only)

Heatmap, correlation, saving

- sns.heatmap(df.corr(numeric_only=True), annot=True, fmt='.2f', cmap='coolwarm', center=0, vmin=-1, vmax=1)
- Pearson = linear; Spearman = rank/monotonic; neither shows causation. Mask the upper triangle with np.triu.
- fig.savefig('a.png', dpi=300, bbox_inches='tight') - PNG slides, PDF/SVG papers; save BEFORE show().

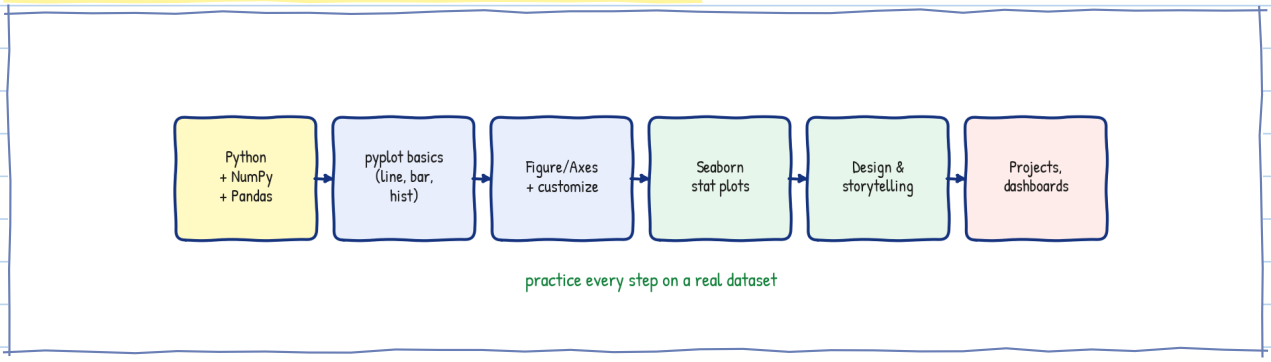
Essential imports and common errors

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
import matplotlib.dates as mdates
from matplotlib.ticker import FuncFormatter
```

Error	Fix
NameError: plt	import it
x and y length mismatch	len(x) == len(y)
blank saved image	savefig before show
wrong column	df.columns
overlapping labels	rotate, barh, tight_layout
style OSError	seaborn-v0_8-...
ci= warning	use errorbar=
displot ignores ax	use histplot

Learning roadmap: beginner to advanced



P11 Practice and Interview Preparation

exercises, solutions, 30+ questions

14 Exercises

attempt first - solutions follow

Beginner

1. Plot a line chart of `[2, 4, 3, 6, 8]` against days 1-5.
2. Create a bar chart of four cities' populations.
3. Add a title, axis labels and a font size of 14 to any chart.
4. Draw a histogram of 200 random normal numbers with 12 bins.
5. Add a legend to two lines and turn on a light grid.

Intermediate

1. Create a 2 x 2 figure of line, scatter, bar and histogram.
2. Compare tip by *time* using *hue* (Seaborn).
3. Show the distribution and outliers of *total_bill* by *day*.
4. Create a correlation heatmap for the penguin data.
5. Draw a Seaborn boxplot and customise it with Matplotlib (median line, annotation).

Advanced

1. Write a reusable function *compare_groups(df, x, y)* returning a figure with box, violin and count panels.
2. Build a 4-panel EDA report with GridSpec for any DataFrame.
3. Plot a time series with a rolling mean and a shaded uncertainty band.
4. Fit a line to two penguin columns and do residual analysis.
5. Produce a polished, business-ready chart with insight title, source note and exported PDF.

15 Solutions

run them, then compare

Beginner solutions

```
import numpy as np, matplotlib.pyplot as plt
plt.plot([1, 2, 3, 4, 5], [2, 4, 3, 6, 8], 'o-')                    # 1
plt.figure(); plt.bar(['A', 'B', 'C', 'D'], [12, 9, 15, 7])         # 2
plt.title('Cities', fontsize=14); plt.xlabel('City'); plt.ylabel('Pop (m)')   # 3
plt.figure(); plt.hist(np.random.default_rng(0).normal(size=200), bins=12)    # 4
plt.figure(); plt.plot([1, 2, 3], label='a'); plt.plot([3, 2, 1], label='b')  # 5
plt.legend(); plt.grid(alpha=.3)
```

Intermediate solutions

```
import numpy as np, seaborn as sns, matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins
tips, pg = make_tips(), make_penguins()
rng = np.random.default_rng(0); x = rng.normal(size=50)
fig, axs = plt.subplots(2, 2)                                        # 6
axs[0, 0].plot(x.cumsum()); axs[0, 1].scatter(x, x + rng.normal(size=50))
axs[1, 0].bar(list('abc'), [3, 5, 2]); axs[1, 1].hist(x)
plt.figure(); sns.boxplot(data=tips, x='time', y='tip', hue='time')  # 7
plt.figure(); sns.boxplot(data=tips, x='day', y='total_bill')        # 8
plt.figure(); sns.heatmap(pg.corr(numeric_only=True), annot=True)    # 9
plt.figure(); ax = sns.boxplot(data=tips, x='day', y='tip')          # 10
ax.axhline(tips.tip.median(), color='r'); ax.annotate('median', (0, tips.tip.median()))
```

Advanced solutions

```
import numpy as np, pandas as pd, seaborn as sns, matplotlib.pyplot as plt
from sample_data import make_tips, make_penguins

def compare_groups(df, x, y):                                        # 11
    fig, axs = plt.subplots(1, 3, figsize=(11, 3.2))
    sns.boxplot(df, x=x, y=y, hue=x, legend=False, ax=axs[0])
    sns.violinplot(df, x=x, y=y, hue=x, legend=False, cut=0, ax=axs[1])
    sns.countplot(df, x=x, hue=x, legend=False, ax=axs[2])
    fig.tight_layout(); return fig

def eda_report(df):                                                  # 12
```

```
    num = df.select_dtypes('number').columns[:3]
    fig = plt.figure(figsize=(9, 5), layout='constrained'); gs = fig.add_gridspec(2, 3)
    for i, c in enumerate(num): sns.histplot(df[c], kde=True, ax=fig.add_subplot(gs[0, i]))
    sns.heatmap(df.corr(numeric_only=True), annot=True, ax=fig.add_subplot(gs[1, :2]))
    df.isna().sum().plot.barh(ax=fig.add_subplot(gs[1, 2]), title='missing')
    return fig

rng = np.random.default_rng(1)                                          # 13
s = pd.Series(rng.normal(size=200).cumsum(), index=pd.date_range('2025-01-01', periods=200))
roll = s.rolling(20); fig, ax = plt.subplots()
ax.plot(s, color='lightgray'); ax.plot(roll.mean(), color='navy')
ax.fill_between(s.index, roll.mean() - roll.std(), roll.mean() + roll.std(), alpha=.2)
d = make_penguins().dropna()                                            # 14
b = np.polyfit(d.flipper_length_mm, d.body_mass_g, 1)
resid = d.body_mass_g - np.polyval(b, d.flipper_length_mm)
sns.residplot(data=d, x='flipper_length_mm', y='body_mass_g')
compare_groups(make_tips(), 'day', 'tip'); eda_report(d)
fig, ax = plt.subplots(figsize=(7, 3.5))                                # 15
ax.plot(s.index, s, color='#14327d'); ax.set_title('Index drifted upward after March')
ax.set_ylabel('Index value'); ax.spines[['top', 'right']].set_visible(False)
fig.text(.01, .01, 'Source: synthetic data', fontsize=8); fig.savefig('report.pdf')
```

16 Interview Questions

35 questions with concise answers

Q1. What is Matplotlib?
A: Python's foundational plotting library; *pyplot* is its state-machine interface.

Q2. Seaborn vs Matplotlib?
A: Seaborn is high-level statistical plotting built on Matplotlib; Matplotlib gives full control.

Q3. Figure vs Axes vs Axis?
A: Figure = canvas; Axes = one plot; Axis = x or y number line with ticks.

Q4. *plt.subplot* vs *plt.subplots*?
A: *subplot* adds one Axes (1-based index); *subplots* creates figure and all Axes (0-based array).

Q5. Why use the object-oriented API?
A: Explicit Axes handles avoid ambiguity with several plots and make reusable functions.

Q6. How do you save a figure?
A: *fig.savefig('a.png', dpi=300, bbox_inches='tight')* before *show()*.

Q7. PNG vs SVG vs PDF?
A: PNG raster for web/slides; SVG/PDF vector for scalable print and papers.

Q8. When use a histogram vs bar chart?
A: Histogram bins a continuous variable; bar chart compares categories.

Q9. When is a pie chart acceptable?
A: Two to five parts of a whole; otherwise sorted bars.

Q10. What does a box plot show?
A: Median, Q1, Q3, whiskers (1.5 x IQR) and outliers.

Q11. Box vs violin?
A: Violin adds a KDE density shape; box is simpler and shows outliers explicitly.

Q12. barplot vs countplot?
A: barplot aggregates a y variable (mean + CI); countplot counts rows.

Q13. What are the error bars in Seaborn by default?
A: 95% bootstrap confidence interval (*errorbar=('ci', 95)*).

Q14. Figure-level vs axes-level?
A: Figure-level (*relplot*, *displot*, *catplot*) own the figure and support row/col; axes-level draw on a given *ax*.

Q15. What does *hue* do?
A: Maps a column to colour and creates a legend.

Q16. When is KDE misleading?
A: Small samples, bounded or discrete data, extreme bandwidth; tails extend past the data.

Q17. What is an ECDF?
A: Proportion of data <= x; shows percentiles without bins.

Q18. Pearson vs Spearman?
A: Pearson linear; Spearman rank-based, robust to outliers and monotonic non-linearity.

Q19. Does correlation imply causation?
A: No - confounders, reverse causation and chance can create it.

Q20. How read a residual plot?

A: Random scatter around 0 is good; curves or funnels reveal non-linearity or non-constant variance.

Q21. What is overplotting and fixes?

A: Points hide each other; use alpha, smaller markers, hexbin or 2-D histograms.

Q22. Why avoid dual-axis charts?

A: Arbitrary scaling can fake correlation; use stacked panels.

Q23. Why must bars start at zero?

A: Bar length encodes value; truncation exaggerates differences.

Q24. Log scale: when and risk?

A: Multi-order-of-magnitude or exponential data; cannot show 0/negatives and is misread - label it.

Q25. How handle missing data in plots?

A: Detect with *isna()*, choose drop/impute/interpolate, and state it.

Q26. How do you plot dates correctly?

A: *pd.to_datetime*, then date locators/formatters and *autofmt_xdate()*.

Q27. Tick labels overlap - fix?

A: Rotate, horizontal bars, abbreviate, *tight_layout*.

Q28. How make charts colour-blind friendly?

A: Use *viridis*/*colorblind*, add markers/line styles, avoid red-green only.

Q29. What is *tight_layout* vs constrained layout?

A: Both prevent overlap; constrained is newer and handles colorbars/legends better.

Q30. How create a complex layout?

A: *GridSpec* or *subplot_mosaic*.

Q31. How do you plot 1 million points?

A: Aggregate or sample, hexbin/hist2d, rasterize, avoid per-point objects.

Q32. Tidy vs wide data?

A: Tidy: one observation per row; Seaborn expects it; reshape with *melt*.

Q33. How to choose a chart?

A: Start from the question: compare, distribute, relate, trend, composition.

Q34. Descriptive chart vs significance?

A: Charts describe; significance needs a statistical test and effect size.

Q35. Name two deprecated Seaborn/Matplotlib patterns.

A: *sns.distplot*, *ci=* argument, *seaborn-whitegrid* style (now *seaborn-v0_8-...*).

P12 Capstone Projects

three complete projects: beginner, intermediate, advanced

Note: the source brief ended at "Part XII – Finish with three complete capstone projects: Beginner" (the text was cut off), so the three capstones below follow the beginner / intermediate / advanced pattern used elsewhere.

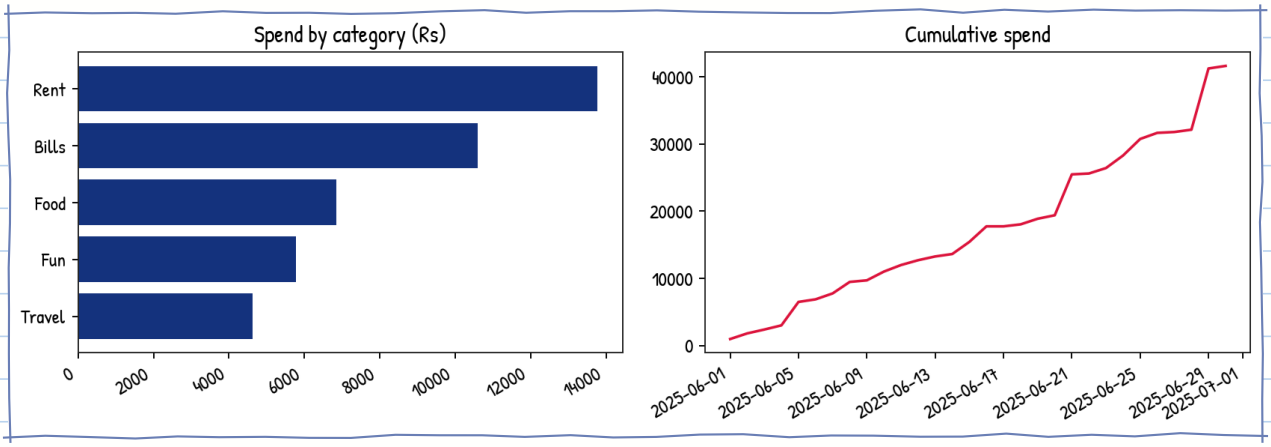
17 Capstones

apply everything

Capstone 1 (Beginner): Personal expense tracker

Goal: visualise a month of spending. **Skills:** bar, pie, line, labels.

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt
rng = np.random.default_rng(10)
days = pd.date_range('2025-06-01', periods=30)
cats = ['Food', 'Travel', 'Rent', 'Fun', 'Bills']
df = pd.DataFrame({'day': rng.choice(days, 90), 'cat': rng.choice(cats, 90, p=[.35, .2, .05, .2, .2])})
typical = {'Food': 250, 'Travel': 180, 'Rent': 9000, 'Fun': 400, 'Bills': 600}
df['amt'] = df['cat'].map(typical) * rng.uniform(.5, 1.5, 90)
by_cat = df.groupby('cat')['amt'].sum().sort_values()
daily = df.groupby('day')['amt'].sum().reindex(days, fill_value=0)
fig, axs = plt.subplots(1, 2, figsize=(10, 3.5))
axs[0].barh(by_cat.index, by_cat.values, color='#14327d'); axs[0].set_title('Spend by category (Rs)')
axs[1].plot(daily.index, daily.cumsum(), color='crimson'); axs[1].set_title('Cumulative spend')
fig.autofmt_xdate(); plt.tight_layout(); plt.show()
```

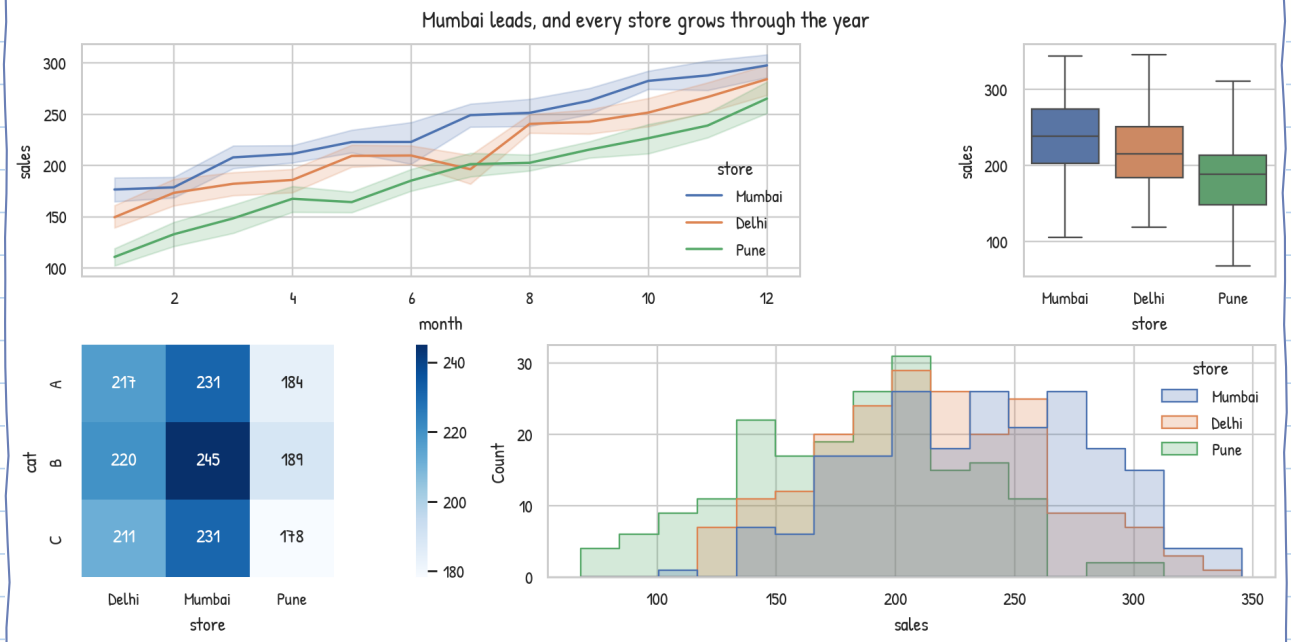


KEY IDEA: Insight: one category (rent) dominates; the cumulative line shows pace against a budget. Extension: add a budget line with `axhline`.

Capstone 2 (Intermediate): Store performance dashboard

Goal: a 4-panel dashboard with a shared theme. **Skills:** Seaborn grouping, GridSpec, annotations, export.

```
import numpy as np, pandas as pd, seaborn as sns, matplotlib.pyplot as plt
rng = np.random.default_rng(11)
n = 600
d = pd.DataFrame({'store': rng.choice(['Mumbai', 'Pune', 'Delhi'], n), 'cat': rng.choice(['A', 'B', 'C'], n),
                  'month': rng.integers(1, 13, n)})
bonus = d['store'].map({'Mumbai': 60, 'Pune': 10, 'Delhi': 40})
d['sales'] = 100 + 12 * d['month'] + bonus + rng.normal(0, 25, n)
sns.set_theme(style='whitegrid')
fig = plt.figure(figsize=(11, 5.5), layout='constrained'); gs = fig.add_gridspec(2, 3)
sns.lineplot(d, x='month', y='sales', hue='store', ax=fig.add_subplot(gs[0, :2]))
sns.boxplot(d, x='store', y='sales', hue='store', legend=False, ax=fig.add_subplot(gs[0, 2]))
pv = d.pivot_table('sales', 'cat', 'store')
sns.heatmap(pv, annot=True, fmt='.0f', cmap='Blues', ax=fig.add_subplot(gs[1, 0]))
sns.histplot(d, x='sales', hue='store', element='step', ax=fig.add_subplot(gs[1, 1:]))
fig.suptitle('Mumbai leads, and every store grows through the year', fontweight='bold')
fig.savefig('dashboard.pdf'); plt.show()
```



KEY IDEA: Check: title states the insight, shared palette, units and source added, exported as PDF. Extension: add a year-on-year comparison.

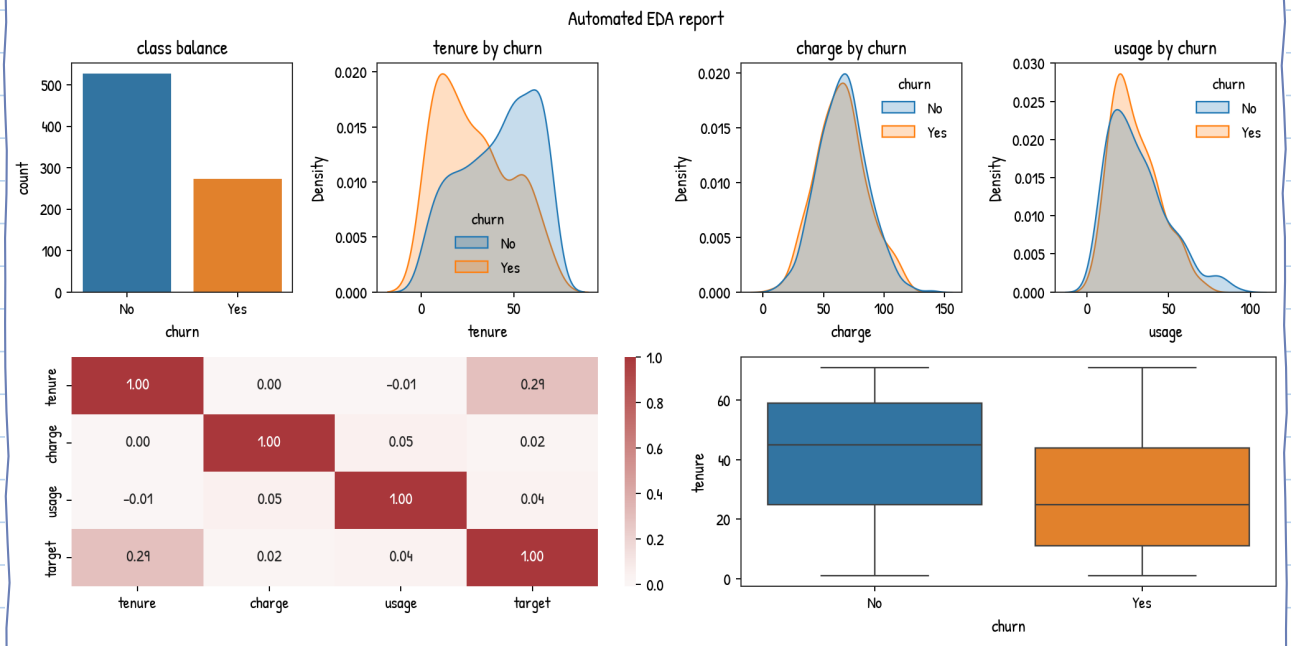
Capstone 3 (Advanced): Churn EDA report with a reusable pipeline

Goal: a function that takes any labelled DataFrame and builds a multi-panel EDA report with correlation, distributions by class and residual-style checks. **Skills:** reusable code, GridSpec, statistics, honest limitations.

```
import numpy as np, pandas as pd, seaborn as sns, matplotlib.pyplot as plt

def eda_report(df, target, max_num=3):
    num = df.select_dtypes('number').columns.drop(target, errors='ignore')[:max_num]
    fig = plt.figure(figsize=(12, 6), layout='constrained'); gs = fig.add_gridspec(2, max_num + 1)
    ax0 = fig.add_subplot(gs[0, 0])
    sns.countplot(df, x=target, hue=target, legend=False, ax=ax0); ax0.set_title('class balance')
    for i, c in enumerate(num, 1):
        ax = fig.add_subplot(gs[0, i]); sns.kdeplot(df, x=c, hue=target, common_norm=False, fill=True, ax=ax)
        ax.set_title(f'{c} by {target}')
    y = (df[target] == df[target].unique()[0]).astype(int)
    cm = df[list(num)].assign(target=y).corr()
    sns.heatmap(cm, annot=True, fmt='.2f', cmap='vlag', center=0, ax=fig.add_subplot(gs[1, :2]))
    sns.boxplot(df, x=target, y=num[0], hue=target, legend=False, ax=fig.add_subplot(gs[1, 2:]))
    fig.suptitle('Automated EDA report', fontweight='bold'); return fig

rng = np.random.default_rng(12); n = 800
df = pd.DataFrame({'tenure': rng.integers(1, 72, n), 'charge': rng.normal(65, 20, n)})
df['usage'] = rng.gamma(3, 10, n)
df['churn'] = np.where(rng.random(n) < (0.6 - 0.007 * df.tenure).clip(.03, .9), 'Yes', 'No')
eda_report(df, 'churn'); plt.show()
```



EXAM TIP: Limitations to state: association not causation; no train/test split; imbalance affects metrics; synthetic data. Next steps: baseline model, cross-validation, calibrated risk scores.

End of notes. Keep practising: change one parameter at a time, predict the result, then run it.